```
============================================================
FILE: backend\server.py
============================================================

from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from typing import List, Optional, Dict, Any
import os
import re
import json
import secrets
from datetime import datetime, timezone
from uuid import uuid4
from dotenv import load_dotenv
from openai import AzureOpenAI
from motor.motor_asyncio import AsyncIOMotorClient

load_dotenv()

app = FastAPI(title="SketchSQL API")

# MongoDB (for shared-diagram snapshots)
_mongo_client = AsyncIOMotorClient(os.environ["MONGO_URL"])
_db = _mongo_client[os.environ["DB_NAME"]]
shares_coll = _db["shares"]

app.add_middleware(
    CORSMiddleware,
```

```
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Azure OpenAI client (lazy init)
_azure_client = None

def get_client():
    global _azure_client
    if _azure_client is None:
        _azure_client = AzureOpenAI(
            azure_endpoint=os.environ.get("AZURE_OPENAI_ENDPOINT", ""),
            api_key=os.environ.get("AZURE_OPENAI_API_KEY", ""),
            api_version=os.environ.get("AZURE_OPENAI_API_VERSION", "2024-12-
01-preview"),
        )
    return _azure_client

DEPLOYMENT = lambda: os.environ.get("AZURE_OPENAI_DEPLOYMENT", "gpt-
5.4")

# ——— Pydantic Models
————————————————————————————————————————
————

class Column(BaseModel):
    id: str
    name: str
```

```
    type: str
    primaryKey: bool = False
    autoIncrement: bool = False
    nullable: bool = True
    unique: bool = False
    defaultValue: str = ""

class Table(BaseModel):
    id: str
    name: str
    color: str = "gray"
    columns: List[Column] = []
    position: Optional[Dict[str, float]] = None

class Relationship(BaseModel):
    id: str
    sourceTableId: str
    sourceColumnId: str
    targetTableId: str
    targetColumnId: str
    type: str = "one-to-many"
    onDelete: str = "RESTRICT"
    onUpdate: str = "RESTRICT"
    label: str = ""

class Diagram(BaseModel):
    id: Optional[str] = None
    name: str = "Untitled"
    dialect: str = "mysql"
```

```
    tables: List[Table] = []
    relationships: List[Relationship] = []

class GenerateSqlRequest(BaseModel):
    diagram: Diagram
    dialect: str = "mysql"

class ImportSqlRequest(BaseModel):
    sql: str
    dialect: str = "mysql"

class AiGenerateSchemaRequest(BaseModel):
    prompt: str

class AiAnalyzeSchemaRequest(BaseModel):
    diagram: Diagram
    dialect: str = "mysql"

class ChatMessage(BaseModel):
    role: str
    content: str

class AiChatRequest(BaseModel):
    messages: List[ChatMessage]
    diagram: Diagram
    dialect: str = "mysql"

# ——— SQL Generator ———————————————————————————————————————————————
```

```
def q(name: str, dialect: str) -> str:
    return f"`{name}`" if dialect == "mysql" else f'"{name}"'


def pg_type(col_type: str, auto_increment: bool) -> str:
    t = col_type.upper()
    if auto_increment:
        if "BIGINT" in t:
            return "BIGSERIAL"
        if "INT" in t:
            return "SERIAL"
    if t == "DATETIME":
        return "TIMESTAMP"
    if t.startswith("TINYINT"):
        return "BOOLEAN"
    return col_type


def topological_sort(tables: List[Table], relationships: List[Relationship]) ->
List[Table]:
    tmap = {t.id: t for t in tables}
    deps: Dict[str, set] = {t.id: set() for t in tables}
    for rel in relationships:
        if rel.type != "many-to-many":
            if rel.sourceTableId in deps and rel.targetTableId in tmap:
                deps[rel.sourceTableId].add(rel.targetTableId)
    visited: set = set()
    result: List[Table] = []

    def dfs(tid: str):
```

```
        if tid in visited:
            return
        visited.add(tid)
        for dep in deps.get(tid, set()):
            dfs(dep)
        if tid in tmap:
            result.append(tmap[tid])

    for t in tables:
        dfs(t.id)
    return result


def fmt_default(dv: str) -> str:
    keywords = {"NULL", "CURRENT_TIMESTAMP", "TRUE", "FALSE", "NOW()",
"CURRENT_DATE", "CURRENT_TIME"}
    if dv.upper() in keywords or dv.lstrip("-").isdigit():
        return dv
    return f"'{dv}'"


def generate_mysql(diagram: Diagram) -> str:
    tables, rels = diagram.tables, diagram.relationships
    sorted_tables = topological_sort(tables, rels)
    tmap = {t.id: t for t in tables}
    cmap = {c.id: c for t in tables for c in t.columns}
    parts = ["-- Generated by SketchSQL\n-- Dialect: MySQL\n\nSET
FOREIGN_KEY_CHECKS=0;\n"]

    for table in sorted_tables:
        col_defs, pks = [], []
        for col in table.columns:
```

```
        defn = f"  {q(col.name, 'mysql')} {col.type}"
        if col.autoIncrement and col.primaryKey:
            defn += " AUTO_INCREMENT"
        if not col.nullable or col.primaryKey:
            defn += " NOT NULL"
        if col.unique and not col.primaryKey:
            defn += " UNIQUE"
        if col.defaultValue:
            defn += f" DEFAULT {fmt_default(col.defaultValue)}"
        col_defs.append(defn)
        if col.primaryKey:
            pks.append(col.name)
    if pks:
        col_defs.append(f"  PRIMARY KEY ({', '.join(q(p, 'mysql') for p in pks)})")
    parts.append(
        f"CREATE TABLE {q(table.name, 'mysql')} (\n" +
        ",\n".join(col_defs) +
        "\n) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;\n"
    )

# Many-to-many junction tables
for rel in rels:
    if rel.type != "many-to-many":
        continue
    src, tgt = tmap.get(rel.sourceTableId), tmap.get(rel.targetTableId)
    sc, tc = cmap.get(rel.sourceColumnId), cmap.get(rel.targetColumnId)
    if not (src and tgt and sc and tc):
        continue
    jn = f"{src.name}_{tgt.name}"
```

```
        sf, tf = f"{src.name}_id", f"{tgt.name}_id"
        spk = next((c for c in src.columns if c.primaryKey), None)
        tpk = next((c for c in tgt.columns if c.primaryKey), None)
        st = (spk.type if spk else "INT").split("(")[0]
        tt = (tpk.type if tpk else "INT").split("(")[0]
        od, ou = rel.onDelete or "CASCADE", rel.onUpdate or "CASCADE"
        parts.append(
            f"CREATE TABLE {q(jn, 'mysql')} (\n"
            f"  {q(sf, 'mysql')} {st} NOT NULL,\n"
            f"  {q(tf, 'mysql')} {tt} NOT NULL,\n"
            f"  PRIMARY KEY ({q(sf, 'mysql')}, {q(tf, 'mysql')}),\n"
            f"  FOREIGN KEY ({q(sf, 'mysql')}) REFERENCES {q(src.name, 'mysql')}({q(sc.name, 'mysql')}) ON DELETE {od} ON UPDATE {ou},\n"
            f"  FOREIGN KEY ({q(tf, 'mysql')}) REFERENCES {q(tgt.name, 'mysql')}({q(tc.name, 'mysql')}) ON DELETE {od} ON UPDATE {ou}\n"
            f") ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;\n"
        )

    # FK constraints
    for rel in rels:
        if rel.type == "many-to-many":
            continue
        src, tgt = tmap.get(rel.sourceTableId), tmap.get(rel.targetTableId)
        sc, tc = cmap.get(rel.sourceColumnId), cmap.get(rel.targetColumnId)
        if not (src and tgt and sc and tc):
            continue
        od, ou = rel.onDelete or "RESTRICT", rel.onUpdate or "RESTRICT"
        fk = f"fk_{src.name}_{sc.name}"
        parts.append(
            f"ALTER TABLE {q(src.name, 'mysql')}\n"
```

```
            f"  ADD CONSTRAINT {q(fk, 'mysql')}\n"
            f"  FOREIGN KEY ({q(sc.name, 'mysql')})\n"
            f"  REFERENCES {q(tgt.name, 'mysql')}({q(tc.name, 'mysql')})\n"
            f"  ON DELETE {od} ON UPDATE {ou};\n"
        )

    parts.append("\nSET FOREIGN_KEY_CHECKS=1;")
    return "\n".join(parts)

def generate_postgresql(diagram: Diagram) -> str:
    tables, rels = diagram.tables, diagram.relationships
    sorted_tables = topological_sort(tables, rels)
    tmap = {t.id: t for t in tables}
    cmap = {c.id: c for t in tables for c in t.columns}
    parts = ["-- Generated by SketchSQL\n-- Dialect: PostgreSQL\n"]

    for table in sorted_tables:
        col_defs, pks, uniq = [], [], []
        for col in table.columns:
            pgt = pg_type(col.type, col.autoIncrement and col.primaryKey)
            defn = f"  {q(col.name, 'postgresql')} {pgt}"
            if not col.nullable or col.primaryKey:
                defn += " NOT NULL"
            if col.defaultValue:
                defn += f" DEFAULT {fmt_default(col.defaultValue)}"
            col_defs.append(defn)
            if col.primaryKey:
                pks.append(col.name)
            elif col.unique:
```

```
            uniq.append(col.name)
    if pks:
        col_defs.append(f"  PRIMARY KEY ({', '.join(q(p, 'postgresql') for p in pks)})")
    for u in uniq:
        col_defs.append(f"  UNIQUE ({q(u, 'postgresql')})")
    parts.append(
        f"CREATE TABLE {q(table.name, 'postgresql')} (\n" +
        ",\n".join(col_defs) +
        "\n);\n"
    )

# Many-to-many
for rel in rels:
    if rel.type != "many-to-many":
        continue
    src, tgt = tmap.get(rel.sourceTableId), tmap.get(rel.targetTableId)
    sc, tc = cmap.get(rel.sourceColumnId), cmap.get(rel.targetColumnId)
    if not (src and tgt and sc and tc):
        continue
    jn = f"{src.name}_{tgt.name}"
    sf, tf = f"{src.name}_id", f"{tgt.name}_id"
    spk = next((c for c in src.columns if c.primaryKey), None)
    tpk = next((c for c in tgt.columns if c.primaryKey), None)
    st = pg_type(spk.type if spk else "INT", False)
    tt = pg_type(tpk.type if tpk else "INT", False)
    od, ou = rel.onDelete or "CASCADE", rel.onUpdate or "CASCADE"
    parts.append(
        f"CREATE TABLE {q(jn, 'postgresql')} (\n"
        f"  {q(sf, 'postgresql')} {st} NOT NULL,\n"
```

```
            f"  {q(tf, 'postgresql')} {tt} NOT NULL,\n"
            f"  PRIMARY KEY ({q(sf, 'postgresql')}, {q(tf, 'postgresql')}),\n"
            f"  FOREIGN KEY ({q(sf, 'postgresql')}) REFERENCES {q(src.name, 'postgresql')}({q(sc.name, 'postgresql')}) ON DELETE {od} ON UPDATE {ou},\n"
            f"  FOREIGN KEY ({q(tf, 'postgresql')}) REFERENCES {q(tgt.name, 'postgresql')}({q(tc.name, 'postgresql')}) ON DELETE {od} ON UPDATE {ou}\n"
            f");\n"
        )

    # FK constraints
    for rel in rels:
        if rel.type == "many-to-many":
            continue
        src, tgt = tmap.get(rel.sourceTableId), tmap.get(rel.targetTableId)
        sc, tc = cmap.get(rel.sourceColumnId), cmap.get(rel.targetColumnId)
        if not (src and tgt and sc and tc):
            continue
        od, ou = rel.onDelete or "RESTRICT", rel.onUpdate or "RESTRICT"
        fk = f"fk_{src.name}_{sc.name}"
        parts.append(
            f"ALTER TABLE ONLY {q(src.name, 'postgresql')}\n"
            f"  ADD CONSTRAINT {q(fk, 'postgresql')}\n"
            f"  FOREIGN KEY ({q(sc.name, 'postgresql')})\n"
            f"  REFERENCES {q(tgt.name, 'postgresql')}({q(tc.name, 'postgresql')})\n"
            f"  ON DELETE {od} ON UPDATE {ou};\n"
        )

    return "\n".join(parts)
```

```
# ——— SQL Parser
————————————————————————————————————————————————————————————

def split_by_commas(s: str) -> List[str]:
    parts, depth, cur = [], 0, []
    for ch in s:
        if ch == "(":
            depth += 1
            cur.append(ch)
        elif ch == ")":
            depth -= 1
            cur.append(ch)
        elif ch == "," and depth == 0:
            parts.append("".join(cur).strip())
            cur = []
        else:
            cur.append(ch)
    if cur:
        parts.append("".join(cur).strip())
    return parts


def parse_sql(sql_string: str, dialect: str = "mysql") -> dict:
    tables, rels, pending_fks = [], [], []
    sql_clean = re.sub(r"--[^\n]*", "", sql_string)
    sql_clean = re.sub(r"/\*.*?\*/", "", sql_clean, flags=re.DOTALL)
    pattern = re.compile(

r"CREATE\s+TABLE\s+(?:IF\s+NOT\s+EXISTS\s+)?[`\"\[]?(\w+)[`\"\]]?\s*\((.*?)\)\s*(?:
ENGINE\s*=\s*\w+[^;]*)?;",
```

```
        re.IGNORECASE | re.DOTALL,
    )
    tname_to_id: Dict[str, str] = {}
    ckey_to_id: Dict[tuple, str] = {}

    for m in pattern.finditer(sql_clean):
        tname, tbody = m.group(1), m.group(2)
        tid = f"t_{uuid4().hex[:8]}"
        tname_to_id[tname.lower()] = tid
        columns, pk_names, fks = [], [], []

        for line in split_by_commas(tbody):
            line = line.strip()
            if not line:
                continue
            # PRIMARY KEY constraint
            pk_m = re.match(r"^PRIMARY\s+KEY\s*\(([^)]+)\)", line, re.IGNORECASE)
            if pk_m:
                pk_names.extend(s.strip().strip("`\"") for s in pk_m.group(1).split(","))
                continue
            # FOREIGN KEY
            fk_m = re.match(

r"^(?:CONSTRAINT\s+\w+\s+)?FOREIGN\s+KEY\s*\(([^)]+)\)\s+REFERENCES\s+[`\
"\[]?(\w+)[`\"\]]?\s*\(([^)]+)\)"

r"(?:\s+ON\s+DELETE\s+(CASCADE|SET\s+NULL|RESTRICT|NO\s+ACTION|SET\
s+DEFAULT))?"

r"(?:\s+ON\s+UPDATE\s+(CASCADE|SET\s+NULL|RESTRICT|NO\s+ACTION|SET\
s+DEFAULT))?",
```

```
            line, re.IGNORECASE,
        )
        if fk_m:
            fks.append({
                "src_table": tname,
                "src_col": fk_m.group(1).strip().strip("`\""),
                "ref_table": fk_m.group(2),
                "ref_col": fk_m.group(3).strip().strip("`\""),
                "on_delete": (fk_m.group(4) or "RESTRICT").upper(),
                "on_update": (fk_m.group(5) or "RESTRICT").upper(),
            })
            continue
        # Skip index lines
        if re.match(r"^(?:UNIQUE\s+)?(?:KEY|INDEX)\s+", line, re.IGNORECASE):
            continue
        # Column definition
        col_m = re.match(r"^[`\"\[]?(\w+)[`\"\]]?\s+(\S+(?:\s*\([^)]*\))?)(.*)", line)
        if col_m and not
re.match(r"^(PRIMARY|UNIQUE|KEY|INDEX|CONSTRAINT|CHECK)\b",
col_m.group(1), re.IGNORECASE):
            cname, raw_t, rest = col_m.group(1), col_m.group(2), col_m.group(3)
            ct = raw_t.upper()
            auto_inc = bool(re.search(r"\bAUTO_INCREMENT\b", rest,
re.IGNORECASE))
            if ct in ("SERIAL", "BIGSERIAL"):
                auto_inc = True
                ct = "BIGINT" if ct == "BIGSERIAL" else "INT"
            cid = f"c_{uuid4().hex[:8]}"
            col = {
                "id": cid, "name": cname, "type": ct,
```

```
                "primaryKey": bool(re.search(r"\bPRIMARY\s+KEY\b", rest,
re.IGNORECASE)),
                "autoIncrement": auto_inc,
                "nullable": not bool(re.search(r"\bNOT\s+NULL\b", rest,
re.IGNORECASE)),
                "unique": bool(re.search(r"\bUNIQUE\b", rest, re.IGNORECASE)),
                "defaultValue": "",
            }
            dv_m = re.search(r"\bDEFAULT\s+('([^']*)'|\"([^\"]*)\"|(\S+))", rest,
re.IGNORECASE)
            if dv_m:
                col["defaultValue"] = dv_m.group(2) or dv_m.group(3) or dv_m.group(4)
or ""
            columns.append(col)
            ckey_to_id[(tname.lower(), cname.lower())] = cid

        for pk in pk_names:
            for c in columns:
                if c["name"].lower() == pk.lower():
                    c["primaryKey"] = True
                    c["nullable"] = False

        pending_fks.extend(fks)
        tables.append({"id": tid, "name": tname, "color": "gray", "columns": columns,
"position": {"x": 0, "y": 0}})

    for fk in pending_fks:
        stid = tname_to_id.get(fk["src_table"].lower())
        ttid = tname_to_id.get(fk["ref_table"].lower())
        scid = ckey_to_id.get((fk["src_table"].lower(), fk["src_col"].lower()))
        tcid = ckey_to_id.get((fk["ref_table"].lower(), fk["ref_col"].lower()))
```

```
        if stid and ttid and scid and tcid:
            rels.append({
                "id": f"r_{uuid4().hex[:8]}",
                "sourceTableId": stid, "sourceColumnId": scid,
                "targetTableId": ttid, "targetColumnId": tcid,
                "type": "one-to-many",
                "onDelete": fk["on_delete"], "onUpdate": fk["on_update"], "label": "",
            })

    return {"tables": tables, "relationships": rels}

# ── Routes ──────────────────────────────────────────────────────────────

@app.get("/api/")
async def health():
    return {"message": "SketchSQL API running", "status": "ok"}


@app.post("/api/generate-sql")
async def generate_sql_route(req: GenerateSqlRequest):
    try:
        sql = generate_mysql(req.diagram) if req.dialect == "mysql" else
generate_postgresql(req.diagram)
        return {"sql": sql}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


@app.post("/api/import-sql")
async def import_sql_route(req: ImportSqlRequest):
```

```
    try:
        return {"diagram": parse_sql(req.sql, req.dialect)}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


@app.post("/api/ai/generate-schema")
async def ai_generate_schema(req: AiGenerateSchemaRequest):
    try:
        system_prompt = """You are a database schema designer. The user will describe a system in plain English.
You must respond ONLY with a valid JSON object representing a database schema.
No explanation, no markdown, no backticks. Only raw JSON.

The JSON must follow this exact schema:
{
  "tables": [
    {
      "id": "unique_string_id",
      "name": "TableName",
      "color": "blue|green|purple|orange|red|gray",
      "columns": [
        {
          "id": "unique_col_id",
          "name": "column_name",
          "type": "INT|VARCHAR(255)|TEXT|BOOLEAN|DATE|DATETIME|TIMESTAMP|DECIMAL(10,2)|UUID|BIGINT|FLOAT|JSON",
          "primaryKey": true,
          "autoIncrement": true,
          "nullable": false,
```

```
          "unique": false,
          "defaultValue": ""
        }
      ]
    }
  ],
  "relationships": [
    {
      "id": "unique_rel_id",
      "sourceTableId": "id",
      "sourceColumnId": "id",
      "targetTableId": "id",
      "targetColumnId": "id",
      "type": "one-to-one|one-to-many|many-to-many",
      "onDelete": "CASCADE|SET NULL|RESTRICT",
      "onUpdate": "CASCADE|SET NULL|RESTRICT"
    }
  ]
}"""
        resp = get_client().chat.completions.create(
            model=DEPLOYMENT(),
            messages=[
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": f"Design a database schema for: {req.prompt}"},
            ],
            max_completion_tokens=16384,
        )
        content = resp.choices[0].message.content.strip()
        content = re.sub(r"^```[a-z]*\n?", "", content)
```

```
        content = re.sub(r"\n?```$", "", content)
        diagram = json.loads(content)
        return {"diagram": diagram}
    except json.JSONDecodeError as e:
        raise HTTPException(status_code=422, detail=f"AI returned invalid JSON: {e}")
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


@app.post("/api/ai/analyze-schema")
async def ai_analyze_schema(req: AiAnalyzeSchemaRequest):
    try:
        schema_json = json.dumps(
            {"tables": [t.dict() for t in req.diagram.tables], "relationships": [r.dict() for r in req.diagram.relationships]},
            indent=2,
        )
        system_prompt = """You are a senior database architect. You will be given a database schema as JSON.
Analyze it and provide:
1. A plain English explanation of what this schema models (2-3 sentences)
2. Normalization issues found (1NF, 2NF, 3NF violations if any) — be specific about which table/column
3. Specific improvement suggestions (missing indexes, redundant columns, naming conventions)
4. A normalized form suggestion if the schema has issues

Format your response in clear sections with headings. Be concise and specific."""
        resp = get_client().chat.completions.create(
            model=DEPLOYMENT(),
            messages=[
```

```
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": f"Analyze this database schema:\n\n{schema_json}"},
        ],
        max_completion_tokens=16384,
    )
    return {"analysis": resp.choices[0].message.content}
  except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))


@app.post("/api/ai/chat")
async def ai_chat(req: AiChatRequest):
  try:
    schema_json = json.dumps(
      {"tables": [t.dict() for t in req.diagram.tables], "relationships": [r.dict() for r in req.diagram.relationships]},
      indent=2,
    )
    system_prompt = f"""You are an expert SQL query writer and database consultant. You have full knowledge of the user's current database schema (provided below as JSON).

When asked to write queries:
- Write clean, well-commented SQL
- Use proper JOINs based on the defined foreign keys
- Always specify the dialect (MySQL or PostgreSQL) in your response
- If the query differs between dialects, show both versions

When asked questions about the schema:
- Reference specific table and column names from the schema
```

```
- Be precise and technically accurate

Current dialect: {req.dialect}
Current schema:
{schema_json}"""
        messages = [{"role": "system", "content": system_prompt}]
        messages.extend({"role": m.role, "content": m.content} for m in req.messages)
        resp = get_client().chat.completions.create(
            model=DEPLOYMENT(),
            messages=messages,
            max_completion_tokens=16384,
        )
        return {"reply": resp.choices[0].message.content}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# ——— Share (Read-Only Snapshot) ——————————————————————————————

class ShareRequest(BaseModel):
    diagram: Diagram

def _gen_share_id() -> str:
    # ~8-char URL-safe token; 48 bits of entropy
    return secrets.token_urlsafe(6).replace("_", "a").replace("-", "b")

@app.post("/api/share")
async def create_share(req: ShareRequest):
    try:
```

```
        for _ in range(5):
            sid = _gen_share_id()
            if not await shares_coll.find_one({"shareId": sid}, {"_id": 1}):
                break
        else:
            raise HTTPException(status_code=500, detail="Failed to allocate share id")
        doc = {
            "shareId": sid,
            "diagram": req.diagram.dict(),
            "createdAt": datetime.now(timezone.utc).isoformat(),
            "views": 0,
        }
        await shares_coll.insert_one(doc)
        return {"shareId": sid}
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


@app.get("/api/share/{share_id}")
async def get_share(share_id: str):
    doc = await shares_coll.find_one({"shareId": share_id}, {"_id": 0})
    if not doc:
        raise HTTPException(status_code=404, detail="Share not found")
    # Best-effort view counter (fire-and-forget)
    try:
        await shares_coll.update_one({"shareId": share_id}, {"$inc": {"views": 1}})
    except Exception:
        pass
```

```
    return {
        "shareId": doc.get("shareId"),
        "diagram": doc.get("diagram"),
        "createdAt": doc.get("createdAt"),
        "views": (doc.get("views") or 0) + 1,
    }
```

```
============================================================
FILE: backend\requirements.txt
============================================================
```

```
annotated-doc==0.0.4
annotated-types==0.7.0
anyio==4.13.0
bcrypt==4.1.3
black==26.3.1
boto3==1.42.95
botocore==1.42.95
certifi==2026.4.22
cffi==2.0.0
charset-normalizer==3.4.7
click==8.3.3
colorama==0.4.6
cryptography==46.0.7
distro==1.9.0
dnspython==2.8.0
ecdsa==0.19.2
email-validator==2.3.0
fastapi==0.110.1
```

flake8==7.3.0

h11==0.16.0

httpcore==1.0.9

httpx==0.28.1

idna==3.13

iniconfig==2.3.0

isort==8.0.1

jiter==0.14.0

jmespath==1.1.0

jq==1.11.0

librt==0.9.0

markdown-it-py==4.0.0

mccabe==0.7.0

mdurl==0.1.2

motor==3.3.1

mypy==1.20.2

mypy-extensions==1.1.0

numpy==2.4.4

oauthlib==3.3.1

openai==2.32.0

packaging==26.1

pandas==3.0.2

passlib==1.7.4

pathspec==1.1.0

platformdirs==4.9.6

pluggy==1.6.0

pyasn1==0.6.3

pycodestyle==2.14.0

pycparser==3.0

pydantic==2.13.3

pydantic-core==2.46.3

pyflakes==3.4.0

pygments==2.20.0

pyjwt==2.12.1

pymongo==4.5.0

pytest==9.0.3

python-dateutil==2.9.0.post0

python-dotenv==1.2.2

python-jose==3.5.0

python-multipart==0.0.26

pytokens==0.4.1

requests==2.33.1

requests-oauthlib==2.0.0

rich==15.0.0

rsa==4.9.1

s3transfer==0.16.1

shellingham==1.5.4

six==1.17.0

sniffio==1.3.1

starlette==0.37.2

tqdm==4.67.3

typer==0.24.2

typing-extensions==4.15.0

typing-inspection==0.4.2

tzdata==2026.1

urllib3==2.6.3

uvicorn==0.25.0

```
============================================================
FILE: frontend\craco.config.js
============================================================

// craco.config.js
const path = require("path");
require("dotenv").config();

// Check if we're in development/preview mode (not production build)
// Craco sets NODE_ENV=development for start, NODE_ENV=production for build
const isDevServer = process.env.NODE_ENV !== "production";

// Environment variable overrides
const config = {
  enableHealthCheck: process.env.ENABLE_HEALTH_CHECK === "true",
};

// Conditionally load health check modules only if enabled
let WebpackHealthPlugin;
let setupHealthEndpoints;
let healthPluginInstance;

if (config.enableHealthCheck) {
  WebpackHealthPlugin = require("./plugins/health-check/webpack-health-plugin");
  setupHealthEndpoints = require("./plugins/health-check/health-endpoints");
  healthPluginInstance = new WebpackHealthPlugin();
}

let webpackConfig = {
```

```
eslint: {
  configure: {
    extends: ["plugin:react-hooks/recommended"],
    rules: {
      "react-hooks/rules-of-hooks": "error",
      "react-hooks/exhaustive-deps": "warn",
    },
  },
},
webpack: {
  alias: {
    '@': path.resolve(__dirname, 'src'),
  },
  configure: (webpackConfig) => {

    // Add ignored patterns to reduce watched directories
      webpackConfig.watchOptions = {
        ...webpackConfig.watchOptions,
        ignored: [
          '**/node_modules/**',
          '**/.git/**',
          '**/build/**',
          '**/dist/**',
          '**/coverage/**',
          '**/public/**',
      ],
    };

    // Add health check plugin to webpack if enabled
```

```
      if (config.enableHealthCheck && healthPluginInstance) {
        webpackConfig.plugins.push(healthPluginInstance);
      }
      return webpackConfig;
    },
  },
};

webpackConfig.devServer = (devServerConfig) => {
  // Add health check endpoints if enabled
  if (config.enableHealthCheck && setupHealthEndpoints && healthPluginInstance) {
    const originalSetupMiddlewares = devServerConfig.setupMiddlewares;

    devServerConfig.setupMiddlewares = (middlewares, devServer) => {
      // Call original setup if exists
      if (originalSetupMiddlewares) {
        middlewares = originalSetupMiddlewares(middlewares, devServer);
      }

      // Setup health endpoints
      setupHealthEndpoints(devServer, healthPluginInstance);

      return middlewares;
    };
  }

  return devServerConfig;
};
```

```
// Wrap with visual edits (automatically adds babel plugin, dev server, and overlay in dev mode)
if (isDevServer) {
  try {
    const { withVisualEdits } = require("@emergentbase/visual-edits/craco");
    webpackConfig = withVisualEdits(webpackConfig);
  } catch (err) {
    if (err.code === 'MODULE_NOT_FOUND' && err.message.includes('@emergentbase/visual-edits/craco')) {
      console.warn(
        "[visual-edits] @emergentbase/visual-edits not installed — visual editing disabled."
      );
    } else {
      throw err;
    }
  }
}

module.exports = webpackConfig;
```

```
============================================================
FILE: frontend\tailwind.config.js
============================================================
```

```
/** @type {import('tailwindcss').Config} */
module.exports = {
    darkMode: ["class"],
    content: [
    "./src/**/*.{js,jsx,ts,tsx}",
```

```
  "./public/index.html"
],
theme: {
        extend: {
                borderRadius: {
                        lg: 'var(--radius)',
                        md: 'calc(var(--radius) - 2px)',
                        sm: 'calc(var(--radius) - 4px)'
                },
                colors: {
                        background: 'hsl(var(--background))',
                        foreground: 'hsl(var(--foreground))',
                        card: {
                                DEFAULT: 'hsl(var(--card))',
                                foreground: 'hsl(var(--card-foreground))'
                        },
                        popover: {
                                DEFAULT: 'hsl(var(--popover))',
                                foreground: 'hsl(var(--popover-foreground))'
                        },
                        primary: {
                                DEFAULT: 'hsl(var(--primary))',
                                foreground: 'hsl(var(--primary-foreground))'
                        },
                        secondary: {
                                DEFAULT: 'hsl(var(--secondary))',
                                foreground: 'hsl(var(--secondary-foreground))'
                        },
                        muted: {
```

```
                DEFAULT: 'hsl(var(--muted))',
                foreground: 'hsl(var(--muted-foreground))'
            },
            accent: {
                DEFAULT: 'hsl(var(--accent))',
                foreground: 'hsl(var(--accent-foreground))'
            },
            destructive: {
                DEFAULT: 'hsl(var(--destructive))',
                foreground: 'hsl(var(--destructive-foreground))'
            },
            border: 'hsl(var(--border))',
            input: 'hsl(var(--input))',
            ring: 'hsl(var(--ring))',
            chart: {
                '1': 'hsl(var(--chart-1))',
                '2': 'hsl(var(--chart-2))',
                '3': 'hsl(var(--chart-3))',
                '4': 'hsl(var(--chart-4))',
                '5': 'hsl(var(--chart-5))'
            }
        },
        keyframes: {
            'accordion-down': {
                from: {
                    height: '0'
                },
                to: {
                    height: 'var(--radix-accordion-content-height)'
```

```
                        }
                    },
                    'accordion-up': {
                        from: {
                            height: 'var(--radix-accordion-content-height)'
                        },
                        to: {
                            height: '0'
                        }
                    }
                },
                animation: {
                    'accordion-down': 'accordion-down 0.2s ease-out',
                    'accordion-up': 'accordion-up 0.2s ease-out'
                }
            }
  },
  plugins: [require("tailwindcss-animate")],
};

============================================================
FILE: frontend\src\App.js
============================================================

import React, { useEffect, useState } from 'react';
import { Toaster } from 'react-hot-toast';
import './App.css';
import Header from './components/Header';
import LeftPanel from './components/LeftPanel/LeftPanel';
```

```
import CanvasArea from './components/Canvas/CanvasArea';
import RightPanel from './components/RightPanel/RightPanel';
import WelcomeScreen from './components/Modals/WelcomeScreen';
import ImportSqlModal from './components/Modals/ImportSqlModal';
import SaveDiagramModal from './components/Modals/SaveDiagramModal';
import ShareModal from './components/Modals/ShareModal';
import useDiagramStore from './store/diagramStore';
import useUIStore from './store/uiStore';
import useAutoSave from './hooks/useAutoSave';
import useKeyboardShortcuts from './hooks/useKeyboardShortcuts';
import { loadFromAutosave } from './utils/persistence';

function App() {
  const { loadDiagram, nodes } = useDiagramStore();
  const { importSqlModalOpen, saveDiagramModalOpen, shareModalOpen, theme }
= useUIStore();
  const [showWelcome, setShowWelcome] = useState(false);

  useAutoSave();
  useKeyboardShortcuts();

  useEffect(() => {
    const saved = loadFromAutosave();
    if (saved && saved.tables && saved.tables.length > 0) {
      loadDiagram(saved);
    } else {
      setShowWelcome(true);
    }
  }, []); // eslint-disable-line
```

```
return (
  <div className={`app-root ${theme}`} data-testid="app-root">
    <Header />
    <div className="app-body">
      <LeftPanel />
      <div className="canvas-wrapper">
        <CanvasArea />
        {showWelcome && nodes.length === 0 && (
          <WelcomeScreen onDismiss={() => setShowWelcome(false)} />
        )}
      </div>
      <RightPanel />
    </div>

    {importSqlModalOpen && <ImportSqlModal />}
    {saveDiagramModalOpen && <SaveDiagramModal />}
    {shareModalOpen && <ShareModal />}

    <Toaster
      position="bottom-right"
      toastOptions={{
        duration: 2500,
        style: {
          background: '#161b27',
          color: '#f1f5f9',
          border: '1px solid #1e293b',
          fontSize: '13px',
          borderRadius: '6px',
```

```
          },
          success: { iconTheme: { primary: '#22c55e', secondary: '#161b27' } },
          error: { iconTheme: { primary: '#f87171', secondary: '#161b27' } },
        }}
      />
    </div>
  );
}

export default App;

============================================================
FILE: frontend\src\index.js
============================================================

import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter, Routes, Route } from "react-router-dom";
import "@/index.css";
import App from "@/App";
import ShareView from "@/components/ShareView";

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <React.StrictMode>
    <BrowserRouter>
      <Routes>
        <Route path="/share/:shareId" element={<ShareView />} />
        <Route path="/*" element={<App />} />
```

```
      </Routes>
    </BrowserRouter>
  </React.StrictMode>,
);

============================================================
FILE: frontend\src\index.css
============================================================

/* ——— SketchSQL Design System
——————————————————————————————————————————— */
@tailwind base;
@tailwind components;
@tailwind utilities;

/* Design tokens */
:root {
  --deepest: #060912;
  --canvas-bg: #0d111a;
  --node-bg: #111827;
  --panel-bg: #161b27;
  --border: #1e293b;
  --divider: #2d3748;
  --muted: #64748b;
  --body: #94a3b8;
  --heading: #f1f5f9;
  --primary: #6366f1;
  --primary-btn: #4f46e5;
  --primary-hover: #5558e0;
```

```
  --pk: #fbbf24;
  --fk: #818cf8;
  --success: #22c55e;
  --error: #f87171;
  --radius: 6px;
  --header-h: 48px;
  --left-w: 220px;
  --right-w: 360px;
}

/* ——— Reset & Base
————————————————————————————————————————————————————————————————
——— */
*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }

html, body, #root { height: 100%; overflow: hidden; }

body {
  font-family: 'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif;
  font-size: 13px;
  background: var(--deepest);
  color: var(--body);
  -webkit-font-smoothing: antialiased;
}

/* Tailwind base overrides */
@layer base {
  :root {
    --background: 222 47% 5%;
    --foreground: 213 31% 91%;
```

```
    --card: 222 47% 8%;
    --card-foreground: 213 31% 91%;
    --popover: 222 47% 8%;
    --popover-foreground: 213 31% 91%;
    --primary: 239 84% 67%;
    --primary-foreground: 0 0% 100%;
    --secondary: 220 13% 15%;
    --secondary-foreground: 213 31% 91%;
    --muted: 220 13% 15%;
    --muted-foreground: 215 16% 47%;
    --accent: 220 13% 18%;
    --accent-foreground: 213 31% 91%;
    --destructive: 0 62% 71%;
    --destructive-foreground: 0 0% 100%;
    --border: 215 27% 15%;
    --input: 215 27% 15%;
    --ring: 239 84% 67%;
    --radius: 0.5rem;
  }
  * { @apply border-border; }
  body { @apply bg-background text-foreground; }
}

/* Scrollbar */
::-webkit-scrollbar { width: 5px; height: 5px; }
::-webkit-scrollbar-track { background: transparent; }
::-webkit-scrollbar-thumb { background: var(--divider); border-radius: 3px; }
::-webkit-scrollbar-thumb:hover { background: var(--muted); }
```

```
/* ——— App Layout
————————————————————————————————————————————————————————————————
———————— */
.app-root {
  display: flex;
  flex-direction: column;
  height: 100vh;
  width: 100vw;
  overflow: hidden;
  background: var(--deepest);
}

.app-body {
  display: flex;
  flex: 1;
  overflow: hidden;
  height: calc(100vh - var(--header-h));
}

.canvas-wrapper {
  flex: 1;
  position: relative;
  overflow: hidden;
  background: var(--canvas-bg);
}

/* ——— Header
————————————————————————————————————————————————————————————————
———————————————— */
.app-header {
```

```
  height: var(--header-h);
  min-height: var(--header-h);
  background: var(--panel-bg);
  border-bottom: 1px solid var(--border);
  display: flex;
  align-items: center;
  padding: 0 12px;
  gap: 12px;
  z-index: 100;
}

.header-left { display: flex; align-items: center; gap: 10px; flex: 1; min-width: 0; }
.header-center { display: flex; align-items: center; justify-content: center; }
.header-right { display: flex; align-items: center; gap: 6px; }

.header-logo { display: flex; align-items: center; gap: 7px; }
.logo-text { font-size: 15px; font-weight: 700; color: var(--heading); letter-spacing: -0.3px; }
.header-divider { width: 1px; height: 18px; background: var(--border); }

.diagram-name {
  font-size: 13px;
  color: var(--body);
  cursor: pointer;
  padding: 3px 6px;
  border-radius: var(--radius);
  transition: background 0.15s;
  max-width: 180px;
  overflow: hidden;
```

```
  text-overflow: ellipsis;
  white-space: nowrap;
}
.diagram-name:hover { background: var(--border); color: var(--heading); }
.diagram-name-input {
  background: var(--border);
  border: 1px solid var(--primary);
  color: var(--heading);
  padding: 2px 6px;
  border-radius: 4px;
  font-size: 13px;
  outline: none;
  width: 160px;
}

.dialect-toggle {
  display: flex;
  background: var(--node-bg);
  border: 1px solid var(--border);
  border-radius: 6px;
  overflow: hidden;
}
.dialect-btn {
  padding: 4px 12px;
  font-size: 12px;
  font-weight: 500;
  color: var(--muted);
  background: transparent;
  border: none;
```

```
  cursor: pointer;
  transition: all 0.15s;
}
.dialect-btn.active {
  background: var(--primary-btn);
  color: #fff;
}
.dialect-btn:not(.active):hover { color: var(--body); background: var(--border); }

.header-action-btn {
  display: flex;
  align-items: center;
  gap: 5px;
  padding: 5px 10px;
  background: var(--border);
  border: 1px solid var(--divider);
  color: var(--body);
  border-radius: var(--radius);
  font-size: 12px;
  cursor: pointer;
  transition: all 0.15s;
}
.header-action-btn:hover { background: var(--divider); color: var(--heading); }

.header-icon-btn {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 30px; height: 30px;
```

```
  background: var(--border);
  border: 1px solid var(--divider);
  color: var(--muted);
  border-radius: var(--radius);
  cursor: pointer;
  transition: all 0.15s;
}
.header-icon-btn:hover { color: var(--heading); background: var(--divider); }

.export-wrap { position: relative; }
.dropdown-backdrop { position: fixed; inset: 0; z-index: 400; }
.dropdown-menu {
  position: absolute;
  right: 0;
  top: calc(100% + 4px);
  background: var(--panel-bg);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  min-width: 160px;
  z-index: 500;
  box-shadow: 0 8px 24px rgba(0,0,0,0.5);
  overflow: hidden;
}
.dropdown-menu button {
  display: block;
  width: 100%;
  text-align: left;
  padding: 8px 12px;
  color: var(--body);
```

```
  background: transparent;
  border: none;
  cursor: pointer;
  font-size: 12px;
  transition: background 0.1s;
}
.dropdown-menu button:hover { background: var(--border); color: var(--heading); }

/* ——— Left Panel ——————————————————————————————————————————————————————— */
.left-panel {
  width: var(--left-w);
  min-width: var(--left-w);
  background: var(--panel-bg);
  border-right: 1px solid var(--border);
  display: flex;
  flex-direction: column;
  overflow: hidden;
}

.left-panel-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 10px 12px 8px;
  border-bottom: 1px solid var(--border);
}
.left-panel-title { font-size: 11px; font-weight: 600; color: var(--muted); text-transform: uppercase; letter-spacing: 0.05em; }
```

```
.icon-btn {
  display: flex; align-items: center; justify-content: center;
  width: 22px; height: 22px;
  background: var(--border);
  border: 1px solid var(--divider);
  color: var(--muted);
  border-radius: 4px;
  cursor: pointer;
  transition: all 0.15s;
}
.icon-btn:hover { background: var(--primary-btn); color: #fff; border-color: var(--primary); }

.diagram-list { flex: 1; overflow-y: auto; padding: 6px; }
.empty-list { padding: 20px 12px; color: var(--muted); font-size: 12px; text-align: center; }

.diagram-item {
  display: flex;
  align-items: center;
  gap: 7px;
  padding: 7px 8px;
  border-radius: var(--radius);
  cursor: pointer;
  transition: background 0.12s;
  margin-bottom: 1px;
}
.diagram-item:hover { background: var(--border); }
```

```
.diagram-item.active { background: rgba(99,102,241,0.15); border: 1px solid rgba(99,102,241,0.3); }

.di-icon { color: var(--muted); flex-shrink: 0; }

.di-info { flex: 1; min-width: 0; }

.di-name { display: block; font-size: 12px; color: var(--heading); font-weight: 500; overflow: hidden; text-overflow: ellipsis; white-space: nowrap; }

.di-meta { display: flex; align-items: center; gap: 5px; margin-top: 2px; color: var(--muted); font-size: 10px; }

.di-tables { background: var(--border); padding: 0 4px; border-radius: 3px; }


.rename-input {
  flex: 1;
  background: var(--border);
  border: 1px solid var(--primary);
  color: var(--heading);
  padding: 2px 6px;
  border-radius: 4px;
  font-size: 12px;
  outline: none;
}

/* ——— Canvas ——————————————————————————————————————————————————————————————— */

.canvas-container {
  width: 100%;
  height: 100%;
  position: relative;
  display: flex;
  flex-direction: column;
}
```

```
/* ReactFlow overrides */
.react-flow {
  background: var(--canvas-bg) !important;
  flex: 1;
}
.react-flow__controls {
  background: var(--panel-bg) !important;
  border: 1px solid var(--border) !important;
  border-radius: var(--radius) !important;
  box-shadow: none !important;
}
.react-flow__controls-button {
  background: transparent !important;
  border: none !important;
  border-bottom: 1px solid var(--border) !important;
  color: var(--muted) !important;
  fill: var(--muted) !important;
  width: 26px !important; height: 26px !important;
}
.react-flow__controls-button:last-child { border-bottom: none !important; }
.react-flow__controls-button:hover { background: var(--border) !important; color: var(--heading) !important; fill: var(--heading) !important; }
.react-flow__minimap { background: var(--node-bg) !important; border: 1px solid var(--border) !important; border-radius: var(--radius) !important; }
.react-flow__edge-path { stroke: var(--fk) !important; }
.react-flow__edge.selected .react-flow__edge-path { stroke: var(--primary) !important; }
.react-flow__connection-line { stroke: var(--primary) !important; stroke-width: 2; }
```

```
/* ——— Canvas Toolbar
————————————————————————————————————————————————————————————————
——— */
.canvas-toolbar {
  display: flex;
  align-items: center;
  gap: 3px;
  padding: 6px 10px;
  background: var(--panel-bg);
  border-bottom: 1px solid var(--border);
  z-index: 10;
}
.tb-btn {
  display: flex; align-items: center; gap: 5px;
  padding: 5px 9px;
  background: var(--border);
  border: 1px solid var(--divider);
  color: var(--body);
  border-radius: var(--radius);
  font-size: 12px;
  cursor: pointer;
  transition: all 0.15s;
  white-space: nowrap;
}
.tb-btn:hover:not(:disabled) { background: var(--divider); color: var(--heading); }
.tb-btn:disabled { opacity: 0.35; cursor: not-allowed; }
.tb-btn.on { background: rgba(99,102,241,0.2); border-color: rgba(99,102,241,0.4);
color: #818cf8; }
.tb-btn.primary { background: var(--primary-btn); border-color: var(--primary); color:
#fff; }
```

```
.tb-btn.primary:hover { background: var(--primary-hover); }
.tb-sep { width: 1px; height: 18px; background: var(--border); margin: 0 3px; }

/* ——— Table Node ——————————————————————————————————————————————————————— */
.react-flow__node-tableNode { padding: 0 !important; background: transparent !important; border: none !important; }

.table-node {
  min-width: 190px;
  max-width: 280px;
  background: var(--node-bg);
  border: 1px solid var(--border);
  border-radius: 8px;
  overflow: visible;
  box-shadow: 0 4px 16px rgba(0,0,0,0.4);
  transition: box-shadow 0.2s, border-color 0.2s;
}
.table-node.selected {
  border-color: var(--primary);
  box-shadow: 0 0 0 2px rgba(99,102,241,0.3), 0 4px 16px rgba(0,0,0,0.4);
}
.table-node:hover { box-shadow: 0 6px 20px rgba(0,0,0,0.5); }

.tn-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 7px 10px;
```

```
  border-radius: 7px 7px 0 0;
  cursor: grab;
  min-height: 34px;
}
.tn-name { font-size: 13px; font-weight: 600; color: #fff; flex: 1; overflow: hidden; text-overflow: ellipsis; white-space: nowrap; }
.tn-name-input {
  flex: 1;
  background: rgba(0,0,0,0.3);
  border: 1px solid rgba(255,255,255,0.3);
  color: #fff;
  padding: 1px 5px;
  border-radius: 3px;
  font-size: 13px;
  font-weight: 600;
  outline: none;
  min-width: 0;
}
.tn-add-col {
  background: rgba(255,255,255,0.15);
  border: none;
  color: #fff;
  width: 18px; height: 18px;
  border-radius: 3px;
  cursor: pointer;
  font-size: 14px;
  line-height: 1;
  opacity: 0.7;
  transition: opacity 0.15s;
```

```
  display: flex; align-items: center; justify-content: center;
}
.tn-add-col:hover { opacity: 1; background: rgba(255,255,255,0.25); }

.tn-columns { padding: 2px 0 4px; }
.tn-col-wrapper { position: relative; }

.tn-col {
  display: flex;
  align-items: center;
  gap: 5px;
  padding: 4px 10px;
  cursor: pointer;
  transition: background 0.1s;
  min-height: 26px;
}
.tn-col:hover { background: rgba(255,255,255,0.04); }
.tn-col.active { background: rgba(99,102,241,0.12); }

.tn-badges { display: flex; gap: 3px; flex-shrink: 0; }
.badge-pk { font-size: 9px; font-weight: 700; color: var(--pk); background:
rgba(251,191,36,0.12); padding: 0 4px; border-radius: 3px; line-height: 14px; }
.badge-uq { font-size: 9px; font-weight: 700; color: var(--fk); background:
rgba(129,140,248,0.12); padding: 0 4px; border-radius: 3px; line-height: 14px; }
.badge-nn { font-size: 9px; font-weight: 700; color: var(--muted); background:
rgba(100,116,139,0.15); padding: 0 4px; border-radius: 3px; line-height: 14px; }

.tn-col-name { flex: 1; font-size: 12px; color: var(--heading); overflow: hidden; text-
overflow: ellipsis; white-space: nowrap; }
```

```
.tn-col-type { font-size: 10px; color: #fb923c; font-family: 'Courier New', monospace; flex-shrink: 0; max-width: 90px; overflow: hidden; text-overflow: ellipsis; white-space: nowrap; }
.tn-empty { padding: 8px 10px; font-size: 11px; color: var(--muted); font-style: italic; }

/* Column handles */
.tn-handle {
  width: 8px !important;
  height: 8px !important;
  background: var(--fk) !important;
  border: 2px solid var(--node-bg) !important;
  border-radius: 50% !important;
  transition: transform 0.15s, background 0.15s !important;
}
.tn-handle:hover {
  transform: scale(1.4) !important;
  background: var(--primary) !important;
}

/* Column inline editor */
.col-editor {
  background: rgba(22,27,39,0.98);
  border: 1px solid var(--border);
  border-top: 1px solid rgba(99,102,241,0.3);
  padding: 8px 10px;
  display: flex;
  flex-direction: column;
  gap: 6px;
  animation: slideDown 0.15s ease;
}
```

```
@keyframes slideDown { from { opacity: 0; transform: translateY(-4px); } to { opacity: 1; transform: translateY(0); } }

.ce-row { display: flex; align-items: center; gap: 6px; }
.ce-row label { font-size: 10px; color: var(--muted); width: 45px; flex-shrink: 0; }
.ce-row input, .ce-row select {
  flex: 1;
  background: var(--node-bg);
  border: 1px solid var(--border);
  color: var(--heading);
  padding: 3px 6px;
  border-radius: 4px;
  font-size: 11px;
  outline: none;
}
.ce-row input:focus, .ce-row select:focus { border-color: var(--primary); }
.ce-row select option { background: var(--node-bg); }

.ce-checks { display: flex; gap: 10px; flex-wrap: wrap; }
.ce-checks label { display: flex; align-items: center; gap: 4px; font-size: 11px; color: var(--body); cursor: pointer; }
.ce-checks input[type="checkbox"] { accent-color: var(--primary); cursor: pointer; }

.ce-actions { display: flex; justify-content: space-between; gap: 6px; margin-top: 2px; }
.ce-delete {
  padding: 3px 8px; font-size: 11px; color: var(--error); background: rgba(248,113,113,0.1);
  border: 1px solid rgba(248,113,113,0.3); border-radius: 4px; cursor: pointer;
  transition: all 0.15s;
```

```
}
.ce-delete:hover { background: rgba(248,113,113,0.2); }
.ce-close {
  padding: 3px 8px; font-size: 11px; color: var(--muted); background: var(--border);
  border: 1px solid var(--divider); border-radius: 4px; cursor: pointer; transition: all 0.15s;
}
.ce-close:hover { color: var(--heading); }

/* ——— Edge Label ——————————————————————————————————————————————————— */
.edge-label {
  background: var(--panel-bg);
  border: 1px solid var(--fk);
  color: var(--fk);
  font-size: 10px;
  font-weight: 700;
  padding: 1px 5px;
  border-radius: 3px;
  letter-spacing: 0.03em;
  transition: border-color 0.15s, color 0.15s;
}

/* ——— Context Menu ——————————————————————————————————————————————————— */
.ctx-backdrop { position: fixed; inset: 0; z-index: 9998; }
.ctx-menu {
  background: var(--panel-bg);
  border: 1px solid var(--border);
```

```
  border-radius: var(--radius);
  min-width: 150px;
  box-shadow: 0 8px 24px rgba(0,0,0,0.6);
  overflow: hidden;
  z-index: 9999;
}
.ctx-menu button {
  display: flex; align-items: center; gap: 7px;
  width: 100%; padding: 7px 12px;
  text-align: left; font-size: 12px; color: var(--body);
  background: transparent; border: none; cursor: pointer; transition: background 0.1s;
}
.ctx-menu button:hover { background: var(--border); color: var(--heading); }
.ctx-menu .danger, .ctx-menu .ctx-danger { color: var(--error) !important; }
.ctx-menu .danger:hover, .ctx-menu .ctx-danger:hover { background:
rgba(248,113,113,0.1) !important; }
.ctx-sep { height: 1px; background: var(--border); margin: 3px 0; }
.ctx-colors { display: flex; gap: 6px; padding: 6px 12px; }
.ctx-dot { width: 16px; height: 16px; border-radius: 50%; border: 2px solid
transparent; cursor: pointer; transition: transform 0.15s; }
.ctx-dot:hover { transform: scale(1.2); }
.ctx-dot.active { border-color: #fff; }

/* ——— Right Panel
————————————————————————————————————————————————————————————————
——————— */
.right-panel {
  width: var(--right-w);
  min-width: var(--right-w);
  background: var(--panel-bg);
```

```
  border-left: 1px solid var(--border);
  display: flex;
  flex-direction: column;
  overflow: hidden;
}

.right-panel-tabs {
  display: flex;
  border-bottom: 1px solid var(--border);
  background: var(--node-bg);
}
.rp-tab {
  display: flex; align-items: center; gap: 5px;
  flex: 1; padding: 9px 6px;
  justify-content: center;
  font-size: 11px; font-weight: 500; color: var(--muted);
  background: transparent; border: none; border-bottom: 2px solid transparent;
  cursor: pointer; transition: all 0.15s;
}
.rp-tab:hover { color: var(--body); }
.rp-tab.active { color: var(--primary); border-bottom-color: var(--primary); }
.right-panel-content { flex: 1; overflow: hidden; display: flex; flex-direction: column; }

/* ——— SQL Tab
————————————————————————————————————————————————————————————————————————————— */
.sql-tab { display: flex; flex-direction: column; height: 100%; }
.sql-toolbar {
  display: flex; align-items: center; justify-content: space-between;
  padding: 8px 10px; border-bottom: 1px solid var(--border);
```

```
}
.sql-gen-btn {
  display: flex; align-items: center; gap: 5px;
  padding: 5px 12px; background: var(--primary-btn); border: none;
  color: #fff; border-radius: var(--radius); font-size: 12px; font-weight: 500;
  cursor: pointer; transition: background 0.15s;
}
.sql-gen-btn:hover:not(:disabled) { background: var(--primary-hover); }
.sql-gen-btn:disabled { opacity: 0.5; cursor: not-allowed; }
.sql-toolbar-right { display: flex; gap: 4px; }
.sql-icon-btn {
  display: flex; align-items: center; justify-content: center;
  width: 28px; height: 28px;
  background: var(--border); border: 1px solid var(--divider);
  color: var(--muted); border-radius: var(--radius); cursor: pointer; transition: all 0.15s;
}
.sql-icon-btn:hover { background: var(--divider); color: var(--heading); }
.monaco-wrap { flex: 1; overflow: hidden; }
.spin { animation: spin 1s linear infinite; }
@keyframes spin { from { transform: rotate(0deg); } to { transform: rotate(360deg); } }

/* ——— AI Tab ——————————————————————————————————————————————————————————————————————————————— */
.ai-tab { display: flex; flex-direction: column; height: 100%; overflow: hidden; }
.ai-gen-section {
  padding: 10px;
  border-bottom: 1px solid var(--border);
}
```

```
.ai-gen-label { display: flex; align-items: center; gap: 5px; font-size: 11px; font-weight: 600; color: var(--muted); margin-bottom: 7px; text-transform: uppercase; letter-spacing: 0.04em; }
.ai-gen-row { display: flex; gap: 6px; }
.ai-gen-input {
  flex: 1;
  background: var(--node-bg);
  border: 1px solid var(--border);
  color: var(--heading);
  padding: 6px 8px;
  border-radius: var(--radius);
  font-size: 12px;
  outline: none;
  transition: border-color 0.15s;
}
.ai-gen-input:focus { border-color: var(--primary); }
.ai-gen-input::placeholder { color: var(--muted); }
.ai-gen-btn {
  padding: 6px 12px;
  background: var(--primary-btn);
  border: none; color: #fff; border-radius: var(--radius);
  font-size: 12px; font-weight: 500; cursor: pointer; white-space: nowrap;
  transition: background 0.15s;
}
.ai-gen-btn:hover:not(:disabled) { background: var(--primary-hover); }
.ai-gen-btn:disabled { opacity: 0.5; cursor: not-allowed; }

.ai-chips { display: flex; flex-wrap: wrap; gap: 5px; margin-top: 7px; }
.ai-chip {
  padding: 3px 8px; font-size: 11px;
```

```
  background: var(--border); border: 1px solid var(--divider);
  color: var(--body); border-radius: 12px; cursor: pointer; transition: all 0.15s;
}
.ai-chip:hover { border-color: var(--fk); color: var(--fk); background:
rgba(129,140,248,0.1); }

.ai-actions {
  display: flex; gap: 6px; align-items: center;
  padding: 8px 10px; border-bottom: 1px solid var(--border);
}
.ai-analyze-btn {
  display: flex; align-items: center; gap: 5px;
  padding: 5px 10px; font-size: 12px; font-weight: 500;
  background: rgba(129,140,248,0.12); border: 1px solid rgba(129,140,248,0.3);
  color: var(--fk); border-radius: var(--radius); cursor: pointer; transition: all 0.15s;
}
.ai-analyze-btn:hover:not(:disabled) { background: rgba(129,140,248,0.2); }
.ai-analyze-btn:disabled { opacity: 0.5; cursor: not-allowed; }
.ai-clear-btn {
  display: flex; align-items: center; gap: 4px;
  padding: 5px 8px; font-size: 11px;
  background: var(--border); border: 1px solid var(--divider);
  color: var(--muted); border-radius: var(--radius); cursor: pointer; transition: all 0.15s;
  margin-left: auto;
}
.ai-clear-btn:hover { color: var(--error); border-color: rgba(248,113,113,0.3); }

.chat-area {
  flex: 1; overflow-y: auto; padding: 10px;
```

```
  display: flex; flex-direction: column; gap: 10px;
}
.chat-empty {
  display: flex; flex-direction: column; align-items: center; justify-content: center;
  flex: 1; color: var(--muted); font-size: 12px; text-align: center;
  padding: 30px 20px;
}
.chat-msg { display: flex; gap: 8px; align-items: flex-start; }
.chat-msg.user { flex-direction: row-reverse; }
.chat-avatar {
  width: 24px; height: 24px; border-radius: 50%; flex-shrink: 0;
  display: flex; align-items: center; justify-content: center;
  background: var(--border); color: var(--muted);
}
.chat-msg.ai .chat-avatar { background: rgba(99,102,241,0.2); color: var(--fk); }
.chat-msg.user .chat-avatar { background: rgba(34,197,94,0.15); color: var(--
success); }

.chat-bubble {
  max-width: calc(100% - 36px);
  background: var(--node-bg);
  border: 1px solid var(--border);
  border-radius: 8px;
  padding: 8px 10px;
  font-size: 12px;
  line-height: 1.6;
  color: var(--body);
}
.chat-msg.user .chat-bubble { background: rgba(99,102,241,0.12); border-color:
rgba(99,102,241,0.25); color: var(--heading); }
```

```
.chat-bubble p { margin: 0 0 6px; }

.chat-bubble p:last-child { margin: 0; }

.chat-bubble pre { background: var(--deepest); border: 1px solid var(--border); border-radius: 4px; padding: 8px; overflow-x: auto; margin: 6px 0; }

.chat-bubble code { font-family: 'Courier New', monospace; font-size: 11px; color: #4ade80; }

.chat-bubble h1,h2,h3 { color: var(--heading); font-weight: 600; margin: 8px 0 4px; font-size: 13px; }

.chat-bubble ul,ol { padding-left: 16px; margin: 4px 0; }

.chat-bubble li { margin-bottom: 2px; }

.chat-bubble strong { color: var(--heading); font-weight: 600; }


.thinking { display: flex; align-items: center; gap: 4px; padding: 10px 12px; }

.dot {

  width: 6px; height: 6px; border-radius: 50%;

  background: var(--fk);

  animation: bounce 1.2s infinite;

}

.dot:nth-child(2) { animation-delay: 0.2s; }

.dot:nth-child(3) { animation-delay: 0.4s; }

@keyframes bounce { 0%,60%,100% { transform: translateY(0); } 30% { transform: translateY(-6px); } }


.chat-input-row {

  display: flex; gap: 6px; padding: 8px 10px;

  border-top: 1px solid var(--border);

  background: var(--panel-bg);

}

.chat-input {

  flex: 1;
```

```
  background: var(--node-bg);
  border: 1px solid var(--border);
  color: var(--heading);
  padding: 6px 8px;
  border-radius: var(--radius);
  font-size: 12px;
  outline: none;
  resize: none;
  transition: border-color 0.15s;
}
.chat-input:focus { border-color: var(--primary); }
.chat-input::placeholder { color: var(--muted); }
.chat-send-btn {
  display: flex; align-items: center; justify-content: center;
  width: 32px; height: 32px;
  background: var(--primary-btn);
  border: none; color: #fff; border-radius: var(--radius); cursor: pointer; transition:
background 0.15s;
}
.chat-send-btn:hover:not(:disabled) { background: var(--primary-hover); }
.chat-send-btn:disabled { opacity: 0.4; cursor: not-allowed; }

/* ——— Properties Tab
———————————————————————————————————————————————————————————————
——— */
.props-panel { padding: 10px; display: flex; flex-direction: column; gap: 12px;
overflow-y: auto; height: 100%; }
.props-empty { display: flex; align-items: center; justify-content: center; height: 100%;
color: var(--muted); font-size: 12px; text-align: center; padding: 20px; }
.props-section { display: flex; flex-direction: column; gap: 8px; }
.props-section-title {
```

```
  display: flex; align-items: center; justify-content: space-between;
  font-size: 11px; font-weight: 600; color: var(--muted); text-transform: uppercase;
letter-spacing: 0.05em;
  padding-bottom: 6px; border-bottom: 1px solid var(--border);
}
.mini-btn {
  padding: 2px 7px; font-size: 11px;
  background: var(--border); border: 1px solid var(--divider);
  color: var(--body); border-radius: 4px; cursor: pointer;
}
.mini-btn:hover { color: var(--heading); }

.props-row { display: flex; align-items: center; gap: 8px; margin-bottom: 4px; }
.props-row label { font-size: 11px; color: var(--muted); min-width: 70px; flex-shrink: 0;
}
.props-row input, .props-row select {
  flex: 1;
  background: var(--node-bg);
  border: 1px solid var(--border);
  color: var(--heading);
  padding: 4px 8px;
  border-radius: 4px;
  font-size: 12px;
  outline: none;
}
.props-row input:focus, .props-row select:focus { border-color: var(--primary); }
.props-row select option { background: var(--node-bg); }

.props-col {
  background: var(--node-bg);
```

```
  border: 1px solid var(--border);
  border-radius: var(--radius);
  padding: 7px 8px;
  display: flex; flex-direction: column; gap: 5px;
}
.props-col .props-row { gap: 5px; }
.props-col .props-row input { padding: 3px 6px; font-size: 11px; }
.props-col .props-row select { padding: 3px 5px; font-size: 11px; }
.props-checks { display: flex; gap: 8px; flex-wrap: wrap; }
.props-checks label { display: flex; align-items: center; gap: 3px; font-size: 11px; color: var(--body); cursor: pointer; }
.props-checks input[type="checkbox"] { accent-color: var(--primary); }
.del-btn {
  display: flex; align-items: center; justify-content: center;
  width: 22px; height: 22px; border-radius: 4px;
  background: transparent; border: 1px solid transparent;
  color: var(--muted); cursor: pointer; transition: all 0.15s; flex-shrink: 0;
}
.del-btn:hover { color: var(--error); border-color: rgba(248,113,113,0.3); background: rgba(248,113,113,0.1); }

.color-swatches { display: flex; gap: 6px; }
.swatch { width: 18px; height: 18px; border-radius: 50%; border: 2px solid transparent; cursor: pointer; transition: transform 0.15s; }
.swatch:hover { transform: scale(1.15); }
.swatch.active { border-color: #fff; }

.props-danger { margin-top: auto; padding-top: 10px; border-top: 1px solid var(--border); }
.danger-btn {
```

```
  display: flex; align-items: center; gap: 5px; width: 100%;
  padding: 6px 10px; font-size: 12px; color: var(--error);
  background: rgba(248,113,113,0.08); border: 1px solid rgba(248,113,113,0.25);
  border-radius: var(--radius); cursor: pointer; transition: all 0.15s; justify-content: center;
}
.danger-btn:hover { background: rgba(248,113,113,0.15); }

/* ——— Modals ——————————————————————————————————————————————————————— */
.modal-overlay {
  position: fixed; inset: 0; z-index: 1000;
  background: rgba(6,9,18,0.8);
  display: flex; align-items: center; justify-content: center;
  backdrop-filter: blur(3px);
  animation: fadeIn 0.15s ease;
}
@keyframes fadeIn { from { opacity: 0; } to { opacity: 1; } }

.modal-box {
  background: var(--panel-bg);
  border: 1px solid var(--border);
  border-radius: 10px;
  width: 560px;
  max-width: 95vw;
  max-height: 90vh;
  display: flex; flex-direction: column;
  box-shadow: 0 20px 60px rgba(0,0,0,0.7);
  animation: slideUp 0.2s ease;
```

```
}
.modal-box.small { width: 380px; }
@keyframes slideUp { from { transform: translateY(16px); opacity: 0; } to { transform: translateY(0); opacity: 1; } }

.modal-header {
  display: flex; align-items: center; justify-content: space-between;
  padding: 14px 16px;
  border-bottom: 1px solid var(--border);
  font-size: 14px; font-weight: 600; color: var(--heading);
}
.modal-close {
  display: flex; align-items: center; justify-content: center;
  width: 24px; height: 24px; border-radius: 4px;
  background: var(--border); border: none; color: var(--muted); cursor: pointer; transition: all 0.15s;
}
.modal-close:hover { color: var(--heading); }

.modal-body { padding: 14px 16px; flex: 1; overflow: auto; }
.modal-hint { font-size: 12px; color: var(--muted); margin-bottom: 10px; }
.modal-label { display: block; font-size: 12px; color: var(--muted); margin-bottom: 6px; font-weight: 500; }
.modal-input {
  width: 100%;
  background: var(--node-bg);
  border: 1px solid var(--border);
  color: var(--heading);
  padding: 8px 10px;
  border-radius: var(--radius);
```

```
  font-size: 13px;
  outline: none;
}
.modal-input:focus { border-color: var(--primary); }

.sql-paste-area {
  width: 100%; resize: vertical;
  background: #0d0f14;
  border: 1px solid var(--border);
  color: #4ade80;
  padding: 10px 12px;
  border-radius: var(--radius);
  font-family: 'Courier New', monospace;
  font-size: 12px; line-height: 1.5;
  outline: none;
  transition: border-color 0.15s;
}
.sql-paste-area:focus { border-color: var(--primary); }
.sql-paste-area::placeholder { color: var(--muted); }

.modal-footer {
  display: flex; justify-content: flex-end; gap: 8px;
  padding: 12px 16px;
  border-top: 1px solid var(--border);
}
.modal-cancel {
  padding: 7px 14px; font-size: 13px;
  background: var(--border); border: 1px solid var(--divider);
  color: var(--body); border-radius: var(--radius); cursor: pointer; transition: all 0.15s;
```

```
}
.modal-cancel:hover { color: var(--heading); }
.modal-confirm {
  display: flex; align-items: center; gap: 5px;
  padding: 7px 16px; font-size: 13px; font-weight: 500;
  background: var(--primary-btn); border: none;
  color: #fff; border-radius: var(--radius); cursor: pointer; transition: background 0.15s;
}
.modal-confirm:hover:not(:disabled) { background: var(--primary-hover); }
.modal-confirm:disabled { opacity: 0.5; cursor: not-allowed; }

/* ——— Welcome Screen
——————————————————————————————————————————————————————————————————
——— */
.welcome-overlay {
  position: absolute; inset: 0; z-index: 50;
  background: rgba(6,9,18,0.88);
  display: flex; align-items: center; justify-content: center;
  backdrop-filter: blur(4px);
}
.welcome-box {
  background: var(--panel-bg);
  border: 1px solid var(--border);
  border-radius: 14px;
  padding: 36px 40px;
  text-align: center;
  max-width: 480px;
  width: 90%;
  box-shadow: 0 20px 60px rgba(0,0,0,0.7);
  animation: slideUp 0.25s ease;
```

```
}
.welcome-logo { margin-bottom: 14px; }
.welcome-title { font-size: 26px; font-weight: 800; color: var(--heading); letter-spacing: -0.5px; margin-bottom: 6px; }
.welcome-tagline { font-size: 13px; color: var(--muted); margin-bottom: 28px; }

.welcome-cards { display: flex; gap: 12px; }
.welcome-card {
  flex: 1; display: flex; flex-direction: column; align-items: center; gap: 8px;
  padding: 18px 14px;
  background: var(--node-bg); border: 1px solid var(--border);
  border-radius: 10px; cursor: pointer; transition: all 0.2s; text-align: center;
}
.welcome-card:hover { border-color: var(--fk); background: rgba(99,102,241,0.08); transform: translateY(-2px); }
.welcome-card strong { font-size: 12px; color: var(--heading); font-weight: 600; display: block; }
.welcome-card span { font-size: 11px; color: var(--muted); }

.example-list { display: flex; flex-direction: column; gap: 7px; }
.example-list-title { font-size: 13px; color: var(--body); margin-bottom: 4px; }
.example-item {
  display: flex; align-items: center; justify-content: space-between;
  padding: 10px 14px;
  background: var(--node-bg); border: 1px solid var(--border);
  border-radius: 7px; cursor: pointer; transition: all 0.15s; font-size: 13px; color: var(--heading);
}
.example-item:hover { border-color: var(--primary); background: rgba(99,102,241,0.1); }
```

```
.example-tables { font-size: 11px; color: var(--muted); background: var(--border); padding: 1px 7px; border-radius: 10px; }
.back-link { font-size: 12px; color: var(--muted); background: none; border: none; cursor: pointer; margin-top: 4px; }
.back-link:hover { color: var(--body); }

/* ——— Light theme ——————————————————————————————————————————————— */
.light {
  --deepest: #f8fafc;
  --canvas-bg: #f1f5f9;
  --node-bg: #ffffff;
  --panel-bg: #f8fafc;
  --border: #e2e8f0;
  --divider: #cbd5e1;
  --muted: #94a3b8;
  --body: #475569;
  --heading: #0f172a;
}
.light .table-node { box-shadow: 0 2px 8px rgba(0,0,0,0.1); }
.light .react-flow { background: var(--canvas-bg) !important; }
.light .chat-bubble { background: #fff; }

/* ——— ORM Tab ——————————————————————————————————————————————————— */
.orm-tab { display: flex; flex-direction: column; height: 100%; }
.orm-toolbar {
  display: flex; align-items: center; justify-content: space-between;
  padding: 8px 10px; border-bottom: 1px solid var(--border); gap: 8px;
```

```
}
.orm-format-btns { display: flex; gap: 3px; background: var(--node-bg); border: 1px solid var(--border); border-radius: var(--radius); padding: 2px; }
.orm-fmt-btn {
  padding: 4px 10px; font-size: 11px; font-weight: 500;
  background: transparent; border: none; color: var(--muted);
  border-radius: 4px; cursor: pointer; transition: all 0.15s;
}
.orm-fmt-btn:hover { color: var(--body); }
.orm-fmt-btn.active { background: var(--primary-btn); color: #fff; }
.orm-toolbar-right { display: flex; gap: 4px; }

/* ——— Misc utilities ——————————————————————————————————————— */
.react-flow__node-tableNode.selected > div { border-color: var(--primary) !important; box-shadow: 0 0 0 2px rgba(99,102,241,0.25) !important; }

============================================================
FILE: frontend\src\api\apiClient.js
============================================================

import axios from "axios";

const API = axios.create({
  baseURL: `${process.env.REACT_APP_BACKEND_URL}/api`,
  headers: { "Content-Type": "application/json" },
  timeout: 120000, // 2 minutes
});
```

```
export const generateSQL = (diagram, dialect) =>
  API.post("/generate-sql", { diagram, dialect }).then((r) => r.data.sql);

export const importSQL = (sql, dialect) =>
  API.post("/import-sql", { sql, dialect }).then((r) => r.data.diagram);

export const aiGenerateSchema = (prompt) =>
  API.post("/ai/generate-schema", { prompt }).then((r) => r.data.diagram);

export const aiAnalyzeSchema = (diagram, dialect) =>
  API.post("/ai/analyze-schema", { diagram, dialect }).then(
    (r) => r.data.analysis,
  );

export const aiChat = (messages, diagram, dialect) =>
  API.post("/ai/chat", { messages, diagram, dialect }).then(
    (r) => r.data.reply,
  );

export const shareDiagram = (diagram) =>
  API.post("/share", { diagram }).then((r) => r.data.shareId);

export const getShare = (shareId) =>
  API.get(`/share/${shareId}`).then((r) => r.data);

export default API;
```

```
============================================================
```

FILE: frontend\src\store\diagramStore.js

```
============================================================

import { create } from 'zustand';
import { applyNodeChanges, applyEdgeChanges } from 'reactflow';

const MAX_HISTORY = 50;
const uid = () => Math.random().toString(36).substr(2, 8);

const snap = (nodes, edges) => ({
  nodes: JSON.parse(JSON.stringify(nodes)),
  edges: JSON.parse(JSON.stringify(edges)),
});

const useDiagramStore = create((set, get) => ({
  nodes: [],
  edges: [],
  diagramId: null,
  diagramName: 'Untitled Diagram',
  createdAt: null,
  updatedAt: null,
  dialect: 'mysql',
  history: [],
  historyIndex: -1,

  // ReactFlow handlers
  onNodesChange: (changes) => set((s) => ({ nodes: applyNodeChanges(changes,
s.nodes) })),
  onEdgesChange: (changes) => set((s) => ({ edges: applyEdgeChanges(changes,
s.edges) })),
```

```
onConnect: (connection) => {
  const rel = {
    id: `r_${uid()}`,
    type: 'one-to-many',
    onDelete: 'RESTRICT',
    onUpdate: 'RESTRICT',
    label: '',
  };
  const newEdge = {
    id: rel.id,
    type: 'relationshipEdge',
    source: connection.source,
    sourceHandle: connection.sourceHandle,
    target: connection.target,
    targetHandle: connection.targetHandle,
    data: rel,
  };
  set((s) => ({ edges: [...s.edges, newEdge] }));
  get().pushHistory();
},

// Table ops
addTable: (position = { x: 200, y: 200 }) => {
  const { nodes } = get();
  const id = `t_${uid()}`;
  const colId = `c_${uid()}`;
  const num = nodes.length + 1;
  const table = {
    id, name: `Table${num}`, color: 'gray',
```

```
    columns: [{ id: colId, name: 'id', type: 'INT', primaryKey: true, autoIncrement: true, nullable: false, unique: true, defaultValue: '' }],
  };
  set((s) => ({ nodes: [...s.nodes, { id, type: 'tableNode', position, data: table }] }));
  get().pushHistory();
  return id;
 },

 updateTable: (tableId, updates) => {
  set((s) => ({ nodes: s.nodes.map(n => n.id === tableId ? { ...n, data: { ...n.data, ...updates } } : n) }));
  get().pushHistory();
 },

 deleteTable: (tableId) => {
  set((s) => ({
   nodes: s.nodes.filter(n => n.id !== tableId),
   edges: s.edges.filter(e => e.source !== tableId && e.target !== tableId),
  }));
  get().pushHistory();
 },

 duplicateTable: (tableId) => {
  const { nodes } = get();
  const orig = nodes.find(n => n.id === tableId);
  if (!orig) return;
  const newId = `t_${uid()}`;
  const newCols = orig.data.columns.map(c => ({ ...c, id: `c_${uid()}` }));
  const newTable = {
   id: newId,
```

```
      type: 'tableNode',
      position: { x: orig.position.x + 30, y: orig.position.y + 30 },
      data: { ...orig.data, id: newId, name: orig.data.name + '_copy', columns: newCols },
    };
    set((s) => ({ nodes: [...s.nodes, newTable] }));
    get().pushHistory();
  },

  addColumn: (tableId) => {
    const colId = `c_${uid()}`;
    set((s) => ({
      nodes: s.nodes.map(n => n.id === tableId
        ? { ...n, data: { ...n.data, columns: [...n.data.columns, { id: colId, name: 'new_column', type: 'VARCHAR(255)', primaryKey: false, autoIncrement: false, nullable: true, unique: false, defaultValue: '' }] } }
        : n),
    }));
    get().pushHistory();
    return colId;
  },

  updateColumn: (tableId, colId, updates) => {
    set((s) => ({
      nodes: s.nodes.map(n => n.id === tableId
        ? { ...n, data: { ...n.data, columns: n.data.columns.map(c => c.id === colId ? { ...c, ...updates } : c) } }
        : n),
    }));
    get().pushHistory();
```

```
  },

  deleteColumn: (tableId, colId) => {
    set((s) => ({
      nodes: s.nodes.map(n => n.id === tableId
        ? { ...n, data: { ...n.data, columns: n.data.columns.filter(c => c.id !== colId) } }
        : n),
      edges: s.edges.filter(e => e.sourceHandle !== `source-${colId}` &&
e.targetHandle !== `target-${colId}`),
    }));
    get().pushHistory();
  },

  updateRelationship: (edgeId, updates) => {
    set((s) => ({ edges: s.edges.map(e => e.id === edgeId ? { ...e, data: { ...e.data,
...updates } } : e) }));
    get().pushHistory();
  },

  deleteRelationship: (edgeId) => {
    set((s) => ({ edges: s.edges.filter(e => e.id !== edgeId) }));
    get().pushHistory();
  },

  // History
  pushHistory: () => {
    const { nodes, edges, history, historyIndex } = get();
    const newHist = [...history.slice(0, historyIndex + 1), snap(nodes, edges)].slice(-
MAX_HISTORY);
    set({ history: newHist, historyIndex: newHist.length - 1 });
```

```
},

undo: () => {
  const { history, historyIndex } = get();
  if (historyIndex > 0) {
    const prev = history[historyIndex - 1];
    set({ nodes: prev.nodes, edges: prev.edges, historyIndex: historyIndex - 1 });
  }
},

redo: () => {
  const { history, historyIndex } = get();
  if (historyIndex < history.length - 1) {
    const next = history[historyIndex + 1];
    set({ nodes: next.nodes, edges: next.edges, historyIndex: historyIndex + 1 });
  }
},

canUndo: () => get().historyIndex > 0,
canRedo: () => get().historyIndex < get().history.length - 1,

selectAll: () => {
  set((s) => ({ nodes: s.nodes.map((n) => ({ ...n, selected: true })) }));
},

createJunctionTable: (edgeId) => {
  const { nodes, edges } = get();
  const edge = edges.find((e) => e.id === edgeId);
  if (!edge) return null;
```

```
    const srcNode = nodes.find((n) => n.id === edge.source);
    const tgtNode = nodes.find((n) => n.id === edge.target);
    if (!srcNode || !tgtNode) return null;
    const srcPK = srcNode.data.columns.find((c) => c.primaryKey);
    const tgtPK = tgtNode.data.columns.find((c) => c.primaryKey);
    const juncId = `t_${uid()}`;
    const srcColId = `c_${uid()}`;
    const tgtColId = `c_${uid()}`;
    const juncName = `${srcNode.data.name}_${tgtNode.data.name}`;
    const juncNode = {
      id: juncId,
      type: 'tableNode',
      position: {
        x: (srcNode.position.x + tgtNode.position.x) / 2,
        y: Math.max(srcNode.position.y, tgtNode.position.y) + 220,
      },
      data: {
        id: juncId,
        name: juncName,
        color: 'gray',
        columns: [
          { id: srcColId, name: `${srcNode.data.name}_id`, type: (srcPK?.type ||
'INT').split('(')[0], primaryKey: true, autoIncrement: false, nullable: false, unique: false,
defaultValue: '' },
          { id: tgtColId, name: `${tgtNode.data.name}_id`, type: (tgtPK?.type ||
'INT').split('(')[0], primaryKey: true, autoIncrement: false, nullable: false, unique: false,
defaultValue: '' },
        ],
      },
    };
```

```
    const mkEdge = (srcCId, tgtNodeId, tgtPkId) => ({
      id: `r_${uid()}`,
      type: 'relationshipEdge',
      source: juncId,
      sourceHandle: `source-${srcCId}`,
      target: tgtNodeId,
      targetHandle: tgtPkId ? `target-${tgtPkId}` : `target-${srcCId}`,
      data: { id: `r_${uid()}`, type: 'one-to-many', onDelete: 'CASCADE', onUpdate:
'CASCADE', label: '' },
    });
    set((s) => ({
      nodes: [...s.nodes, juncNode],
      edges: [...s.edges.filter((e) => e.id !== edgeId), mkEdge(srcColId, srcNode.id,
srcPK?.id), mkEdge(tgtColId, tgtNode.id, tgtPK?.id)],
    }));
    get().pushHistory();
    return juncName;
  },

  // Diagram ops
  getDiagramJSON: () => {
    const { nodes, edges, diagramId, diagramName, createdAt, dialect } = get();
    return {
      id: diagramId || `d_${uid()}`,
      name: diagramName,
      dialect,
      createdAt: createdAt || new Date().toISOString(),
      updatedAt: new Date().toISOString(),
      tables: nodes.map(n => ({ ...n.data, position: n.position })),
      relationships: edges.map(e => ({
```

```
      id: e.id,
      sourceTableId: e.source,
      sourceColumnId: (e.sourceHandle || '').replace('source-', ''),
      targetTableId: e.target,
      targetColumnId: (e.targetHandle || '').replace('target-', ''),
      ...e.data,
    })),
  };
},

loadDiagram: (diagram) => {
  const nodes = (diagram.tables || []).map(t => ({
    id: t.id, type: 'tableNode',
    position: t.position || { x: 100 + Math.random() * 200, y: 100 + Math.random() * 200 },
    data: { id: t.id, name: t.name, color: t.color || 'gray', columns: t.columns || [] },
  }));
  const edges = (diagram.relationships || []).map(r => ({
    id: r.id, type: 'relationshipEdge',
    source: r.sourceTableId, sourceHandle: `source-${r.sourceColumnId}`,
    target: r.targetTableId, targetHandle: `target-${r.targetColumnId}`,
    data: { id: r.id, type: r.type || 'one-to-many', onDelete: r.onDelete || 'RESTRICT', onUpdate: r.onUpdate || 'RESTRICT', label: r.label || '' },
  }));
  set({
    nodes, edges,
    diagramId: diagram.id || null,
    diagramName: diagram.name || 'Untitled Diagram',
    dialect: diagram.dialect || 'mysql',
    createdAt: diagram.createdAt || new Date().toISOString(),
```

```
      updatedAt: diagram.updatedAt || null,
      history: [snap(nodes, edges)],
      historyIndex: 0,
    });
  },

  clearDiagram: () => set({ nodes: [], edges: [], diagramId: null, diagramName:
'Untitled Diagram', createdAt: null, updatedAt: null, history: [], historyIndex: -1 }),

  setDiagramName: (name) => set({ diagramName: name }),
  setDialect: (d) => set({ dialect: d }),
}));

export default useDiagramStore;

============================================================
FILE: frontend\src\store\uiStore.js
============================================================

import { create } from 'zustand';

const useUIStore = create((set) => ({
  activeTab: 'sql',
  theme: 'dark',
  showMinimap: true,
  showGrid: true,
  snapToGrid: false,
  selectedNodeId: null,
  selectedEdgeId: null,
```

```
  importSqlModalOpen: false,
  saveDiagramModalOpen: false,
  shareModalOpen: false,
  generatedSql: '',
  sqlLoading: false,

  setActiveTab: (tab) => set({ activeTab: tab }),
  toggleTheme: () => set((s) => ({ theme: s.theme === 'dark' ? 'light' : 'dark' })),
  toggleMinimap: () => set((s) => ({ showMinimap: !s.showMinimap })),
  toggleGrid: () => set((s) => ({ showGrid: !s.showGrid })),
  toggleSnapToGrid: () => set((s) => ({ snapToGrid: !s.snapToGrid })),
  setSelectedNode: (id) => set({ selectedNodeId: id, selectedEdgeId: null }),
  setSelectedEdge: (id) => set({ selectedEdgeId: id, selectedNodeId: null }),
  clearSelection: () => set({ selectedNodeId: null, selectedEdgeId: null }),
  openImportSqlModal: () => set({ importSqlModalOpen: true }),
  closeImportSqlModal: () => set({ importSqlModalOpen: false }),
  openSaveDiagramModal: () => set({ saveDiagramModalOpen: true }),
  closeSaveDiagramModal: () => set({ saveDiagramModalOpen: false }),
  openShareModal: () => set({ shareModalOpen: true }),
  closeShareModal: () => set({ shareModalOpen: false }),
  setGeneratedSql: (sql) => set({ generatedSql: sql }),
  setSqlLoading: (v) => set({ sqlLoading: v }),
}));

export default useUIStore;

============================================================
FILE: frontend\src\hooks\useAutoSave.js
============================================================
```

```
import { useEffect, useRef } from 'react';
import useDiagramStore from '../store/diagramStore';
import { saveToAutosave } from '../utils/persistence';

export default function useAutoSave() {
  const nodes = useDiagramStore((s) => s.nodes);
  const edges = useDiagramStore((s) => s.edges);
  const timer = useRef(null);

  useEffect(() => {
    clearTimeout(timer.current);
    timer.current = setTimeout(() => {
      const diagram = useDiagramStore.getState().getDiagramJSON();
      saveToAutosave(diagram);
    }, 2000);
    return () => clearTimeout(timer.current);
  }, [nodes, edges]);
}

============================================================
FILE: frontend\src\hooks\useKeyboardShortcuts.js
============================================================

import { useEffect } from 'react';
import useDiagramStore from '../store/diagramStore';
import useUIStore from '../store/uiStore';

export default function useKeyboardShortcuts() {
```

```
  useEffect(() => {
    const handler = (e) => {
      const inInput = e.target.tagName === 'INPUT' || e.target.tagName ===
'TEXTAREA' || e.target.isContentEditable;
      if (inInput) return;


      const { undo, redo, deleteTable, deleteRelationship, duplicateTable } =
useDiagramStore.getState();
      const { selectedNodeId, selectedEdgeId, openSaveDiagramModal,
clearSelection } = useUIStore.getState();


      if (e.ctrlKey || e.metaKey) {
        switch (e.key) {
          case 'z':
            e.shiftKey ? redo() : undo();
            e.preventDefault();
            break;
          case 'y':
            redo();
            e.preventDefault();
            break;
          case 's':
            openSaveDiagramModal();
            e.preventDefault();
            break;
          case 'd':
            if (selectedNodeId) duplicateTable(selectedNodeId);
            e.preventDefault();
            break;
          case 'a':
```

```
            useDiagramStore.getState().selectAll();
            e.preventDefault();
            break;
          default:
            break;
        }
      }

      if ((e.key === 'Delete' || e.key === 'Backspace') && !inInput) {
        if (selectedNodeId) deleteTable(selectedNodeId);
        if (selectedEdgeId) deleteRelationship(selectedEdgeId);
      }

      if (e.key === 'Escape') clearSelection();
    };

    window.addEventListener('keydown', handler);
    return () => window.removeEventListener('keydown', handler);
  }, []);
}

============================================================
FILE: frontend\src\utils\sqlGenerator.js
============================================================

// Client-side SQL generation (instant, no network)

const q = (name, dialect) => dialect === 'mysql' ? `\`${name}\`` : `"${name}"`;
```

```
const pgType = (type, autoInc) => {
  if (autoInc) {
    if (/BIGINT/i.test(type)) return 'BIGSERIAL';
    if (/INT/i.test(type)) return 'SERIAL';
  }
  if (/^DATETIME$/i.test(type)) return 'TIMESTAMP';
  if (/^TINYINT/i.test(type)) return 'BOOLEAN';
  return type;
};

const fmtDefault = (dv) => {
  const kw = ['NULL', 'CURRENT_TIMESTAMP', 'TRUE', 'FALSE', 'NOW()',
'CURRENT_DATE'];
  if (kw.includes(dv.toUpperCase()) || /^-?\d+(\.\d+)?$/.test(dv)) return dv;
  return `'${dv}'`;
};

function topoSort(tables, relationships) {
  const tmap = Object.fromEntries(tables.map(t => [t.id, t]));
  const deps = Object.fromEntries(tables.map(t => [t.id, new Set()]));
  for (const r of relationships) {
    if (r.type !== 'many-to-many' && deps[r.sourceTableId]) {
      deps[r.sourceTableId].add(r.targetTableId);
    }
  }
  const visited = new Set(), result = [];
  const dfs = (id) => {
    if (visited.has(id)) return;
    visited.add(id);
```

```
    for (const dep of (deps[id] || [])) dfs(dep);
    if (tmap[id]) result.push(tmap[id]);
  };
  tables.forEach(t => dfs(t.id));
  return result;
}

export function generateSQL(diagram, dialect = 'mysql') {
  if (!diagram || !diagram.tables || diagram.tables.length === 0) {
    return `-- No tables defined\n-- Add tables to the canvas to generate SQL`;
  }

  const { tables, relationships = [] } = diagram;
  const sorted = topoSort(tables, relationships);
  const tmap = Object.fromEntries(tables.map(t => [t.id, t]));
  const cmap = {};
  tables.forEach(t => t.columns?.forEach(c => { cmap[c.id] = c; }));

  const parts = dialect === 'mysql'
    ? ['-- Generated by SketchSQL\n-- Dialect: MySQL\n\nSET
FOREIGN_KEY_CHECKS=0;\n']
    : ['-- Generated by SketchSQL\n-- Dialect: PostgreSQL\n'];

  for (const table of sorted) {
    if (!table.columns?.length) continue;
    const colDefs = [], pks = [], uniq = [];

    for (const col of table.columns) {
      let colType = dialect === 'postgresql' ? pgType(col.type, col.autoIncrement &&
col.primaryKey) : col.type;
```

```
      let defn = `  ${q(col.name, dialect)} ${colType}`;
      if (dialect === 'mysql' && col.autoIncrement && col.primaryKey) defn += ' AUTO_INCREMENT';
      if (!col.nullable || col.primaryKey) defn += ' NOT NULL';
      if (col.unique && !col.primaryKey) {
        if (dialect === 'mysql') defn += ' UNIQUE';
        else uniq.push(col.name);
      }
      if (col.defaultValue) defn += ` DEFAULT ${fmtDefault(col.defaultValue)}`;
      colDefs.push(defn);
      if (col.primaryKey) pks.push(col.name);
    }
    if (pks.length) colDefs.push(`  PRIMARY KEY (${pks.map(p => q(p, dialect)).join(', ')})`);
    if (dialect === 'postgresql') uniq.forEach(u => colDefs.push(`  UNIQUE (${q(u, dialect)})`));

    const ending = dialect === 'mysql' ? ') ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;\n' : ');\n';
    parts.push(`CREATE TABLE ${q(table.name, dialect)} (\n${colDefs.join(',\n')}\n${ending}`);
  }

  // M:N junction tables
  for (const rel of relationships) {
    if (rel.type !== 'many-to-many') continue;
    const src = tmap[rel.sourceTableId], tgt = tmap[rel.targetTableId];
    const sc = cmap[rel.sourceColumnId], tc = cmap[rel.targetColumnId];
    if (!src || !tgt || !sc || !tc) continue;
    const jn = `${src.name}_${tgt.name}`;
    const sf = `${src.name}_id`, tf = `${tgt.name}_id`;
```

```
    const spk = src.columns.find(c => c.primaryKey);
    const tpk = tgt.columns.find(c => c.primaryKey);
    const st = pgType((spk?.type || 'INT').split('(')[0], false);
    const tt = pgType((tpk?.type || 'INT').split('(')[0], false);
    const od = rel.onDelete || 'CASCADE', ou = rel.onUpdate || 'CASCADE';
    const end = dialect === 'mysql' ? ') ENGINE=InnoDB DEFAULT
CHARSET=utf8mb4;\n' : ');\n';
    parts.push(
      `CREATE TABLE ${q(jn, dialect)} (\n` +
      `  ${q(sf, dialect)} ${st} NOT NULL,\n` +
      `  ${q(tf, dialect)} ${tt} NOT NULL,\n` +
      `  PRIMARY KEY (${q(sf, dialect)}, ${q(tf, dialect)}),\n` +
      `  FOREIGN KEY (${q(sf, dialect)}) REFERENCES ${q(src.name,
dialect)}(${q(sc.name, dialect)}) ON DELETE ${od} ON UPDATE ${ou},\n` +
      `  FOREIGN KEY (${q(tf, dialect)}) REFERENCES ${q(tgt.name,
dialect)}(${q(tc.name, dialect)}) ON DELETE ${od} ON UPDATE ${ou}\n` +
      end
    );
  }

  // FK constraints
  for (const rel of relationships) {
    if (rel.type === 'many-to-many') continue;
    const src = tmap[rel.sourceTableId], tgt = tmap[rel.targetTableId];
    const sc = cmap[rel.sourceColumnId], tc = cmap[rel.targetColumnId];
    if (!src || !tgt || !sc || !tc) continue;
    const od = rel.onDelete || 'RESTRICT', ou = rel.onUpdate || 'RESTRICT';
    const fk = `fk_${src.name}_${sc.name}`;
    const alter = dialect === 'mysql'
```

```
      ? `ALTER TABLE ${q(src.name, dialect)}\n  ADD CONSTRAINT ${q(fk, dialect)}\n FOREIGN KEY (${q(sc.name, dialect)})\n  REFERENCES ${q(tgt.name, dialect)}(${q(tc.name, dialect)})\n  ON DELETE ${od} ON UPDATE ${ou};\n`
      : `ALTER TABLE ONLY ${q(src.name, dialect)}\n  ADD CONSTRAINT ${q(fk, dialect)}\n  FOREIGN KEY (${q(sc.name, dialect)})\n  REFERENCES ${q(tgt.name, dialect)}(${q(tc.name, dialect)})\n  ON DELETE ${od} ON UPDATE ${ou};\n`;
    parts.push(alter);
  }

  if (dialect === 'mysql') parts.push('\nSET FOREIGN_KEY_CHECKS=1;');
  return parts.join('\n');
}

============================================================
FILE: frontend\src\utils\ormGenerator.js
============================================================

// ORM Code Generator — Django, Prisma, SQLAlchemy

function extractType(sqlType) {
  const m = sqlType.match(/^([A-Z]+)\(([^)]+)\)/i);
  return {
    base: (m ? m[1] : sqlType).toUpperCase(),
    params: m ? m[2].split(',').map((s) => s.trim()) : [],
  };
}

function toPascal(str) {
  return str.replace(/[_\s]+(.)/g, (_, c) => c.toUpperCase()).replace(/^[a-z]/, (c) => c.toUpperCase());
}
```

```
// ——— Django ————————————————————————————————————————————————————————————

function djangoField(col) {
  const { base, params } = extractType(col.type);
  const nu = col.nullable && !col.primaryKey ? ', null=True, blank=True' : '';
  const uq = col.unique && !col.primaryKey ? ', unique=True' : '';
  const dv = col.defaultValue && !col.defaultValue.includes('CURRENT') ? `, default='${col.defaultValue}'` : '';
  switch (base) {
    case 'INT': return `models.IntegerField(${nu}${uq}${dv})`;
    case 'BIGINT': return `models.BigIntegerField(${nu}${uq}${dv})`;
    case 'SMALLINT': case 'TINYINT': return `models.SmallIntegerField(${nu}${uq}${dv})`;
    case 'VARCHAR': return `models.CharField(max_length=${params[0] || 255}${nu}${uq}${dv})`;
    case 'TEXT': return `models.TextField(${nu}${uq}${dv})`;
    case 'BOOLEAN': return `models.BooleanField(default=False${uq})`;
    case 'DATE': return `models.DateField(${nu}${dv})`;
    case 'DATETIME': case 'TIMESTAMP':
      return col.defaultValue?.includes('CURRENT') ? 'models.DateTimeField(auto_now_add=True)' : `models.DateTimeField(${nu}${dv})`;
    case 'DECIMAL': return `models.DecimalField(max_digits=${params[0] || 10}, decimal_places=${params[1] || 2}${nu}${dv})`;
    case 'UUID': return `models.UUIDField(${col.primaryKey ? 'primary_key=True, default=uuid.uuid4, editable=False' : `${nu}${uq}`})`;
    case 'FLOAT': return `models.FloatField(${nu}${dv})`;
    case 'JSON': return `models.JSONField(${nu}${dv})`;
    default: return `models.TextField(${nu}${uq}${dv})`;
  }
```

```
}

function djangoOnDelete(a) {
  const m = { CASCADE: 'models.CASCADE', 'SET NULL': 'models.SET_NULL',
RESTRICT: 'models.RESTRICT', 'NO ACTION': 'models.RESTRICT' };
  return m[a] || 'models.RESTRICT';
}

export function generateDjango(diagram) {
  const { tables = [], relationships = [] } = diagram;
  if (!tables.length) return '# No tables defined\n# Add tables to the canvas to
generate Django models';
  const tmap = Object.fromEntries(tables.map((t) => [t.id, t]));
  const cmap = {};
  tables.forEach((t) => t.columns?.forEach((c) => { cmap[c.id] = c; }));
  const fkRels = (tid) => relationships.filter((r) => r.type !== 'many-to-many' &&
r.sourceTableId === tid);
  const m2mRels = (tid) => relationships.filter((r) => r.type === 'many-to-many' &&
r.sourceTableId === tid);
  const fkColIds = (tid) => new Set(fkRels(tid).map((r) => r.sourceColumnId));

  const lines = [
    '# Generated by SketchSQL',
    '# Django Models\n',
    'from django.db import models',
    'import uuid\n\n',
  ];

  for (const table of tables) {
    const model = toPascal(table.name);
```

```
    const fkCols = fkColIds(table.id);
    const fields = [];

    for (const col of table.columns || []) {
      if (col.primaryKey && col.autoIncrement && col.name === 'id') continue; // Django auto-id
      if (fkCols.has(col.id)) continue;
      fields.push(`    ${col.name} = ${djangoField(col)}`);
    }

    for (const rel of fkRels(table.id)) {
      const srcCol = cmap[rel.sourceColumnId];
      const tgt = tmap[rel.targetTableId];
      if (!srcCol || !tgt) continue;
      const fname = srcCol.name.endsWith('_id') ? srcCol.name.slice(0, -3) : srcCol.name;
      const RelClass = rel.type === 'one-to-one' ? 'OneToOneField' : 'ForeignKey';
      const nu = srcCol.nullable ? ', null=True, blank=True' : '';
      fields.push(`    ${fname} = models.${RelClass}('${toPascal(tgt.name)}', on_delete=${djangoOnDelete(rel.onDelete)}${nu})`);
    }

    for (const rel of m2mRels(table.id)) {
      const tgt = tmap[rel.targetTableId];
      if (!tgt) continue;
      fields.push(`    ${tgt.name.toLowerCase()}s = models.ManyToManyField('${toPascal(tgt.name)}')`);
    }

    if (!fields.length) fields.push('    pass');
```

```
    lines.push(`class ${model}(models.Model):`);
    lines.push(...fields);
    lines.push(`\n    class Meta:`);
    lines.push(`        db_table = '${table.name}'`);
    lines.push('\n');
  }

  return lines.join('\n');
}

// —— Prisma ——————————————————————————————————————————————————————————

function prismaType(col) {
  const { base } = extractType(col.type);
  const m = { INT: 'Int', BIGINT: 'BigInt', SMALLINT: 'Int', TINYINT: 'Int', VARCHAR: 'String', TEXT: 'String', BOOLEAN: 'Boolean', DATE: 'DateTime', DATETIME: 'DateTime', TIMESTAMP: 'DateTime', DECIMAL: 'Decimal', UUID: 'String', FLOAT: 'Float', JSON: 'Json' };
  return m[base] || 'String';
}

export function generatePrisma(diagram) {
  const { tables = [], relationships = [] } = diagram;
  if (!tables.length) return '// No tables defined\n// Add tables to the canvas to generate Prisma schema';
  const tmap = Object.fromEntries(tables.map((t) => [t.id, t]));
  const cmap = {};
  tables.forEach((t) => t.columns?.forEach((c) => { cmap[c.id] = c; }));
```

```
  const lines = [
    '// Generated by SketchSQL',
    '// Prisma Schema\n',
    'generator client {',
    '  provider = "prisma-client-js"',
    '}\n',
    'datasource db {',
    '  provider = "postgresql"',
    '  url      = env("DATABASE_URL")',
    '}\n',
  ];

  for (const table of tables) {
    const model = toPascal(table.name);
    const fkRelsForTable = relationships.filter((r) => r.type !== 'many-to-many' &&
r.sourceTableId === table.id);
    const m2mRelsForTable = relationships.filter((r) => r.type === 'many-to-many' &&
r.sourceTableId === table.id);
    const fkColIds = new Set(fkRelsForTable.map((r) => r.sourceColumnId));
    const backRels = relationships.filter((r) => r.type !== 'many-to-many' &&
r.targetTableId === table.id);

    const fields = [];

    for (const col of table.columns || []) {
      if (fkColIds.has(col.id)) continue;
      const pt = prismaType(col);
      const opt = col.nullable && !col.primaryKey ? '?' : '';
      const attrs = [];
      if (col.primaryKey) attrs.push('@id');
```

```
      if (col.autoIncrement) attrs.push('@default(autoincrement())');
      else if (col.defaultValue?.includes('CURRENT')) attrs.push('@default(now())');
      else if (col.defaultValue) attrs.push(`@default(${col.defaultValue})`);
      if (col.unique && !col.primaryKey) attrs.push('@unique');
      fields.push(`  ${col.name}  ${pt}${opt}  ${attrs.join(' ')}`);
    }

    for (const rel of fkRelsForTable) {
      const srcCol = cmap[rel.sourceColumnId];
      const tgt = tmap[rel.targetTableId];
      const tgtCol = cmap[rel.targetColumnId];
      if (!srcCol || !tgt || !tgtCol) continue;
      const pt = prismaType(srcCol);
      const opt = srcCol.nullable ? '?' : '';
      const relField = tgt.name.toLowerCase();
      const tgtModel = toPascal(tgt.name);
      fields.push(`  ${srcCol.name}  ${pt}${opt}`);
      fields.push(`  ${relField}  ${tgtModel}${opt}  @relation(fields: [${srcCol.name}],
references: [${tgtCol.name}])`);
    }

    for (const rel of backRels) {
      const src = tmap[rel.sourceTableId];
      if (!src) continue;
      const srcModel = toPascal(src.name);
      const fname = src.name.toLowerCase() + (rel.type !== 'one-to-one' ? 's' : '');
      fields.push(`  ${fname}  ${rel.type === 'one-to-one' ? `${srcModel}?` :
`${srcModel}[]`}`);
    }
```

```
    for (const rel of m2mRelsForTable) {
      const tgt = tmap[rel.targetTableId];
      if (!tgt) continue;
      fields.push(`  ${tgt.name.toLowerCase()}s  ${toPascal(tgt.name)}[]`);
    }

    lines.push(`model ${model} {`);
    lines.push(...fields);
    lines.push(`\n  @@map("${table.name}")`);
    lines.push('}\n');
  }

  return lines.join('\n');
}

// ——— SQLAlchemy ———————————————————————————————————————————————————————

function saType(sqlType) {
  const { base, params } = extractType(sqlType);
  const m = { INT: 'Integer', BIGINT: 'BigInteger', SMALLINT: 'SmallInteger', TINYINT: 'SmallInteger', VARCHAR: `String(${params[0] || 255})`, TEXT: 'Text', BOOLEAN: 'Boolean', DATE: 'Date', DATETIME: 'DateTime', TIMESTAMP: 'DateTime', DECIMAL: `Numeric(${params.join(', ') || '10, 2'})`, UUID: 'UUID(as_uuid=True)', FLOAT: 'Float', JSON: 'JSON' };
  return m[base] || 'String';
}

export function generateSQLAlchemy(diagram) {
  const { tables = [], relationships = [] } = diagram;
```

```
  if (!tables.length) return '# No tables defined\n# Add tables to the canvas to generate SQLAlchemy models';
  const tmap = Object.fromEntries(tables.map((t) => [t.id, t]));
  const cmap = {};
  tables.forEach((t) => t.columns?.forEach((c) => { cmap[c.id] = c; }));

  const lines = [
    '# Generated by SketchSQL',
    '# SQLAlchemy Models\n',
    'from sqlalchemy import Column, Integer, BigInteger, SmallInteger, String, Text,',
    '    Boolean, Float, Numeric, Date, DateTime, JSON, ForeignKey',
    'from sqlalchemy.dialects.postgresql import UUID',
    'from sqlalchemy.orm import relationship, DeclarativeBase',
    'import uuid\n\n',
    'class Base(DeclarativeBase):',
    '    pass\n\n',
  ];

  for (const table of tables) {
    const model = toPascal(table.name);
    const fkRelsForTable = relationships.filter((r) => r.type !== 'many-to-many' && r.sourceTableId === table.id);
    const fkColIds = new Set(fkRelsForTable.map((r) => r.sourceColumnId));

    const fields = [`    __tablename__ = '${table.name}'\n`];

    for (const col of table.columns || []) {
      if (fkColIds.has(col.id)) continue;
      const t = saType(col.type);
      const attrs = [];
```

```
      if (col.primaryKey) attrs.push('primary_key=True');
      if (col.autoIncrement) attrs.push('autoincrement=True');
      if (!col.nullable && !col.primaryKey) attrs.push('nullable=False');
      if (col.unique && !col.primaryKey) attrs.push('unique=True');
      if (col.defaultValue && !col.defaultValue.includes('CURRENT')) attrs.push(`default='${col.defaultValue}'`);
      const attrsStr = attrs.length ? ', ' + attrs.join(', ') : '';
      fields.push(`    ${col.name} = Column(${t}${attrsStr})`);
    }

    for (const rel of fkRelsForTable) {
      const srcCol = cmap[rel.sourceColumnId];
      const tgt = tmap[rel.targetTableId];
      const tgtPK = tgt?.columns.find((c) => c.primaryKey);
      if (!srcCol || !tgt) continue;
      const t = saType(srcCol.type);
      const nu = srcCol.nullable ? '' : ', nullable=False';
      const onDel = rel.onDelete === 'CASCADE' ? ', ondelete="CASCADE"' : '';
      fields.push(`    ${srcCol.name} = Column(${t}, ForeignKey('${tgt.name}.${tgtPK?.name || 'id'}'${onDel})${nu})`);
      const relField = tgt.name.toLowerCase();
      fields.push(`    ${relField} = relationship('${toPascal(tgt.name)}', back_populates='${table.name.toLowerCase()}s')`);
    }

    lines.push(`class ${model}(Base):`);
    lines.push(...fields);
    lines.push('\n');
  }
```

```
  return lines.join('\n');
}

============================================================
FILE: frontend\src\utils\diagramLayout.js
============================================================

// Auto-layout algorithm: layered left-to-right based on FK dependencies

export function autoLayout(tables, relationships = [], startX = 80, startY = 80) {
  if (!tables.length) return tables;

  const COL_GAP = 320;
  const ROW_GAP = 220;
  const COLS = Math.max(1, Math.ceil(Math.sqrt(tables.length)));

  // Try dependency-based layering
  const tmap = Object.fromEntries(tables.map(t => [t.id, t]));
  const deps = Object.fromEntries(tables.map(t => [t.id, new Set()]));

  for (const r of relationships) {
    if (r.type !== 'many-to-many' && deps[r.sourceTableId]) {
      deps[r.sourceTableId].add(r.targetTableId);
    }
  }

  // Assign layers
  const layers = {};
  const visited = new Set();
```

```
const assignLayer = (id, layer) => {
  if (visited.has(id)) return;
  visited.add(id);
  layers[id] = Math.max(layers[id] || 0, layer);
  // Children get layer + 1
  for (const tid of Object.keys(deps)) {
    if (deps[tid].has(id)) assignLayer(tid, layer + 1);
  }
};

// Find roots (tables not referenced by others)
const referenced = new Set(relationships.map(r => r.targetTableId));
const roots = tables.filter(t => !referenced.has(t.id));
if (!roots.length) roots.push(tables[0]);

roots.forEach(t => assignLayer(t.id, 0));
tables.forEach(t => { if (layers[t.id] === undefined) layers[t.id] = 0; });

// Group by layer
const layerGroups = {};
for (const [id, layer] of Object.entries(layers)) {
  if (!layerGroups[layer]) layerGroups[layer] = [];
  layerGroups[layer].push(id);
}

// If too many layers (>5), fall back to grid
const maxLayer = Math.max(...Object.values(layers));
if (maxLayer > 4 || Object.keys(layerGroups).length < 2) {
```

```
    return tables.map((t, i) => ({
      ...t,
      position: {
        x: startX + (i % COLS) * COL_GAP,
        y: startY + Math.floor(i / COLS) * ROW_GAP,
      },
    }));
  }

  // Layered layout
  const positions = {};
  for (const [layer, ids] of Object.entries(layerGroups)) {
    ids.forEach((id, idx) => {
      positions[id] = {
        x: startX + parseInt(layer) * COL_GAP,
        y: startY + idx * ROW_GAP,
      };
    });
  }

  return tables.map(t => ({ ...t, position: positions[t.id] || { x: startX, y: startY } }));
}

============================================================
FILE: frontend\src\utils\exportUtils.js
============================================================

import { toPng } from 'html-to-image';
import jsPDF from 'jspdf';
```

```
export function downloadFile(content, filename, mimeType) {
  const blob = new Blob([content], { type: mimeType });
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = filename;
  a.click();
  URL.revokeObjectURL(url);
}

export function exportSQL(sql, diagramName = 'schema') {
  downloadFile(sql, `${diagramName.replace(/\s+/g, '_')}.sql`, 'text/plain');
}

export function exportJSON(diagram) {
  const name = diagram.name || 'diagram';
  downloadFile(JSON.stringify(diagram, null, 2), `${name.replace(/\s+/g, '_')}.json`,
'application/json');
}

export async function exportPNG(elementSelector = '.react-flow', diagramName =
'schema') {
  const el = document.querySelector(elementSelector);
  if (!el) return;
  try {
    const dataUrl = await toPng(el, {
      backgroundColor: '#0d111a',
      quality: 0.95,
      pixelRatio: 2,
```

```
    });
    const a = document.createElement('a');
    a.href = dataUrl;
    a.download = `${diagramName.replace(/\s+/g, '_')}.png`;
    a.click();
  } catch (e) {
    console.error('PNG export failed', e);
    throw e;
  }
}

export async function exportPDF(elementSelector = '.react-flow', diagramName =
'schema') {
  const el = document.querySelector(elementSelector);
  if (!el) return;
  try {
    const dataUrl = await toPng(el, { backgroundColor: '#0d111a', pixelRatio: 1.5 });
    const img = new Image();
    img.src = dataUrl;
    await new Promise(r => { img.onload = r; });
    const pdf = new jsPDF({
      orientation: img.width > img.height ? 'landscape' : 'portrait',
      unit: 'px',
      format: [img.width, img.height],
    });
    pdf.addImage(dataUrl, 'PNG', 0, 0, img.width, img.height);
    pdf.save(`${diagramName.replace(/\s+/g, '_')}.pdf`);
  } catch (e) {
    console.error('PDF export failed', e);
```

```
    throw e;
  }
}

============================================================
FILE: frontend\src\utils\persistence.js
============================================================

// localStorage + IndexedDB persistence

const AUTOSAVE_KEY = 'sketchsql_autosave';
const DB_NAME = 'SketchSQLDB';
const STORE_NAME = 'diagrams';
const DB_VERSION = 1;

// ——— localStorage autosave ———————————————————————————————

export function saveToAutosave(diagram) {
  try {
    localStorage.setItem(AUTOSAVE_KEY, JSON.stringify(diagram));
  } catch (e) {
    console.warn('Autosave failed', e);
  }
}

export function loadFromAutosave() {
  try {
    const raw = localStorage.getItem(AUTOSAVE_KEY);
```

```
    return raw ? JSON.parse(raw) : null;
  } catch {
    return null;
  }
}

export function clearAutosave() {
  localStorage.removeItem(AUTOSAVE_KEY);
}

// ——— IndexedDB named diagrams ———————————————————————————

function openDB() {
  return new Promise((resolve, reject) => {
    const req = indexedDB.open(DB_NAME, DB_VERSION);
    req.onupgradeneeded = (e) => {
      const db = e.target.result;
      if (!db.objectStoreNames.contains(STORE_NAME)) {
        const store = db.createObjectStore(STORE_NAME, { keyPath: 'id' });
        store.createIndex('updatedAt', 'updatedAt');
      }
    };
    req.onsuccess = () => resolve(req.result);
    req.onerror = () => reject(req.error);
  });
}

export async function saveDiagram(diagram) {
```

```
  const db = await openDB();
  return new Promise((resolve, reject) => {
    const tx = db.transaction(STORE_NAME, 'readwrite');
    tx.objectStore(STORE_NAME).put({ ...diagram, updatedAt: new
Date().toISOString() });
    tx.oncomplete = () => resolve();
    tx.onerror = () => reject(tx.error);
  });
}

export async function loadAllDiagrams() {
  try {
    const db = await openDB();
    return new Promise((resolve, reject) => {
      const tx = db.transaction(STORE_NAME, 'readonly');
      const req = tx.objectStore(STORE_NAME).getAll();
      req.onsuccess = () => resolve(req.result.sort((a, b) => (b.updatedAt ||
'').localeCompare(a.updatedAt || '')));
      req.onerror = () => reject(req.error);
    });
  } catch {
    return [];
  }
}

export async function loadDiagram(id) {
  const db = await openDB();
  return new Promise((resolve, reject) => {
    const req = db.transaction(STORE_NAME,
'readonly').objectStore(STORE_NAME).get(id);
```

```
    req.onsuccess = () => resolve(req.result);
    req.onerror = () => reject(req.error);
  });
}

export async function deleteDiagram(id) {
  const db = await openDB();
  return new Promise((resolve, reject) => {
    const tx = db.transaction(STORE_NAME, 'readwrite');
    tx.objectStore(STORE_NAME).delete(id);
    tx.oncomplete = () => resolve();
    tx.onerror = () => reject(tx.error);
  });
}

export async function renameDiagram(id, newName) {
  const db = await openDB();
  return new Promise((resolve, reject) => {
    const tx = db.transaction(STORE_NAME, 'readwrite');
    const store = tx.objectStore(STORE_NAME);
    const req = store.get(id);
    req.onsuccess = () => {
      if (req.result) {
        store.put({ ...req.result, name: newName, updatedAt: new Date().toISOString()
});
      }
    };
    tx.oncomplete = () => resolve();
    tx.onerror = () => reject(tx.error);
```

```
  });
}
```

```
============================================================
FILE: frontend\src\utils\exampleSchemas.js
============================================================
```

```
export const EXAMPLE_SCHEMAS = {
  ecommerce: {
    id: 'example-ecommerce',
    name: 'E-Commerce Store',
    dialect: 'mysql',
    tables: [
      { id: 't_ec_users', name: 'users', color: 'blue', position: { x: 80, y: 80 }, columns: [
        { id: 'c_eu_id', name: 'id', type: 'INT', primaryKey: true, autoIncrement: true, nullable: false, unique: true, defaultValue: '' },
        { id: 'c_eu_email', name: 'email', type: 'VARCHAR(255)', primaryKey: false, autoIncrement: false, nullable: false, unique: true, defaultValue: '' },
        { id: 'c_eu_name', name: 'name', type: 'VARCHAR(255)', primaryKey: false, autoIncrement: false, nullable: true, unique: false, defaultValue: '' },
        { id: 'c_eu_created', name: 'created_at', type: 'TIMESTAMP', primaryKey: false, autoIncrement: false, nullable: true, unique: false, defaultValue: 'CURRENT_TIMESTAMP' },
      ]},
      { id: 't_ec_categories', name: 'categories', color: 'green', position: { x: 80, y: 340 }, columns: [
        { id: 'c_ec_id', name: 'id', type: 'INT', primaryKey: true, autoIncrement: true, nullable: false, unique: true, defaultValue: '' },
        { id: 'c_ec_name', name: 'name', type: 'VARCHAR(255)', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },
      ]},
```

{ id: 't_ec_products', name: 'products', color: 'purple', position: { x: 460, y: 80 }, columns: [

{ id: 'c_ep_id', name: 'id', type: 'INT', primaryKey: true, autoIncrement: true, nullable: false, unique: true, defaultValue: '' },

{ id: 'c_ep_cat', name: 'category_id', type: 'INT', primaryKey: false, autoIncrement: false, nullable: true, unique: false, defaultValue: '' },

{ id: 'c_ep_name', name: 'name', type: 'VARCHAR(255)', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },

{ id: 'c_ep_price', name: 'price', type: 'DECIMAL(10,2)', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },

{ id: 'c_ep_stock', name: 'stock', type: 'INT', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '0' },

]},

{ id: 't_ec_orders', name: 'orders', color: 'orange', position: { x: 460, y: 380 }, columns: [

{ id: 'c_eo_id', name: 'id', type: 'INT', primaryKey: true, autoIncrement: true, nullable: false, unique: true, defaultValue: '' },

{ id: 'c_eo_uid', name: 'user_id', type: 'INT', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },

{ id: 'c_eo_status', name: 'status', type: 'VARCHAR(50)', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: 'pending' },

{ id: 'c_eo_total', name: 'total', type: 'DECIMAL(10,2)', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },

{ id: 'c_eo_created', name: 'created_at', type: 'TIMESTAMP', primaryKey: false, autoIncrement: false, nullable: true, unique: false, defaultValue: 'CURRENT_TIMESTAMP' },

]},

{ id: 't_ec_order_items', name: 'order_items', color: 'red', position: { x: 840, y: 380 }, columns: [

{ id: 'c_ei_id', name: 'id', type: 'INT', primaryKey: true, autoIncrement: true, nullable: false, unique: true, defaultValue: '' },

{ id: 'c_ei_oid', name: 'order_id', type: 'INT', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },

{ id: 'c_ei_pid', name: 'product_id', type: 'INT', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },

{ id: 'c_ei_qty', name: 'quantity', type: 'INT', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '1' },

{ id: 'c_ei_price', name: 'unit_price', type: 'DECIMAL(10,2)', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },

]},

{ id: 't_ec_payments', name: 'payments', color: 'gray', position: { x: 840, y: 80 }, columns: [

{ id: 'c_epay_id', name: 'id', type: 'INT', primaryKey: true, autoIncrement: true, nullable: false, unique: true, defaultValue: '' },

{ id: 'c_epay_oid', name: 'order_id', type: 'INT', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },

{ id: 'c_epay_method', name: 'method', type: 'VARCHAR(50)', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },

{ id: 'c_epay_amount', name: 'amount', type: 'DECIMAL(10,2)', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },

{ id: 'c_epay_status', name: 'status', type: 'VARCHAR(50)', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: 'pending' },

]},

],

relationships: [

{ id: 'r_ec1', sourceTableId: 't_ec_products', sourceColumnId: 'c_ep_cat', targetTableId: 't_ec_categories', targetColumnId: 'c_ec_id', type: 'one-to-many', onDelete: 'SET NULL', onUpdate: 'CASCADE', label: '' },

{ id: 'r_ec2', sourceTableId: 't_ec_orders', sourceColumnId: 'c_eo_uid', targetTableId: 't_ec_users', targetColumnId: 'c_eu_id', type: 'one-to-many', onDelete: 'CASCADE', onUpdate: 'CASCADE', label: '' },

{ id: 'r_ec3', sourceTableId: 't_ec_order_items', sourceColumnId: 'c_ei_oid', targetTableId: 't_ec_orders', targetColumnId: 'c_eo_id', type: 'one-to-many', onDelete: 'CASCADE', onUpdate: 'CASCADE', label: '' },

{ id: 'r_ec4', sourceTableId: 't_ec_order_items', sourceColumnId: 'c_ei_pid', targetTableId: 't_ec_products', targetColumnId: 'c_ep_id', type: 'one-to-many', onDelete: 'RESTRICT', onUpdate: 'CASCADE', label: '' },

{ id: 'r_ec5', sourceTableId: 't_ec_payments', sourceColumnId: 'c_epay_oid', targetTableId: 't_ec_orders', targetColumnId: 'c_eo_id', type: 'one-to-one', onDelete: 'CASCADE', onUpdate: 'CASCADE', label: '' },

```
    ],
  },


  blog: {
    id: 'example-blog',
    name: 'Blog Platform',
    dialect: 'mysql',
    tables: [
      { id: 't_bl_users', name: 'users', color: 'blue', position: { x: 80, y: 80 }, columns: [
        { id: 'c_bu_id', name: 'id', type: 'INT', primaryKey: true, autoIncrement: true, nullable: false, unique: true, defaultValue: '' },
        { id: 'c_bu_email', name: 'email', type: 'VARCHAR(255)', primaryKey: false, autoIncrement: false, nullable: false, unique: true, defaultValue: '' },
        { id: 'c_bu_username', name: 'username', type: 'VARCHAR(100)', primaryKey: false, autoIncrement: false, nullable: false, unique: true, defaultValue: '' },
      ]},
      { id: 't_bl_posts', name: 'posts', color: 'purple', position: { x: 460, y: 80 }, columns: [
        { id: 'c_bp_id', name: 'id', type: 'INT', primaryKey: true, autoIncrement: true, nullable: false, unique: true, defaultValue: '' },
        { id: 'c_bp_uid', name: 'author_id', type: 'INT', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },
        { id: 'c_bp_title', name: 'title', type: 'VARCHAR(255)', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },
        { id: 'c_bp_body', name: 'body', type: 'TEXT', primaryKey: false, autoIncrement: false, nullable: true, unique: false, defaultValue: '' },
        { id: 'c_bp_pub', name: 'published_at', type: 'TIMESTAMP', primaryKey: false, autoIncrement: false, nullable: true, unique: false, defaultValue: '' },
      ]},
      { id: 't_bl_comments', name: 'comments', color: 'green', position: { x: 840, y: 80 }, columns: [
        { id: 'c_bc_id', name: 'id', type: 'INT', primaryKey: true, autoIncrement: true, nullable: false, unique: true, defaultValue: '' },
```

```
      { id: 'c_bc_pid', name: 'post_id', type: 'INT', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },
      { id: 'c_bc_uid', name: 'user_id', type: 'INT', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },
      { id: 'c_bc_body', name: 'body', type: 'TEXT', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },
    ]},
    { id: 't_bl_tags', name: 'tags', color: 'orange', position: { x: 80, y: 360 }, columns: [
      { id: 'c_bt_id', name: 'id', type: 'INT', primaryKey: true, autoIncrement: true, nullable: false, unique: true, defaultValue: '' },
      { id: 'c_bt_name', name: 'name', type: 'VARCHAR(100)', primaryKey: false, autoIncrement: false, nullable: false, unique: true, defaultValue: '' },
    ]},
    { id: 't_bl_post_tags', name: 'post_tags', color: 'gray', position: { x: 460, y: 360 }, columns: [
      { id: 'c_bpt_pid', name: 'post_id', type: 'INT', primaryKey: true, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },
      { id: 'c_bpt_tid', name: 'tag_id', type: 'INT', primaryKey: true, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },
    ]},
  ],
  relationships: [
    { id: 'r_bl1', sourceTableId: 't_bl_posts', sourceColumnId: 'c_bp_uid', targetTableId: 't_bl_users', targetColumnId: 'c_bu_id', type: 'one-to-many', onDelete: 'CASCADE', onUpdate: 'CASCADE', label: '' },
    { id: 'r_bl2', sourceTableId: 't_bl_comments', sourceColumnId: 'c_bc_pid', targetTableId: 't_bl_posts', targetColumnId: 'c_bp_id', type: 'one-to-many', onDelete: 'CASCADE', onUpdate: 'CASCADE', label: '' },
    { id: 'r_bl3', sourceTableId: 't_bl_comments', sourceColumnId: 'c_bc_uid', targetTableId: 't_bl_users', targetColumnId: 'c_bu_id', type: 'one-to-many', onDelete: 'CASCADE', onUpdate: 'CASCADE', label: '' },
    { id: 'r_bl4', sourceTableId: 't_bl_post_tags', sourceColumnId: 'c_bpt_pid', targetTableId: 't_bl_posts', targetColumnId: 'c_bp_id', type: 'one-to-many', onDelete: 'CASCADE', onUpdate: 'CASCADE', label: '' },
```

```
    { id: 'r_bl5', sourceTableId: 't_bl_post_tags', sourceColumnId: 'c_bpt_tid', targetTableId: 't_bl_tags', targetColumnId: 'c_bt_id', type: 'one-to-many', onDelete: 'CASCADE', onUpdate: 'CASCADE', label: '' },
  ],
},

university: {
  id: 'example-university',
  name: 'University System',
  dialect: 'mysql',
  tables: [
    { id: 't_un_depts', name: 'departments', color: 'blue', position: { x: 80, y: 80 }, columns: [
      { id: 'c_ud_id', name: 'id', type: 'INT', primaryKey: true, autoIncrement: true, nullable: false, unique: true, defaultValue: '' },
      { id: 'c_ud_name', name: 'name', type: 'VARCHAR(255)', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },
    ]},
    { id: 't_un_instructors', name: 'instructors', color: 'green', position: { x: 80, y: 320 }, columns: [
      { id: 'c_ui_id', name: 'id', type: 'INT', primaryKey: true, autoIncrement: true, nullable: false, unique: true, defaultValue: '' },
      { id: 'c_ui_did', name: 'department_id', type: 'INT', primaryKey: false, autoIncrement: false, nullable: true, unique: false, defaultValue: '' },
      { id: 'c_ui_name', name: 'name', type: 'VARCHAR(255)', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },
      { id: 'c_ui_email', name: 'email', type: 'VARCHAR(255)', primaryKey: false, autoIncrement: false, nullable: false, unique: true, defaultValue: '' },
    ]},
    { id: 't_un_courses', name: 'courses', color: 'purple', position: { x: 460, y: 80 }, columns: [
      { id: 'c_uc_id', name: 'id', type: 'INT', primaryKey: true, autoIncrement: true, nullable: false, unique: true, defaultValue: '' },
```

{ id: 'c_uc_did', name: 'department_id', type: 'INT', primaryKey: false, autoIncrement: false, nullable: true, unique: false, defaultValue: '' },

{ id: 'c_uc_iid', name: 'instructor_id', type: 'INT', primaryKey: false, autoIncrement: false, nullable: true, unique: false, defaultValue: '' },

{ id: 'c_uc_title', name: 'title', type: 'VARCHAR(255)', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },

{ id: 'c_uc_credits', name: 'credits', type: 'INT', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '3' },

]},

{ id: 't_un_students', name: 'students', color: 'orange', position: { x: 460, y: 340 }, columns: [

{ id: 'c_us_id', name: 'id', type: 'INT', primaryKey: true, autoIncrement: true, nullable: false, unique: true, defaultValue: '' },

{ id: 'c_us_name', name: 'name', type: 'VARCHAR(255)', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },

{ id: 'c_us_email', name: 'email', type: 'VARCHAR(255)', primaryKey: false, autoIncrement: false, nullable: false, unique: true, defaultValue: '' },

]},

{ id: 't_un_enrollments', name: 'enrollments', color: 'gray', position: { x: 840, y: 200 }, columns: [

{ id: 'c_ue_id', name: 'id', type: 'INT', primaryKey: true, autoIncrement: true, nullable: false, unique: true, defaultValue: '' },

{ id: 'c_ue_sid', name: 'student_id', type: 'INT', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },

{ id: 'c_ue_cid', name: 'course_id', type: 'INT', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },

{ id: 'c_ue_grade', name: 'grade', type: 'VARCHAR(5)', primaryKey: false, autoIncrement: false, nullable: true, unique: false, defaultValue: '' },

]},

],

relationships: [

{ id: 'r_un1', sourceTableId: 't_un_instructors', sourceColumnId: 'c_ui_did', targetTableId: 't_un_depts', targetColumnId: 'c_ud_id', type: 'one-to-many', onDelete: 'SET NULL', onUpdate: 'CASCADE', label: '' },

```
    { id: 'r_un2', sourceTableId: 't_un_courses', sourceColumnId: 'c_uc_did', targetTableId: 't_un_depts', targetColumnId: 'c_ud_id', type: 'one-to-many', onDelete: 'SET NULL', onUpdate: 'CASCADE', label: '' },

    { id: 'r_un3', sourceTableId: 't_un_courses', sourceColumnId: 'c_uc_iid', targetTableId: 't_un_instructors', targetColumnId: 'c_ui_id', type: 'one-to-many', onDelete: 'SET NULL', onUpdate: 'CASCADE', label: '' },

    { id: 'r_un4', sourceTableId: 't_un_enrollments', sourceColumnId: 'c_ue_sid', targetTableId: 't_un_students', targetColumnId: 'c_us_id', type: 'one-to-many', onDelete: 'CASCADE', onUpdate: 'CASCADE', label: '' },

    { id: 'r_un5', sourceTableId: 't_un_enrollments', sourceColumnId: 'c_ue_cid', targetTableId: 't_un_courses', targetColumnId: 'c_uc_id', type: 'one-to-many', onDelete: 'CASCADE', onUpdate: 'CASCADE', label: '' },

   ],

 },


 hospital: {

   id: 'example-hospital',

   name: 'Hospital Management',

   dialect: 'mysql',

   tables: [

    { id: 't_ho_depts', name: 'departments', color: 'blue', position: { x: 80, y: 80 }, columns: [

      { id: 'c_hd_id', name: 'id', type: 'INT', primaryKey: true, autoIncrement: true, nullable: false, unique: true, defaultValue: '' },

      { id: 'c_hd_name', name: 'name', type: 'VARCHAR(255)', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },

    ]},

    { id: 't_ho_doctors', name: 'doctors', color: 'green', position: { x: 80, y: 300 }, columns: [

      { id: 'c_hdr_id', name: 'id', type: 'INT', primaryKey: true, autoIncrement: true, nullable: false, unique: true, defaultValue: '' },

      { id: 'c_hdr_did', name: 'department_id', type: 'INT', primaryKey: false, autoIncrement: false, nullable: true, unique: false, defaultValue: '' },
```

```
        { id: 'c_hdr_name', name: 'name', type: 'VARCHAR(255)', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },
        { id: 'c_hdr_spec', name: 'specialization', type: 'VARCHAR(255)', primaryKey: false, autoIncrement: false, nullable: true, unique: false, defaultValue: '' },
      ]},
      { id: 't_ho_patients', name: 'patients', color: 'purple', position: { x: 460, y: 80 }, columns: [
        { id: 'c_hp_id', name: 'id', type: 'INT', primaryKey: true, autoIncrement: true, nullable: false, unique: true, defaultValue: '' },
        { id: 'c_hp_name', name: 'name', type: 'VARCHAR(255)', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },
        { id: 'c_hp_dob', name: 'date_of_birth', type: 'DATE', primaryKey: false, autoIncrement: false, nullable: true, unique: false, defaultValue: '' },
        { id: 'c_hp_email', name: 'email', type: 'VARCHAR(255)', primaryKey: false, autoIncrement: false, nullable: true, unique: true, defaultValue: '' },
      ]},
      { id: 't_ho_appointments', name: 'appointments', color: 'orange', position: { x: 460, y: 360 }, columns: [
        { id: 'c_ha_id', name: 'id', type: 'INT', primaryKey: true, autoIncrement: true, nullable: false, unique: true, defaultValue: '' },
        { id: 'c_ha_pid', name: 'patient_id', type: 'INT', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },
        { id: 'c_ha_did', name: 'doctor_id', type: 'INT', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },
        { id: 'c_ha_dt', name: 'appointment_date', type: 'DATETIME', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },
        { id: 'c_ha_status', name: 'status', type: 'VARCHAR(50)', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: 'scheduled' },
      ]},
      { id: 't_ho_medications', name: 'medications', color: 'gray', position: { x: 840, y: 80 }, columns: [
        { id: 'c_hm_id', name: 'id', type: 'INT', primaryKey: true, autoIncrement: true, nullable: false, unique: true, defaultValue: '' },
```

{ id: 'c_hm_name', name: 'name', type: 'VARCHAR(255)', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },

{ id: 'c_hm_dosage', name: 'dosage', type: 'VARCHAR(100)', primaryKey: false, autoIncrement: false, nullable: true, unique: false, defaultValue: '' },

]},

{ id: 't_ho_prescriptions', name: 'prescriptions', color: 'red', position: { x: 840, y: 360 }, columns: [

{ id: 'c_hpr_id', name: 'id', type: 'INT', primaryKey: true, autoIncrement: true, nullable: false, unique: true, defaultValue: '' },

{ id: 'c_hpr_aid', name: 'appointment_id', type: 'INT', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },

{ id: 'c_hpr_mid', name: 'medication_id', type: 'INT', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '' },

{ id: 'c_hpr_qty', name: 'quantity', type: 'INT', primaryKey: false, autoIncrement: false, nullable: false, unique: false, defaultValue: '1' },

]},

],

relationships: [

{ id: 'r_ho1', sourceTableId: 't_ho_doctors', sourceColumnId: 'c_hdr_did', targetTableId: 't_ho_depts', targetColumnId: 'c_hd_id', type: 'one-to-many', onDelete: 'SET NULL', onUpdate: 'CASCADE', label: '' },

{ id: 'r_ho2', sourceTableId: 't_ho_appointments', sourceColumnId: 'c_ha_pid', targetTableId: 't_ho_patients', targetColumnId: 'c_hp_id', type: 'one-to-many', onDelete: 'CASCADE', onUpdate: 'CASCADE', label: '' },

{ id: 'r_ho3', sourceTableId: 't_ho_appointments', sourceColumnId: 'c_ha_did', targetTableId: 't_ho_doctors', targetColumnId: 'c_hdr_id', type: 'one-to-many', onDelete: 'RESTRICT', onUpdate: 'CASCADE', label: '' },

{ id: 'r_ho4', sourceTableId: 't_ho_prescriptions', sourceColumnId: 'c_hpr_aid', targetTableId: 't_ho_appointments', targetColumnId: 'c_ha_id', type: 'one-to-many', onDelete: 'CASCADE', onUpdate: 'CASCADE', label: '' },

{ id: 'r_ho5', sourceTableId: 't_ho_prescriptions', sourceColumnId: 'c_hpr_mid', targetTableId: 't_ho_medications', targetColumnId: 'c_hm_id', type: 'one-to-many', onDelete: 'RESTRICT', onUpdate: 'CASCADE', label: '' },

],

},

```
};

============================================================

FILE: frontend\src\components\Header.jsx

============================================================

import React, { useState } from 'react';

import useDiagramStore from '../store/diagramStore';

import useUIStore from '../store/uiStore';

import { exportSQL, exportJSON, exportPNG, exportPDF } from '../utils/exportUtils';

import toast from 'react-hot-toast';

import { Database, Sun, Moon, ChevronDown, Save, Download, Share2 } from
'lucide-react';


export default function Header() {

  const { diagramName, setDiagramName, setDialect } = useDiagramStore();

  const dialect = useDiagramStore((s) => s.dialect);

  const { theme, toggleTheme, openSaveDiagramModal, openShareModal } =
useUIStore();

  const generatedSql = useUIStore((s) => s.generatedSql);

  const [exportOpen, setExportOpen] = useState(false);

  const [editingName, setEditingName] = useState(false);

  const [nameVal, setNameVal] = useState('');


  const handleExport = async (type) => {

    setExportOpen(false);

    const diagram = useDiagramStore.getState().getDiagramJSON();

    try {

      if (type === 'sql') exportSQL(generatedSql || '-- Click Generate SQL first',
diagram.name);
```

```
    else if (type === 'json') exportJSON(diagram);
    else if (type === 'png') await exportPNG('.react-flow', diagram.name);
    else if (type === 'pdf') await exportPDF('.react-flow', diagram.name);
    toast.success(`Exported as ${type.toUpperCase()}`);
  } catch {
    toast.error('Export failed');
  }
};

const handleDialect = (d) => {
  setDialect(d);
  toast.success(`Switched to ${d === 'mysql' ? 'MySQL' : 'PostgreSQL'}`);
};

const commitName = () => {
  setEditingName(false);
  if (nameVal.trim()) setDiagramName(nameVal.trim());
};

return (
  <header className="app-header" data-testid="app-header">
    <div className="header-left">
      <div className="header-logo">
        <Database size={20} style={{ color: '#6366f1' }} />
        <span className="logo-text">SketchSQL</span>
      </div>
      <div className="header-divider" />
      {editingName ? (
        <input
```

```
            className="diagram-name-input"
            value={nameVal}
            onChange={(e) => setNameVal(e.target.value)}
            onBlur={commitName}
            onKeyDown={(e) => { if (e.key === 'Enter') commitName(); if (e.key === 'Escape') setEditingName(false); }}
            autoFocus
          />
        ) : (
          <span
            className="diagram-name"
            onDoubleClick={() => { setEditingName(true); setNameVal(diagramName); }}
            title="Double-click to rename"
            data-testid="diagram-name"
          >{diagramName}</span>
        )}
      </div>

      <div className="header-center">
        <div className="dialect-toggle" data-testid="dialect-toggle">
          <button
            className={`dialect-btn ${dialect === 'mysql' ? 'active' : ''}`}
            onClick={() => handleDialect('mysql')}
            data-testid="dialect-mysql"
          >MySQL</button>
          <button
            className={`dialect-btn ${dialect === 'postgresql' ? 'active' : ''}`}
            onClick={() => handleDialect('postgresql')}
            data-testid="dialect-postgresql"
```

```
          >PostgreSQL</button>
        </div>
      </div>

      <div className="header-right">
        <button className="header-action-btn" onClick={openSaveDiagramModal}
data-testid="save-btn">
          <Save size={13} /> Save
        </button>
        {/* <button className="header-action-btn" onClick={openShareModal} data-
testid="share-btn" title="Create a public shareable link">
          <Share2 size={13} /> Share
        </button> */}
        <div className="export-wrap">
          <button className="header-action-btn" onClick={() =>
setExportOpen(!exportOpen)} data-testid="export-btn">
            <Download size={13} /> Export <ChevronDown size={11} />
          </button>
          {exportOpen && (
            <>
              <div className="dropdown-backdrop" onClick={() =>
setExportOpen(false)} />
              <div className="dropdown-menu" data-testid="export-menu">
                <button onClick={() => handleExport('sql')}>Export SQL (.sql)</button>
                <button onClick={() => handleExport('json')}>Export JSON (.json)</button>
                <button onClick={() => handleExport('png')}>Export PNG</button>
                <button onClick={() => handleExport('pdf')}>Export PDF</button>
              </div>
            </>
          )}
```

```
        </div>
        <button className="header-icon-btn" onClick={toggleTheme} title="Toggle theme" data-testid="theme-toggle">
          {theme === 'dark' ? <Sun size={15} /> : <Moon size={15} />}
        </button>
      </div>
    </header>
  );
}
```

```
============================================================
FILE: frontend\src\components\ShareView.jsx
============================================================
```

```
import React, { useEffect, useMemo, useState } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import ReactFlow, { Background, Controls, MiniMap, BackgroundVariant } from 'reactflow';
import 'reactflow/dist/style.css';
import Editor from '@monaco-editor/react';
import { Toaster } from 'react-hot-toast';
import toast from 'react-hot-toast';
import { Database, ExternalLink, GitFork, Eye, Copy, Download, Loader2, AlertCircle } from 'lucide-react';
import TableNode from './Canvas/TableNode';
import RelationshipEdge from './Canvas/RelationshipEdge';
import { getShare } from '../api/apiClient';
import { generateSQL } from '../utils/sqlGenerator';
import { generateDjango, generatePrisma, generateSQLAlchemy } from '../utils/ormGenerator';
```

```
import { saveToAutosave } from '../utils/persistence';
import useDiagramStore from '../store/diagramStore';

const nodeTypes = { tableNode: TableNode };
const edgeTypes = { relationshipEdge: RelationshipEdge };

function toRFNodes(diagram) {
  return (diagram?.tables || []).map((t) => ({
    id: t.id,
    type: 'tableNode',
    position: t.position || { x: 100, y: 100 },
    data: { id: t.id, name: t.name, color: t.color || 'gray', columns: t.columns || [] },
    draggable: false,
    selectable: false,
  }));
}
function toRFEdges(diagram) {
  return (diagram?.relationships || []).map((r) => ({
    id: r.id,
    type: 'relationshipEdge',
    source: r.sourceTableId,
    sourceHandle: `source-${r.sourceColumnId}`,
    target: r.targetTableId,
    targetHandle: `target-${r.targetColumnId}`,
    data: { id: r.id, type: r.type || 'one-to-many', onDelete: r.onDelete || 'RESTRICT',
onUpdate: r.onUpdate || 'RESTRICT', label: r.label || '' },
    selectable: false,
  }));
}
```

```
export default function ShareView() {
  const { shareId } = useParams();
  const navigate = useNavigate();
  const loadDiagram = useDiagramStore((s) => s.loadDiagram);

  const [state, setState] = useState({ status: 'loading', diagram: null, err: '', views: 0 });
  const [dialect, setDialect] = useState('mysql');
  const [tab, setTab] = useState('sql');
  const [ormFmt, setOrmFmt] = useState('django');

  useEffect(() => {
    let cancelled = false;
    getShare(shareId)
      .then((data) => {
        if (cancelled) return;
        setState({ status: 'ok', diagram: data.diagram, err: '', views: data.views || 1 });
        setDialect(data.diagram?.dialect || 'mysql');
      })
      .catch((e) => {
        if (cancelled) return;
        const code = e?.response?.status;
        setState({
          status: 'error',
          diagram: null,
          err: code === 404 ? 'This share link does not exist or has been removed.' :
(e?.message || 'Failed to load share'),
          views: 0,
        });
```

```
    });
  return () => { cancelled = true; };
}, [shareId]);

const nodes = useMemo(() => toRFNodes(state.diagram), [state.diagram]);
const edges = useMemo(() => toRFEdges(state.diagram), [state.diagram]);

const sql = useMemo(() => {
  if (!state.diagram) return '';
  return generateSQL(state.diagram, dialect);
}, [state.diagram, dialect]);

const orm = useMemo(() => {
  if (!state.diagram) return '';
  if (ormFmt === 'django') return generateDjango(state.diagram);
  if (ormFmt === 'prisma') return generatePrisma(state.diagram);
  return generateSQLAlchemy(state.diagram);
}, [state.diagram, ormFmt]);

const handleFork = () => {
  if (!state.diagram) return;
  const forked = {
    ...state.diagram,
    id: null,
    name: `${state.diagram.name || 'Forked Diagram'} (fork)`,
    createdAt: null,
    updatedAt: null,
  };
```

```
    // Persist BEFORE navigate so App's useEffect hydrates from the fork, not the
previous autosave.
    saveToAutosave(forked);
    loadDiagram(forked);
    toast.success('Forked to editor');
    navigate('/');
  };

  const copy = async (text) => {
    try {
      await navigator.clipboard.writeText(text);
      toast.success('Copied');
    } catch {
      toast.error('Copy failed');
    }
  };

  const download = (text, ext, mime) => {
    const name = state.diagram?.name || 'diagram';
    const blob = new Blob([text], { type: mime });
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = `${name}.${ext}`;
    a.click();
    URL.revokeObjectURL(url);
  };

  if (state.status === 'loading') {
```

```
    return (
      <div className="app-root dark" style={{ display: 'grid', placeItems: 'center', height: '100vh' }} data-testid="share-loading">
        <div style={{ display: 'flex', alignItems: 'center', gap: 10, color: '#94a3b8' }}>
          <Loader2 size={18} className="spin" /> Loading shared diagram…
        </div>
      </div>
    );
  }
  if (state.status === 'error') {
    return (
      <div className="app-root dark" style={{ display: 'grid', placeItems: 'center', height: '100vh' }} data-testid="share-error-view">
        <div style={{ textAlign: 'center', maxWidth: 420 }}>
          <AlertCircle size={42} style={{ color: '#f87171', margin: '0 auto 12px' }} />
          <div style={{ color: '#f1f5f9', fontSize: 16, fontWeight: 600, marginBottom: 6 }}>Share not available</div>
          <div style={{ color: '#94a3b8', fontSize: 13, marginBottom: 18 }}>{state.err}</div>
          <button className="modal-confirm" onClick={() => navigate('/')} data-testid="share-goto-editor">
            Go to Editor
          </button>
        </div>
      </div>
    );
  }

  const { diagram } = state;

  return (
```

```
    <div className="app-root dark" data-testid="share-view">
      <header className="app-header">
        <div className="header-left">
          <div className="header-logo">
            <Database size={20} style={{ color: '#6366f1' }} />
            <span className="logo-text">SketchSQL</span>
          </div>
          <div className="header-divider" />
          <span className="diagram-name" data-testid="share-diagram-name">{diagram.name || 'Shared Diagram'}</span>
          <span
            style={{
              marginLeft: 10, padding: '2px 8px', borderRadius: 4, fontSize: 10,
              background: 'rgba(99,102,241,0.15)', color: '#a5b4fc',
              border: '1px solid rgba(99,102,241,0.35)', textTransform: 'uppercase', letterSpacing: 0.6, fontWeight: 600,
            }}
          >Read-only · Shared</span>
        </div>

        <div className="header-center">
          <div className="dialect-toggle">
            <button
              className={`dialect-btn ${dialect === 'mysql' ? 'active' : ''}`}
              onClick={() => setDialect('mysql')}
              data-testid="share-dialect-mysql"
            >MySQL</button>
            <button
              className={`dialect-btn ${dialect === 'postgresql' ? 'active' : ''}`}
              onClick={() => setDialect('postgresql')}
```

```
            data-testid="share-dialect-postgresql"
          >PostgreSQL</button>
        </div>
      </div>

      <div className="header-right">
        <span style={{ color: '#64748b', fontSize: 11, display: 'inline-flex', alignItems: 'center', gap: 4 }} data-testid="share-views">
          <Eye size={12} /> {state.views} view{state.views === 1 ? '' : 's'}
        </span>
        <button className="header-action-btn" onClick={handleFork} data-testid="share-fork-btn" title="Open an editable copy in the editor">
          <GitFork size={13} /> Fork to edit
        </button>
        <a href="/" className="header-action-btn" data-testid="share-open-editor" style={{ textDecoration: 'none' }}>
          Editor <ExternalLink size={12} />
        </a>
      </div>
    </header>

    <div className="app-body">
      <div className="canvas-wrapper" style={{ flex: 1 }}>
        <div className="canvas-container" data-testid="share-canvas">
          <ReactFlow
            nodes={nodes}
            edges={edges}
            nodeTypes={nodeTypes}
            edgeTypes={edgeTypes}
            nodesDraggable={false}
```

```
            nodesConnectable={false}
            elementsSelectable={false}
            zoomOnDoubleClick={false}
            proOptions={{ hideAttribution: true }}
            fitView
            fitViewOptions={{ padding: 0.2, maxZoom: 1 }}
            minZoom={0.05}
            maxZoom={2.5}
          >
            <Background variant={BackgroundVariant.Dots} color="#1e293b" size={1}
gap={24} style={{ opacity: 0.5 }} />
            <Controls style={{ background: '#161b27', border: '1px solid #1e293b',
borderRadius: 6 }} showInteractive={false} />
            <MiniMap
              style={{ background: '#111827', border: '1px solid #1e293b', borderRadius:
6 }}
              nodeColor={(n) => {
                const c = { blue: '#1e3a5f', green: '#14362a', purple: '#2d1b5e', orange:
'#3b2200', red: '#3b1212', gray: '#1e293b' };
                return c[n.data?.color] || '#1e293b';
              }}
              maskColor="rgba(6,9,18,0.7)"
            />
          </ReactFlow>
        </div>
      </div>

      <aside className="right-panel" style={{ width: 380, minWidth: 380, maxWidth:
380, borderLeft: '1px solid var(--border)', display: 'flex', flexDirection: 'column' }}>
        <div className="right-panel-tabs">
          <button
```

```
            className={`rp-tab ${tab === 'sql' ? 'active' : ''}`}
            onClick={() => setTab('sql')}
            data-testid="share-tab-sql"
          >SQL</button>
          <button
            className={`rp-tab ${tab === 'orm' ? 'active' : ''}`}
            onClick={() => setTab('orm')}
            data-testid="share-tab-orm"
          >ORM</button>
        </div>

        {tab === 'sql' && (
          <div style={{ flex: 1, display: 'flex', flexDirection: 'column' }}>
            <div style={{ padding: '8px 10px', display: 'flex', gap: 6, borderBottom: '1px solid #1e293b' }}>
              <button className="header-action-btn" onClick={() => copy(sql)} data-testid="share-copy-sql"><Copy size={12} /> Copy</button>
              <button className="header-action-btn" onClick={() => download(sql, 'sql', 'text/plain')} data-testid="share-download-sql"><Download size={12} /> Download .sql</button>
            </div>
            <div style={{ flex: 1 }}>
              <Editor
                height="100%"
                defaultLanguage="sql"
                value={sql}
                theme="vs-dark"
                options={{ readOnly: true, minimap: { enabled: false }, fontSize: 12, scrollBeyondLastLine: false }}
              />
            </div>
```

```
          </div>
        )}

        {tab === 'orm' && (
          <div style={{ flex: 1, display: 'flex', flexDirection: 'column' }}>
            <div style={{ padding: '8px 10px', display: 'flex', gap: 6, borderBottom: '1px solid #1e293b', flexWrap: 'wrap' }}>
              {[
                ['django', 'Django'],
                ['prisma', 'Prisma'],
                ['sqlalchemy', 'SQLAlchemy'],
              ].map(([k, label]) => (
                <button
                  key={k}
                  className={`dialect-btn ${ormFmt === k ? 'active' : ''}`}
                  onClick={() => setOrmFmt(k)}
                  data-testid={`share-orm-${k}`}
                >{label}</button>
              ))}
              <div style={{ flex: 1 }} />
              <button className="header-action-btn" onClick={() => copy(orm)} data-testid="share-copy-orm"><Copy size={12} /></button>
              <button
                className="header-action-btn"
                onClick={() => download(orm, ormFmt === 'prisma' ? 'prisma' : 'py', 'text/plain')}
                data-testid="share-download-orm"
              ><Download size={12} /></button>
            </div>
            <div style={{ flex: 1 }}>
```

```
              <Editor
                height="100%"
                defaultLanguage={ormFmt === 'prisma' ? 'javascript' : 'python'}
                value={orm}
                theme="vs-dark"
                options={{ readOnly: true, minimap: { enabled: false }, fontSize: 12, scrollBeyondLastLine: false }}
              />
            </div>
          </div>
        )}
      </aside>
    </div>

    <Toaster position="bottom-right" toastOptions={{
      duration: 2500,
      style: { background: '#161b27', color: '#f1f5f9', border: '1px solid #1e293b', fontSize: 13, borderRadius: 6 },
    }} />
  </div>
  );
}

============================================================
FILE: frontend\src\components\Canvas\CanvasArea.jsx
============================================================

import React, { useCallback, useRef } from 'react';
import ReactFlow, {
  Background, Controls, MiniMap, BackgroundVariant,
```

```
} from 'reactflow';
import 'reactflow/dist/style.css';
import useDiagramStore from '../../store/diagramStore';
import useUIStore from '../../store/uiStore';
import TableNode from './TableNode';
import RelationshipEdge from './RelationshipEdge';
import CanvasToolbar from './CanvasToolbar';

const nodeTypes = { tableNode: TableNode };
const edgeTypes = { relationshipEdge: RelationshipEdge };

export default function CanvasArea() {
  const { nodes, edges, onNodesChange, onEdgesChange, onConnect, addTable } =
useDiagramStore();
  const { showMinimap, showGrid, clearSelection, snapToGrid } = useUIStore();
  const wrapperRef = useRef(null);
  const rfInstance = useRef(null);

  const onInit = useCallback((inst) => { rfInstance.current = inst; }, []);

  const onDoubleClick = useCallback((e) => {
    if (!rfInstance.current || !wrapperRef.current) return;
    if (e.target.closest('.react-flow__node') || e.target.closest('.react-flow__edge'))
return;
    const bounds = wrapperRef.current.getBoundingClientRect();
    const pos = rfInstance.current.project({ x: e.clientX - bounds.left, y: e.clientY -
bounds.top });
    addTable(pos);
  }, [addTable]);
```

```
const onPaneClick = useCallback(() => clearSelection(), [clearSelection]);

return (
  <div className="canvas-container" ref={wrapperRef} data-testid="canvas-area">
    <CanvasToolbar rfInstance={rfInstance} />
    <ReactFlow
      nodes={nodes}
      edges={edges}
      onNodesChange={onNodesChange}
      onEdgesChange={onEdgesChange}
      onConnect={onConnect}
      onInit={onInit}
      onDoubleClick={onDoubleClick}
      onPaneClick={onPaneClick}
      nodeTypes={nodeTypes}
      edgeTypes={edgeTypes}
      connectionLineStyle={{ stroke: '#6366f1', strokeWidth: 2 }}
      defaultEdgeOptions={{ type: 'relationshipEdge' }}
      fitView
      fitViewOptions={{ padding: 0.2, maxZoom: 1 }}
      proOptions={{ hideAttribution: true }}
      minZoom={0.05}
      maxZoom={2.5}
      selectNodesOnDrag={false}
      snapToGrid={snapToGrid}
      snapGrid={[20, 20]}
    >
      <Background
        variant={showGrid ? BackgroundVariant.Dots : BackgroundVariant.Lines}
```

```
          color="#1e293b"
          size={1}
          gap={24}
          style={{ opacity: showGrid ? 0.5 : 0 }}
        />
        <Controls
          style={{ background: '#161b27', border: '1px solid #1e293b', borderRadius: 6 }}
          showInteractive={false}
        />
        {showMinimap && (
          <MiniMap
            style={{ background: '#111827', border: '1px solid #1e293b', borderRadius: 6
}}
            nodeColor={(n) => {
              const c = { blue: '#1e3a5f', green: '#14362a', purple: '#2d1b5e', orange:
'#3b2200', red: '#3b1212', gray: '#1e293b' };
              return c[n.data?.color] || '#1e293b';
            }}
            maskColor="rgba(6,9,18,0.7)"
          />
        )}
      </ReactFlow>
    </div>
  );
}
```

==========

FILE: frontend\src\components\Canvas\CanvasToolbar.jsx

==========

```
import React from 'react';
import useDiagramStore from '../../store/diagramStore';
import useUIStore from '../../store/uiStore';
import { Plus, Upload, Maximize2, Grid3X3, Map, Undo2, Redo2, Magnet } from 'lucide-react';
import toast from 'react-hot-toast';

export default function CanvasToolbar({ rfInstance }) {
  const { addTable, undo, redo, canUndo, canRedo, selectAll } = useDiagramStore();
  const { showGrid, showMinimap, snapToGrid, toggleGrid, toggleMinimap, toggleSnapToGrid, openImportSqlModal } = useUIStore();

  const fitView = () => rfInstance.current?.fitView({ padding: 0.15, duration: 400 });

  const handleAddTable = () => {
    const pos = rfInstance.current
      ? rfInstance.current.project({ x: 300 + Math.random() * 100, y: 200 + Math.random() * 100 })
      : { x: 300, y: 200 };
    addTable(pos);
    toast.success('Table added to canvas');
  };

  return (
    <div className="canvas-toolbar" data-testid="canvas-toolbar">
      <button className="tb-btn primary" onClick={handleAddTable} data-testid="add-table-btn" title="Add Table">
        <Plus size={13} /><span>Add Table</span>
      </button>
      <div className="tb-sep" />
```

```
      <button className="tb-btn" onClick={openImportSqlModal} data-testid="import-sql-btn" title="Import SQL">
        <Upload size={13} /><span>Import SQL</span>
      </button>
      <div className="tb-sep" />
      <button className="tb-btn" onClick={fitView} data-testid="fit-view-btn" title="Fit View"><Maximize2 size={13} /></button>
      <button className={`tb-btn ${showGrid ? 'on' : ''}`} onClick={toggleGrid} data-testid="toggle-grid-btn" title="Toggle Grid"><Grid3X3 size={13} /></button>
      <button className={`tb-btn ${showMinimap ? 'on' : ''}`} onClick={toggleMinimap} data-testid="toggle-minimap-btn" title="Toggle Minimap"><Map size={13} /></button>
      <button className={`tb-btn ${snapToGrid ? 'on' : ''}`} onClick={toggleSnapToGrid} data-testid="toggle-snap-btn" title="Snap to Grid"><Magnet size={13} /></button>
      <div className="tb-sep" />
      <button className="tb-btn" onClick={selectAll} data-testid="select-all-btn" title="Select All (Ctrl+A)" style={{ fontSize: 11, padding: '5px 8px' }}>All</button>
      <div className="tb-sep" />
      <button className="tb-btn" onClick={undo} disabled={!canUndo()} data-testid="undo-btn" title="Undo (Ctrl+Z)"><Undo2 size={13} /></button>
      <button className="tb-btn" onClick={redo} disabled={!canRedo()} data-testid="redo-btn" title="Redo (Ctrl+Shift+Z)"><Redo2 size={13} /></button>
    </div>
  );
}

============================================================
FILE: frontend\src\components\Canvas\TableNode.jsx
============================================================

import React, { useState } from 'react';
import { Handle, Position } from 'reactflow';
```

```
import useDiagramStore from '../../store/diagramStore';
import useUIStore from '../../store/uiStore';

const COLORS = {
  blue: '#1e3a5f', green: '#14362a', purple: '#2d1b5e',
  orange: '#3b2200', red: '#3b1212', gray: '#1e293b',
};

const TYPES = [
  'INT','BIGINT','VARCHAR(255)','VARCHAR(100)','VARCHAR(50)',
  'TEXT','BOOLEAN','DATE','DATETIME','TIMESTAMP',
  'DECIMAL(10,2)','UUID','FLOAT','JSON','SMALLINT','TINYINT',
];

function ColEditor({ col, tableId, onClose }) {
  const { updateColumn, deleteColumn } = useDiagramStore();
  const upd = (k, v) => updateColumn(tableId, col.id, { [k]: v });

  return (
    <div className="col-editor nodrag nowheel" onClick={(e) =>
e.stopPropagation()}>
      <div className="ce-row">
        <label>Name</label>
        <input className="nodrag" value={col.name} onChange={(e) => upd('name',
e.target.value)} />
      </div>
      <div className="ce-row">
        <label>Type</label>
        <select className="nodrag" value={col.type} onChange={(e) => upd('type',
e.target.value)}>
```

```
          {TYPES.map((t) => <option key={t} value={t}>{t}</option>)}
        </select>
      </div>
      <div className="ce-row">
        <label>Default</label>
        <input className="nodrag" value={col.defaultValue} placeholder="optional"
onChange={(e) => upd('defaultValue', e.target.value)} />
      </div>
      <div className="ce-checks">
        <label><input type="checkbox" className="nodrag" checked={col.primaryKey}
onChange={(e) => upd('primaryKey', e.target.checked)} />PK</label>
        <label><input type="checkbox" className="nodrag"
checked={col.autoIncrement} onChange={(e) => upd('autoIncrement',
e.target.checked)} />AI</label>
        <label><input type="checkbox" className="nodrag" checked={!col.nullable}
onChange={(e) => upd('nullable', !e.target.checked)} />NN</label>
        <label><input type="checkbox" className="nodrag" checked={col.unique}
onChange={(e) => upd('unique', e.target.checked)} />UQ</label>
      </div>
      <div className="ce-actions">
        <button className="ce-delete nodrag" onClick={() => deleteColumn(tableId,
col.id)}>
          Delete column
        </button>
        <button className="ce-close nodrag" onClick={onClose}>Done</button>
      </div>
    </div>
  );
}

function CtxMenu({ x, y, tableId, color, onRename, onAddCol, onDelete, onColor,
onClose }) {
```

```
  return (
    <>
      <div className="ctx-backdrop" onClick={onClose} />
      <div className="ctx-menu nodrag" style={{ position: 'fixed', left: x, top: y, zIndex:
9999 }}>
        <button onClick={onRename}>Rename</button>
        <button onClick={onAddCol}>Add Column</button>
        <div className="ctx-sep" />
        <div className="ctx-colors">
          {Object.entries(COLORS).map(([name, hex]) => (
            <button key={name} className={`ctx-dot ${name === color ? 'active' : ''}`}
              style={{ background: hex }} onClick={() => { onColor(name); onClose(); }}
title={name} />
          ))}
        </div>
        <div className="ctx-sep" />
        <button className="ctx-danger" onClick={onDelete}>Delete Table</button>
      </div>
    </>
  );
}

export default function TableNode({ id, data, selected }) {
  const [editCol, setEditCol] = useState(null);
  const [editingName, setEditingName] = useState(false);
  const [nameVal, setNameVal] = useState('');
  const [ctxMenu, setCtxMenu] = useState(null);
  const { updateTable, deleteTable, addColumn } = useDiagramStore();
  const { setSelectedNode, setActiveTab } = useUIStore();
```

```
  const accentBg = COLORS[data.color] || COLORS.gray;

  const commitName = () => {
    setEditingName(false);
    if (nameVal.trim()) updateTable(id, { name: nameVal.trim() });
  };

  return (
    <div
      className={`table-node${selected ? ' selected' : ''}`}
      onClick={() => { setSelectedNode(id); setActiveTab('properties'); }}
      onContextMenu={(e) => { e.preventDefault(); setCtxMenu({ x: e.clientX, y: e.clientY }); }}
      data-testid={`table-node-${id}`}
    >
      {/* Table header */}
      <div className="tn-header" style={{ background: accentBg }}
        onDoubleClick={(e) => { e.stopPropagation(); setEditingName(true); setNameVal(data.name); }}>
        {editingName ? (
          <input
            className="tn-name-input nodrag"
            value={nameVal}
            onChange={(e) => setNameVal(e.target.value)}
            onBlur={commitName}
            onKeyDown={(e) => { if (e.key === 'Enter') commitName(); if (e.key === 'Escape') setEditingName(false); }}
            autoFocus
            onClick={(e) => e.stopPropagation()}
          />
```

```
        ) : (
          <span className="tn-name">{data.name}</span>
        )}
        <button className="tn-add-col nodrag" onClick={(e) => { e.stopPropagation(); addColumn(id); }} title="Add column">+</button>
      </div>

      {/* Columns */}
      <div className="tn-columns">
        {(data.columns || []).map((col) => (
          <div key={col.id} className="tn-col-wrapper">
            <Handle type="target" position={Position.Left} id={`target-${col.id}`} className="tn-handle" />
            <div
              className={`tn-col nodrag${editCol === col.id ? ' active' : ''}`}
              onClick={(e) => { e.stopPropagation(); setSelectedNode(id); setActiveTab('properties'); setEditCol(editCol === col.id ? null : col.id); }}
              data-testid={`col-${col.id}`}
            >
              <div className="tn-badges">
                {col.primaryKey && <span className="badge-pk">PK</span>}
                {!col.primaryKey && col.unique && <span className="badge-uq">UQ</span>}
                {!col.primaryKey && !col.nullable && <span className="badge-nn">NN</span>}
              </div>
              <span className="tn-col-name">{col.name}</span>
              <span className="tn-col-type">{col.type}</span>
            </div>
            {editCol === col.id && (
              <ColEditor col={col} tableId={id} onClose={() => setEditCol(null)} />
```

```
          )}
          <Handle type="source" position={Position.Right} id={`source-${col.id}`}
classNameName="tn-handle" />
        </div>
      ))}
      {!data.columns?.length && <div classNameName="tn-empty">No columns</div>}
    </div>

    {ctxMenu && (
      <CtxMenu
        x={ctxMenu.x} y={ctxMenu.y}
        tableId={id} color={data.color}
        onRename={() => { setEditingName(true); setNameVal(data.name);
setCtxMenu(null); }}
        onAddCol={() => { addColumn(id); setCtxMenu(null); }}
        onDelete={() => { deleteTable(id); setCtxMenu(null); }}
        onColor={(c) => updateTable(id, { color: c })}
        onClose={() => setCtxMenu(null)}
      />
    )}
  </div>
 );
}
```

```
============================================================
FILE: frontend\src\components\Canvas\RelationshipEdge.jsx
============================================================
```

```
import React from 'react';
import { getBezierPath, EdgeLabelRenderer } from 'reactflow';
```

```
import useUIStore from '../../store/uiStore';

const LABELS = {
  'one-to-one': '1:1',
  'one-to-many': '1:N',
  'many-to-many': 'M:N',
  'many-to-one': 'N:1',
};

export default function RelationshipEdge({
  id, sourceX, sourceY, targetX, targetY,
  sourcePosition, targetPosition, data, selected,
}) {
  const [edgePath, labelX, labelY] = getBezierPath({
    sourceX, sourceY, sourcePosition,
    targetX, targetY, targetPosition,
  });

  const { setSelectedEdge, setActiveTab } = useUIStore();
  const relType = data?.type || 'one-to-many';
  const label = LABELS[relType] || '1:N';
  const stroke = selected ? '#6366f1' : '#818cf8';

  const handleClick = (e) => {
    e.stopPropagation();
    setSelectedEdge(id);
    setActiveTab('properties');
  };
```

```
  return (
    <>
      {/* Invisible wide path for easier clicking */}
      <path d={edgePath} stroke="transparent" strokeWidth={20} fill="none"
onClick={handleClick} style={{ cursor: 'pointer' }} />
      <path
        id={id}
        d={edgePath}
        stroke={stroke}
        strokeWidth={selected ? 2 : 1.5}
        fill="none"
        className="react-flow__edge-path"
        onClick={handleClick}
        style={{ cursor: 'pointer' }}
        data-testid={`edge-${id}`}
      />
      <EdgeLabelRenderer>
        <div
          className="edge-label nodrag nopan"
          style={{
            position: 'absolute',
            transform: `translate(-50%,-50%) translate(${labelX}px,${labelY}px)`,
            pointerEvents: 'all',
            cursor: 'pointer',
          }}
          onClick={handleClick}
        >
          {label}
        </div>
```

```
      </EdgeLabelRenderer>
    </>
  );
}

============================================================
FILE: frontend\src\components\LeftPanel\LeftPanel.jsx
============================================================

import React, { useState, useEffect } from 'react';
import useDiagramStore from '../../store/diagramStore';
import { loadAllDiagrams, deleteDiagram, renameDiagram } from
'../../utils/persistence';
import toast from 'react-hot-toast';
import { Plus, Trash2, Edit2, Database, Clock } from 'lucide-react';

export default function LeftPanel() {
  const [diagrams, setDiagrams] = useState([]);
  const [ctxMenu, setCtxMenu] = useState(null);
  const [renaming, setRenaming] = useState(null);
  const [renameVal, setRenameVal] = useState('');
  const { loadDiagram, clearDiagram, diagramId } = useDiagramStore();

  const refresh = async () => {
    const all = await loadAllDiagrams();
    setDiagrams(all);
  };

  useEffect(() => {
```

```
  refresh();
  window.addEventListener('diagram-saved', refresh);
  return () => window.removeEventListener('diagram-saved', refresh);
}, []);

const handleNew = () => {
  clearDiagram();
  toast.success('New diagram started');
};

const handleLoad = (d) => {
  loadDiagram(d);
  toast.success(`Loaded "${d.name}"`);
};

const handleDelete = async (id, name) => {
  await deleteDiagram(id);
  setCtxMenu(null);
  refresh();
  toast.success(`Deleted "${name}"`);
};

const handleRename = async (id) => {
  if (renameVal.trim()) {
    await renameDiagram(id, renameVal.trim());
    setRenaming(null);
    refresh();
  }
};
```

```
  const fmt = (iso) => {
    if (!iso) return '';
    return new Date(iso).toLocaleDateString(undefined, { month: 'short', day: 'numeric' });
  };

  return (
    <aside className="left-panel" data-testid="left-panel">
      <div className="left-panel-header">
        <span className="left-panel-title">Saved Diagrams</span>
        <button className="icon-btn" onClick={handleNew} data-testid="new-diagram-btn" title="New Diagram">
          <Plus size={14} />
        </button>
      </div>

      <div className="diagram-list">
        {diagrams.length === 0 && (
          <div className="empty-list">No saved diagrams</div>
        )}
        {diagrams.map((d) => (
          <div
            key={d.id}
            className={`diagram-item ${d.id === diagramId ? 'active' : ''}`}
            onClick={() => handleLoad(d)}
            onContextMenu={(e) => { e.preventDefault(); setCtxMenu({ id: d.id, name: d.name, x: e.clientX, y: e.clientY }); }}
            data-testid={`diagram-item-${d.id}`}
          >
```

```
          {renaming === d.id ? (
            <input
              className="rename-input"
              value={renameVal}
              onChange={(e) => setRenameVal(e.target.value)}
              onBlur={() => handleRename(d.id)}
              onKeyDown={(e) => { if (e.key === 'Enter') handleRename(d.id); if (e.key
=== 'Escape') setRenaming(null); }}
              autoFocus
              onClick={(e) => e.stopPropagation()}
            />
          ) : (
            <>
              <Database size={12} className="di-icon" />
              <div className="di-info">
                <span className="di-name">{d.name}</span>
                <div className="di-meta">
                  <Clock size={9} />
                  <span>{fmt(d.updatedAt)}</span>
                  <span className="di-tables">{d.tables?.length || 0}t</span>
                </div>
              </div>
            </>
          )}
        </div>
      ))}
    </div>

    {ctxMenu && (
```

```
        <>
          <div className="ctx-backdrop" onClick={() => setCtxMenu(null)} />
          <div className="ctx-menu" style={{ position: 'fixed', left: ctxMenu.x, top: ctxMenu.y }}>
            <button onClick={() => { setRenaming(ctxMenu.id); setRenameVal(ctxMenu.name); setCtxMenu(null); }}>
              <Edit2 size={11} /> Rename
            </button>
            <button className="danger" onClick={() => handleDelete(ctxMenu.id, ctxMenu.name)}>
              <Trash2 size={11} /> Delete
            </button>
          </div>
        </>
      )}
    </aside>
  );
}
```

```
============================================================
FILE: frontend\src\components\RightPanel\RightPanel.jsx
============================================================
```

```
import React from 'react';
import useUIStore from '../../store/uiStore';
import SqlOutputTab from './SqlOutputTab';
import AiAssistantTab from './AiAssistantTab';
import PropertiesTab from './PropertiesTab';
import OrmOutputTab from './OrmOutputTab';
import { Code2, Bot, Settings, Layers } from 'lucide-react';
```

```
const TABS = [
  { id: 'sql', label: 'SQL', icon: Code2 },
  { id: 'orm', label: 'ORM', icon: Layers },
  { id: 'ai', label: 'AI', icon: Bot },
  { id: 'properties', label: 'Properties', icon: Settings },
];

export default function RightPanel() {
  const { activeTab, setActiveTab } = useUIStore();

  return (
    <aside className="right-panel" data-testid="right-panel">
      <div className="right-panel-tabs">
        {TABS.map(({ id, label, icon: Icon }) => (
          <button
            key={id}
            className={`rp-tab ${activeTab === id ? 'active' : ''}`}
            onClick={() => setActiveTab(id)}
            data-testid={`tab-${id}`}
          >
            <Icon size={13} />
            <span>{label}</span>
          </button>
        ))}
      </div>
      <div className="right-panel-content">
        {activeTab === 'sql' && <SqlOutputTab />}
        {activeTab === 'orm' && <OrmOutputTab />}
```

```
        {activeTab === 'ai' && <AiAssistantTab />}
        {activeTab === 'properties' && <PropertiesTab />}
      </div>
    </aside>
  );
}

============================================================
FILE: frontend\src\components\RightPanel\AiAssistantTab.jsx
============================================================

import React, { useState, useRef, useEffect } from 'react';
import ReactMarkdown from 'react-markdown';
import remarkGfm from 'remark-gfm';
import useDiagramStore from '../../store/diagramStore';
import useUIStore from '../../store/uiStore';
import { aiGenerateSchema, aiAnalyzeSchema, aiChat } from '../../api/apiClient';
import { autoLayout } from '../../utils/diagramLayout';
import toast from 'react-hot-toast';
import { Send, Sparkles, BarChart2, Trash2, User, Bot } from 'lucide-react';

const CHIPS = ['E-commerce store', 'Blog with comments', 'University management',
'Hospital system'];

function ChatMessage({ msg }) {
  const isAI = msg.role === 'assistant';
  return (
    <div className={`chat-msg ${isAI ? 'ai' : 'user'}`} data-testid={`chat-msg-
${msg.role}`}>
```

```
      <div className="chat-avatar">{isAI ? <Bot size={13} /> : <User size={13}
/>}</div>
      <div className="chat-bubble">
        <ReactMarkdown
remarkPlugins={[remarkGfm]}>{msg.content}</ReactMarkdown>
      </div>
    </div>
  );
}

function Thinking() {
  return (
    <div className="chat-msg ai">
      <div className="chat-avatar"><Bot size={13} /></div>
      <div className="chat-bubble thinking">
        <span className="dot" /><span className="dot" /><span className="dot"
/>
      </div>
    </div>
  );
}

export default function AiAssistantTab() {
  const [prompt, setPrompt] = useState('');
  const [messages, setMessages] = useState([]);
  const [chatInput, setChatInput] = useState('');
  const [loading, setLoading] = useState(false);
  const chatEndRef = useRef(null);
  const { loadDiagram, getDiagramJSON } = useDiagramStore();
  const dialect = useDiagramStore((s) => s.dialect);
```

```
  const { setActiveTab } = useUIStore();

  useEffect(() => {
    chatEndRef.current?.scrollIntoView({ behavior: 'smooth' });
  }, [messages, loading]);

  const handleGenerate = async () => {
    if (!prompt.trim() || loading) return;
    const currentNodes = useDiagramStore.getState().nodes;
    if (currentNodes.length > 0) {
      const confirmed = window.confirm('Canvas has content. Replace with new schema?');
      if (!confirmed) return;
    }
    setLoading(true);
    try {
      const diagram = await aiGenerateSchema(prompt);
      const laid = autoLayout(diagram.tables || [], diagram.relationships || []);
      loadDiagram({ ...diagram, tables: laid });
      toast.success(`Schema generated — ${diagram.tables?.length || 0} tables added`);
      setPrompt('');
      setActiveTab('sql');
    } catch (e) {
      console.error(e.message);
      toast.error('AI unavailable, please try again');
    } finally {
      setLoading(false);
    }
  };
```

```
  const handleAnalyze = async () => {
    if (loading) return;
    const diagram = getDiagramJSON();
    if (!diagram.tables?.length) { toast.error('Add tables to the canvas first'); return; }
    const userMsg = { role: 'user', content: 'Please analyze my current database
schema and suggest improvements.' };
    setMessages((m) => [...m, userMsg]);
    setLoading(true);
    try {
      const analysis = await aiAnalyzeSchema(diagram, dialect);
      setMessages((m) => [...m, { role: 'assistant', content: analysis }]);
    } catch {
      toast.error('AI unavailable, please try again');
      setMessages((m) => m.slice(0, -1));
    } finally {
      setLoading(false);
    }
  };


  const handleChatSend = async () => {
    if (!chatInput.trim() || loading) return;
    const userMsg = { role: 'user', content: chatInput };
    const newMessages = [...messages, userMsg];
    setMessages(newMessages);
    setChatInput('');
    setLoading(true);
    try {
      const diagram = getDiagramJSON();
```

```
      const reply = await aiChat(newMessages, diagram, dialect);
      setMessages((m) => [...m, { role: 'assistant', content: reply }]);
    } catch {
      toast.error('AI unavailable, please try again');
      setMessages((m) => m.slice(0, -1));
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="ai-tab" data-testid="ai-assistant-tab">
      {/* NL Generate */}
      <div className="ai-gen-section">
        <div className="ai-gen-label"><Sparkles size={12} /> Generate from description</div>
        <div className="ai-gen-row">
          <input
            className="ai-gen-input"
            value={prompt}
            onChange={(e) => setPrompt(e.target.value)}
            onKeyDown={(e) => e.key === 'Enter' && handleGenerate()}
            placeholder="Describe a schema in plain English..."
            disabled={loading}
            data-testid="ai-prompt-input"
          />
          <button
            className="ai-gen-btn"
            onClick={handleGenerate}
```

```
            disabled={loading || !prompt.trim()}
            data-testid="ai-generate-btn"
          >
            {loading ? '...' : 'Build'}
          </button>
        </div>
        <div className="ai-chips">
          {CHIPS.map((c) => (
            <button key={c} className="ai-chip" onClick={() => setPrompt(c)} data-testid={`chip-${c}`}>{c}</button>
          ))}
        </div>
      </div>

      {/* Actions */}
      <div className="ai-actions">
        <button className="ai-analyze-btn" onClick={handleAnalyze} disabled={loading} data-testid="analyze-schema-btn">
          <BarChart2 size={12} /> Analyze My Schema
        </button>
        {messages.length > 0 && (
          <button className="ai-clear-btn" onClick={() => setMessages([])} data-testid="clear-chat-btn">
            <Trash2 size={12} /> Clear
          </button>
        )}
      </div>

      {/* Chat */}
      <div className="chat-area" data-testid="chat-area">
```

```
  {messages.length === 0 && !loading && (
    <div className="chat-empty">
      <Bot size={28} style={{ color: '#64748b', marginBottom: 8 }} />
      <p>Ask about your schema or request SQL queries</p>
    </div>
  )}
  {messages.map((msg, i) => <ChatMessage key={i} msg={msg} />)}
  {loading && <Thinking />}
  <div ref={chatEndRef} />
</div>

{/* Chat input */}
<div className="chat-input-row">
  <input
    className="chat-input"
    value={chatInput}
    onChange={(e) => setChatInput(e.target.value)}
    onKeyDown={(e) => e.key === 'Enter' && !e.shiftKey && handleChatSend()}
    placeholder="Ask about your schema or request a query..."
    disabled={loading}
    data-testid="chat-input"
  />
  <button
    className="chat-send-btn"
    onClick={handleChatSend}
    disabled={loading || !chatInput.trim()}
    data-testid="chat-send-btn"
  >
    <Send size={14} />
```

```
        </button>
      </div>
    </div>
  );
}

============================================================
FILE: frontend\src\components\RightPanel\SqlOutputTab.jsx
============================================================

import React, { useEffect, useRef, useCallback } from 'react';
import Editor from '@monaco-editor/react';
import useDiagramStore from '../../store/diagramStore';
import useUIStore from '../../store/uiStore';
import { generateSQL } from '../../utils/sqlGenerator';
import { generateSQL as generateSQLAPI } from '../../api/apiClient';
import { exportSQL } from '../../utils/exportUtils';
import toast from 'react-hot-toast';
import { Copy, Download, RefreshCw } from 'lucide-react';

export default function SqlOutputTab() {
  const nodes = useDiagramStore((s) => s.nodes);
  const edges = useDiagramStore((s) => s.edges);
  const dialect = useDiagramStore((s) => s.dialect);
  const diagramName = useDiagramStore((s) => s.diagramName);
  const { generatedSql, setGeneratedSql, sqlLoading, setSqlLoading } =
useUIStore();
  const debounceRef = useRef(null);
```

```
const regenerate = useCallback(() => {
  const diagram = useDiagramStore.getState().getDiagramJSON();
  const sql = generateSQL(diagram, dialect);
  setGeneratedSql(sql);
}, [dialect, setGeneratedSql]);

// Auto-regenerate on diagram/dialect change
useEffect(() => {
  clearTimeout(debounceRef.current);
  debounceRef.current = setTimeout(regenerate, 800);
  return () => clearTimeout(debounceRef.current);
}, [nodes, edges, dialect, regenerate]);

// Initial generation
useEffect(() => { regenerate(); }, []); // eslint-disable-line

const handleGenerateViaAPI = async () => {
  setSqlLoading(true);
  try {
    const diagram = useDiagramStore.getState().getDiagramJSON();
    const sql = await generateSQLAPI(diagram, dialect);
    setGeneratedSql(sql);
    toast.success('SQL generated');
  } catch {
    toast.error('Generation failed — showing local version');
    regenerate();
  } finally {
    setSqlLoading(false);
  }
```

```
  };

  const handleCopy = () => {
    navigator.clipboard.writeText(generatedSql).then(() => toast.success('Copied to clipboard'));
  };

  const handleDownload = () => {
    exportSQL(generatedSql, diagramName);
    toast.success('SQL file downloaded');
  };

  return (
    <div className="sql-tab" data-testid="sql-tab">
      <div className="sql-toolbar">
        <button
          className="sql-gen-btn"
          onClick={handleGenerateViaAPI}
          disabled={sqlLoading}
          data-testid="generate-sql-btn"
        >
          <RefreshCw size={12} className={sqlLoading ? 'spin' : ''} />
          {sqlLoading ? 'Generating...' : 'Generate SQL'}
        </button>
        <div className="sql-toolbar-right">
          <button className="sql-icon-btn" onClick={handleCopy} title="Copy" data-testid="copy-sql-btn">
            <Copy size={13} />
          </button>
```

```
          <button className="sql-icon-btn" onClick={handleDownload} title="Download
.sql" data-testid="download-sql-btn">
            <Download size={13} />
          </button>
        </div>
      </div>
      <div className="monaco-wrap" data-testid="monaco-editor">
        <Editor
          height="100%"
          language="sql"
          value={generatedSql}
          theme="vs-dark"
          options={{
            readOnly: true,
            minimap: { enabled: false },
            fontSize: 12,
            lineNumbers: 'on',
            scrollBeyondLastLine: false,
            wordWrap: 'on',
            automaticLayout: true,
            padding: { top: 12 },
            renderLineHighlight: 'none',
            scrollbar: { useShadows: false },
          }}
        />
      </div>
    </div>
  );
}
```

```
============================================================
FILE: frontend\src\components\RightPanel\OrmOutputTab.jsx
============================================================

import React, { useState, useEffect, useRef } from 'react';
import Editor from '@monaco-editor/react';
import useDiagramStore from '../../store/diagramStore';
import { generateDjango, generatePrisma, generateSQLAlchemy } from
'../../utils/ormGenerator';
import { downloadFile } from '../../utils/exportUtils';
import toast from 'react-hot-toast';
import { Copy, Download } from 'lucide-react';

const FORMATS = [
  { id: 'django', label: 'Django', ext: '.py', lang: 'python' },
  { id: 'prisma', label: 'Prisma', ext: '.prisma', lang: 'typescript' },
  { id: 'sqlalchemy', label: 'SQLAlchemy', ext: '.py', lang: 'python' },
];

export default function OrmOutputTab() {
  const [format, setFormat] = useState('django');
  const [code, setCode] = useState('');
  const nodes = useDiagramStore((s) => s.nodes);
  const edges = useDiagramStore((s) => s.edges);
  const debounceRef = useRef(null);

  const generate = () => {
    const diagram = useDiagramStore.getState().getDiagramJSON();
```

```
    let result = '';
    if (format === 'django') result = generateDjango(diagram);
    else if (format === 'prisma') result = generatePrisma(diagram);
    else result = generateSQLAlchemy(diagram);
    setCode(result);
  };

  useEffect(() => {
    clearTimeout(debounceRef.current);
    debounceRef.current = setTimeout(generate, 600);
    return () => clearTimeout(debounceRef.current);
  }, [nodes, edges, format]); // eslint-disable-line

  const handleCopy = () => {
    navigator.clipboard.writeText(code).then(() => toast.success('Copied to
clipboard'));
  };

  const handleDownload = () => {
    const fmt = FORMATS.find((f) => f.id === format);
    const name = useDiagramStore.getState().diagramName || 'models';
    downloadFile(code, `${name.replace(/\s+/g, '_')}${fmt.ext}`, 'text/plain');
    toast.success(`Downloaded ${fmt.ext} file`);
  };

  const currentLang = FORMATS.find((f) => f.id === format)?.lang || 'python';

  return (
    <div className="orm-tab" data-testid="orm-tab">
```

```
      <div className="orm-toolbar">
        <div className="orm-format-btns">
          {FORMATS.map((f) => (
            <button
              key={f.id}
              className={`orm-fmt-btn ${format === f.id ? 'active' : ''}`}
              onClick={() => setFormat(f.id)}
              data-testid={`orm-format-${f.id}`}
            >
              {f.label}
            </button>
          ))}
        </div>
        <div className="orm-toolbar-right">
          <button className="sql-icon-btn" onClick={handleCopy} title="Copy" data-testid="copy-orm-btn">
            <Copy size={13} />
          </button>
          <button className="sql-icon-btn" onClick={handleDownload} title="Download" data-testid="download-orm-btn">
            <Download size={13} />
          </button>
        </div>
      </div>
      <div className="monaco-wrap" data-testid="orm-monaco-editor">
        <Editor
          height="100%"
          language={currentLang}
          value={code}
          theme="vs-dark"
```

```
          options={{
            readOnly: true,
            minimap: { enabled: false },
            fontSize: 12,
            lineNumbers: 'on',
            scrollBeyondLastLine: false,
            wordWrap: 'on',
            automaticLayout: true,
            padding: { top: 12 },
            renderLineHighlight: 'none',
            scrollbar: { useShadows: false },
          }}
        />
      </div>
    </div>
  );
}

============================================================
FILE: frontend\src\components\RightPanel\PropertiesTab.jsx
============================================================

import React from 'react';
import useDiagramStore from '../../store/diagramStore';
import useUIStore from '../../store/uiStore';
import { Trash2, GitMerge } from 'lucide-react';
import toast from 'react-hot-toast';

const COLORS = {
```

```
  blue: '#1e3a5f', green: '#14362a', purple: '#2d1b5e',
  orange: '#3b2200', red: '#3b1212', gray: '#1e293b',
};

const REL_TYPES = ['one-to-one', 'one-to-many', 'many-to-many', 'many-to-one'];
const FK_ACTIONS = ['RESTRICT', 'CASCADE', 'SET NULL', 'NO ACTION'];
const COL_TYPES = [
  'INT','BIGINT','VARCHAR(255)','VARCHAR(100)','TEXT','BOOLEAN',
  'DATE','DATETIME','TIMESTAMP','DECIMAL(10,2)','UUID','FLOAT','JSON',
];

function TableProps({ node }) {
  const { updateTable, addColumn, deleteColumn, updateColumn, deleteTable } =
useDiagramStore();
  const { clearSelection } = useUIStore();
  const table = node.data;

  return (
    <div className="props-panel" data-testid="table-props">
      <div className="props-section">
        <div className="props-row">
          <label>Table Name</label>
          <input
            value={table.name}
            onChange={(e) => updateTable(node.id, { name: e.target.value })}
            data-testid="props-table-name"
          />
        </div>
        <div className="props-row">
```

```
          <label>Color</label>
          <div className="color-swatches">
           {Object.entries(COLORS).map(([name, hex]) => (
             <button
               key={name}
               className={`swatch ${table.color === name ? 'active' : ''}`}
               style={{ background: hex }}
               onClick={() => updateTable(node.id, { color: name })}
               title={name}
               data-testid={`color-swatch-${name}`}
             />
           ))}
          </div>
        </div>
      </div>

      <div className="props-section">
        <div className="props-section-title">Columns
          <button className="mini-btn" onClick={() => addColumn(node.id)}>+
Add</button>
        </div>
        {(table.columns || []).map((col) => (
          <div key={col.id} className="props-col" data-testid={`props-col-${col.id}`}>
            <div className="props-row">
              <input
                placeholder="Name"
                value={col.name}
                onChange={(e) => updateColumn(node.id, col.id, { name: e.target.value })}
              />
```

```
            <select value={col.type} onChange={(e) => updateColumn(node.id, col.id, { type: e.target.value })}>
              {COL_TYPES.map((t) => <option key={t} value={t}>{t}</option>)}
            </select>
            <button className="del-btn" onClick={() => deleteColumn(node.id, col.id)}>
              <Trash2 size={11} />
            </button>
          </div>
          <div className="props-checks">
            <label><input type="checkbox" checked={col.primaryKey} onChange={(e) => updateColumn(node.id, col.id, { primaryKey: e.target.checked })} />PK</label>
            <label><input type="checkbox" checked={col.autoIncrement} onChange={(e) => updateColumn(node.id, col.id, { autoIncrement: e.target.checked })} />AI</label>
            <label><input type="checkbox" checked={!col.nullable} onChange={(e) => updateColumn(node.id, col.id, { nullable: !e.target.checked })} />NN</label>
            <label><input type="checkbox" checked={col.unique} onChange={(e) => updateColumn(node.id, col.id, { unique: e.target.checked })} />UQ</label>
          </div>
        </div>
      ))}
    </div>

    <div className="props-danger">
      <button className="danger-btn" onClick={() => { deleteTable(node.id); clearSelection(); }} data-testid="delete-table-btn">
        <Trash2 size={12} /> Delete Table
      </button>
    </div>
  </div>
);
```

```
}

function EdgeProps({ edge }) {
  const { updateRelationship, deleteRelationship, createJunctionTable } =
useDiagramStore();
  const { clearSelection } = useUIStore();
  const rel = edge.data || {};

  const handleTypeChange = (newType) => {
    updateRelationship(edge.id, { type: newType });
    if (newType === 'many-to-many') {
      toast(
        (t) => (
          <span style={{ display: 'flex', alignItems: 'center', gap: 8, fontSize: 12 }}>
            M:N detected
            <button
              onClick={() => {
                const name = createJunctionTable(edge.id);
                toast.dismiss(t.id);
                if (name) toast.success(`Junction table "${name}" created`);
              }}
              style={{ background: '#4f46e5', color: '#fff', border: 'none', borderRadius: 4,
padding: '3px 8px', cursor: 'pointer', fontSize: 11 }}
            >
              Auto-create junction table
            </button>
          </span>
        ),
        { duration: 8000, id: `mn-${edge.id}` }
      );
```

```
  }
};

return (
  <div className="props-panel" data-testid="edge-props">
    <div className="props-section">
      <div className="props-section-title">Relationship</div>
      <div className="props-row">
        <label>Type</label>
        <select value={rel.type || 'one-to-many'} onChange={(e) =>
handleTypeChange(e.target.value)} data-testid="rel-type-select">
          {REL_TYPES.map((t) => <option key={t} value={t}>{t}</option>)}
        </select>
      </div>
      <div className="props-row">
        <label>ON DELETE</label>
        <select value={rel.onDelete || 'RESTRICT'} onChange={(e) =>
updateRelationship(edge.id, { onDelete: e.target.value })} data-testid="rel-on-
delete">
          {FK_ACTIONS.map((a) => <option key={a} value={a}>{a}</option>)}
        </select>
      </div>
      <div className="props-row">
        <label>ON UPDATE</label>
        <select value={rel.onUpdate || 'RESTRICT'} onChange={(e) =>
updateRelationship(edge.id, { onUpdate: e.target.value })} data-testid="rel-on-
update">
          {FK_ACTIONS.map((a) => <option key={a} value={a}>{a}</option>)}
        </select>
      </div>
    </div>
```

```
      <div className="props-danger">
        <button className="danger-btn" onClick={() => { deleteRelationship(edge.id); clearSelection(); }} data-testid="delete-rel-btn">
          <Trash2 size={12} /> Delete Relationship
        </button>
      </div>
    </div>
  );
}

export default function PropertiesTab() {
  const { selectedNodeId, selectedEdgeId } = useUIStore();
  const nodes = useDiagramStore((s) => s.nodes);
  const edges = useDiagramStore((s) => s.edges);

  if (selectedNodeId) {
    const node = nodes.find((n) => n.id === selectedNodeId);
    if (node) return <TableProps node={node} />;
  }

  if (selectedEdgeId) {
    const edge = edges.find((e) => e.id === selectedEdgeId);
    if (edge) return <EdgeProps edge={edge} />;
  }

  return (
    <div className="props-empty" data-testid="properties-empty-state">
      <p>Select a table or relationship to edit its properties.</p>
    </div>
```

```
  );
}

============================================================
FILE: frontend\src\components\Modals\ImportSqlModal.jsx
============================================================

import React, { useState } from 'react';
import useDiagramStore from '../../store/diagramStore';
import useUIStore from '../../store/uiStore';
import { importSQL } from '../../api/apiClient';
import { autoLayout } from '../../utils/diagramLayout';
import toast from 'react-hot-toast';
import { X, Upload } from 'lucide-react';

const PLACEHOLDER = `-- Paste CREATE TABLE SQL here
CREATE TABLE users (
  id INT AUTO_INCREMENT NOT NULL,
  email VARCHAR(255) NOT NULL UNIQUE,
  name VARCHAR(255),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id)
);

CREATE TABLE posts (
  id INT AUTO_INCREMENT NOT NULL,
  user_id INT NOT NULL,
  title VARCHAR(255) NOT NULL,
  body TEXT,
```

```
  PRIMARY KEY (id),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);`;

export default function ImportSqlModal() {
  const [sql, setSql] = useState('');
  const [loading, setLoading] = useState(false);
  const { closeImportSqlModal } = useUIStore();
  const { loadDiagram } = useDiagramStore();
  const dialect = useDiagramStore((s) => s.dialect);

  const handleImport = async () => {
    if (!sql.trim()) return;
    setLoading(true);
    try {
      const diagram = await importSQL(sql, dialect);
      if (!diagram.tables?.length) {
        toast.error('No tables found in SQL');
        return;
      }
      const laid = autoLayout(diagram.tables, diagram.relationships || []);
      loadDiagram({ ...diagram, tables: laid, dialect });
      toast.success(`Imported ${diagram.tables.length} tables`);
      closeImportSqlModal();
    } catch {
      toast.error('Import failed — check SQL syntax');
    } finally {
      setLoading(false);
    }
```

```
  };


  return (
    <div className="modal-overlay" data-testid="import-sql-modal" onClick={closeImportSqlModal}>
      <div className="modal-box" onClick={(e) => e.stopPropagation()}>
        <div className="modal-header">
          <span>Import SQL</span>
          <button className="modal-close" onClick={closeImportSqlModal}><X size={16} /></button>
        </div>
        <div className="modal-body">
          <p className="modal-hint">Paste existing CREATE TABLE SQL statements. Foreign key relationships will be detected automatically.</p>
          <textarea
            className="sql-paste-area"
            value={sql}
            onChange={(e) => setSql(e.target.value)}
            placeholder={PLACEHOLDER}
            rows={14}
            data-testid="sql-import-textarea"
          />
        </div>
        <div className="modal-footer">
          <button className="modal-cancel" onClick={closeImportSqlModal}>Cancel</button>
          <button
            className="modal-confirm"
            onClick={handleImport}
            disabled={loading || !sql.trim()}
```

```
            data-testid="import-sql-confirm"
          >
            <Upload size={13} /> {loading ? 'Importing...' : 'Import to Canvas'}
          </button>
        </div>
      </div>
    </div>
  );
}

============================================================
FILE: frontend\src\components\Modals\SaveDiagramModal.jsx
============================================================

import React, { useState } from 'react';
import useDiagramStore from '../../store/diagramStore';
import useUIStore from '../../store/uiStore';
import { saveDiagram } from '../../utils/persistence';
import toast from 'react-hot-toast';
import { X, Save } from 'lucide-react';

export default function SaveDiagramModal() {
  const diagramName = useDiagramStore((s) => s.diagramName);
  const setDiagramName = useDiagramStore((s) => s.setDiagramName);
  const getDiagramJSON = useDiagramStore((s) => s.getDiagramJSON);
  const { closeSaveDiagramModal } = useUIStore();
  const [name, setName] = useState(diagramName);
  const [saving, setSaving] = useState(false);
```

```
  const handleSave = async () => {
    if (!name.trim()) return;
    setSaving(true);
    try {
      setDiagramName(name.trim());
      const diagram = getDiagramJSON();
      await saveDiagram({ ...diagram, name: name.trim() });
      window.dispatchEvent(new Event('diagram-saved'));
      toast.success(`Saved "${name.trim()}"`);
      closeSaveDiagramModal();
    } catch {
      toast.error('Save failed');
    } finally {
      setSaving(false);
    }
  };

  return (
    <div className="modal-overlay" data-testid="save-diagram-modal"
onClick={closeSaveDiagramModal}>
      <div className="modal-box small" onClick={(e) => e.stopPropagation()}>
        <div className="modal-header">
          <span>Save Diagram</span>
          <button className="modal-close" onClick={closeSaveDiagramModal}><X
size={16} /></button>
        </div>
        <div className="modal-body">
          <label className="modal-label">Diagram Name</label>
          <input
            className="modal-input"
```

```
            value={name}
            onChange={(e) => setName(e.target.value)}
            onKeyDown={(e) => e.key === 'Enter' && handleSave()}
            placeholder="My Schema"
            autoFocus
            data-testid="save-diagram-name-input"
          />
        </div>
        <div className="modal-footer">
          <button className="modal-cancel"
onClick={closeSaveDiagramModal}>Cancel</button>
          <button
            className="modal-confirm"
            onClick={handleSave}
            disabled={saving || !name.trim()}
            data-testid="save-diagram-confirm"
          >
            <Save size={13} /> {saving ? 'Saving...' : 'Save'}
          </button>
        </div>
      </div>
    </div>
  );
}
```

==========

FILE: frontend\src\components\Modals\ShareModal.jsx

==========

```
import React, { useEffect, useState } from 'react';
import useDiagramStore from '../../store/diagramStore';
import useUIStore from '../../store/uiStore';
import { shareDiagram } from '../../api/apiClient';
import toast from 'react-hot-toast';
import { X, Copy, Share2, ExternalLink, CheckCircle2, Loader2 } from 'lucide-react';

export default function ShareModal() {
  const getDiagramJSON = useDiagramStore((s) => s.getDiagramJSON);
  const nodes = useDiagramStore((s) => s.nodes);
  const { closeShareModal } = useUIStore();
  const [creating, setCreating] = useState(false);
  const [shareId, setShareId] = useState(null);
  const [copied, setCopied] = useState(false);
  const [err, setErr] = useState('');

  const shareUrl = shareId ? `${window.location.origin}/share/${shareId}` : '';

  const handleCreate = async () => {
    setCreating(true);
    setErr('');
    try {
      const diagram = getDiagramJSON();
      const id = await shareDiagram(diagram);
      setShareId(id);
    } catch (e) {
      setErr(e?.response?.data?.detail || 'Failed to create share link');
    } finally {
      setCreating(false);
```

```
  }
};

// Auto-create on open if diagram has tables
useEffect(() => {
  if (nodes.length > 0 && !shareId && !creating) {
    handleCreate();
  }
  // eslint-disable-next-line
}, []);

const handleCopy = async () => {
  try {
    await navigator.clipboard.writeText(shareUrl);
    setCopied(true);
    toast.success('Link copied');
    setTimeout(() => setCopied(false), 2000);
  } catch {
    toast.error('Copy failed');
  }
};

const isEmpty = nodes.length === 0;

return (
  <div className="modal-overlay" data-testid="share-modal" onClick={closeShareModal}>
    <div className="modal-box" style={{ width: 520 }} onClick={(e) => e.stopPropagation()}>
      <div className="modal-header">
```

```
        <span><Share2 size={14} style={{ marginRight: 6, verticalAlign: '-2px' }}
/>Share Diagram</span>
        <button className="modal-close" onClick={closeShareModal}><X size={16}
/></button>
      </div>
      <div className="modal-body">
        {isEmpty ? (
          <div style={{ padding: '16px 0', color: '#94a3b8', fontSize: 13 }}>
            Add at least one table to your canvas before sharing.
          </div>
        ) : creating ? (
          <div style={{ display: 'flex', alignItems: 'center', gap: 10, padding: '20px 0',
color: '#94a3b8', fontSize: 13 }}>
            <Loader2 size={16} className="spin" /> Creating public link…
          </div>
        ) : err ? (
          <div style={{ color: '#f87171', fontSize: 13, padding: '12px 0' }} data-
testid="share-error">
            {err}
            <button
              className="modal-confirm"
              onClick={handleCreate}
              style={{ marginLeft: 12 }}
              data-testid="share-retry-btn"
            >Retry</button>
          </div>
        ) : shareId ? (
          <>
            <label className="modal-label">Public read-only link</label>
            <div style={{ display: 'flex', gap: 8 }}>
```

```
            <input
              className="modal-input"
              value={shareUrl}
              readOnly
              onFocus={(e) => e.target.select()}
              data-testid="share-url-input"
              style={{ flex: 1, fontFamily: 'Menlo, monospace', fontSize: 12 }}
            />
            <button
              className="modal-confirm"
              onClick={handleCopy}
              data-testid="share-copy-btn"
              style={{ padding: '0 12px' }}
            >
              {copied ? <CheckCircle2 size={14} /> : <Copy size={14} />}
              {copied ? 'Copied' : 'Copy'}
            </button>
          </div>
          <div style={{ marginTop: 14, padding: 12, background:
'rgba(99,102,241,0.08)', border: '1px solid rgba(99,102,241,0.25)', borderRadius: 6,
fontSize: 12, color: '#c7d2fe', lineHeight: 1.5 }}>
            Anyone with this link can view the ERD, generated SQL (MySQL &
PostgreSQL), and ORM models.
            They can also fork the diagram to edit their own copy. Edits in this editor
will <b>not</b> update the shared snapshot — generate a new link to share
updates.
          </div>
          <a
            href={shareUrl}
            target="_blank"
            rel="noopener noreferrer"
```

```
              className="share-preview-link"
              data-testid="share-open-btn"
              style={{ display: 'inline-flex', alignItems: 'center', gap: 6, marginTop: 14, fontSize: 12, color: '#818cf8', textDecoration: 'none' }}
            >
              Open preview <ExternalLink size={12} />
            </a>
          </>
        ) : (
          <button
            className="modal-confirm"
            onClick={handleCreate}
            data-testid="share-create-btn"
          >
            Create public link
          </button>
        )}
      </div>
      <div className="modal-footer">
        <button className="modal-cancel" onClick={closeShareModal} data-testid="share-close-btn">Close</button>
      </div>
    </div>
  </div>
  );
}
```

==========

FILE: frontend\src\components\Modals\WelcomeScreen.jsx

==========

```
import React, { useState } from 'react';
import useDiagramStore from '../../store/diagramStore';
import useUIStore from '../../store/uiStore';
import { EXAMPLE_SCHEMAS } from '../../utils/exampleSchemas';
import { Database, Sparkles, BookOpen } from 'lucide-react';
import toast from 'react-hot-toast';

export default function WelcomeScreen({ onDismiss }) {
  const [showExamples, setShowExamples] = useState(false);
  const { loadDiagram, addTable } = useDiagramStore();
  const { setActiveTab } = useUIStore();

  const handleScratch = () => {
    addTable({ x: 300, y: 200 });
    onDismiss();
    toast.success('Add tables by double-clicking the canvas');
  };

  const handleExample = (key) => {
    const schema = EXAMPLE_SCHEMAS[key];
    if (schema) {
      loadDiagram(schema);
      toast.success(`Loaded "${schema.name}" example schema`);
      onDismiss();
    }
  };

  const handleAI = () => {
```

```
    setActiveTab('ai');
    onDismiss();
    toast.success('Describe your schema in the AI Assistant tab');
  };

  const examples = [
    { key: 'ecommerce', label: 'E-Commerce Store', tables: 6 },
    { key: 'blog', label: 'Blog Platform', tables: 5 },
    { key: 'university', label: 'University System', tables: 5 },
    { key: 'hospital', label: 'Hospital Management', tables: 6 },
  ];

  return (
    <div className="welcome-overlay" data-testid="welcome-screen">
      <div className="welcome-box">
        <div className="welcome-logo">
          <Database size={36} style={{ color: '#6366f1' }} />
        </div>
        <h1 className="welcome-title">SketchSQL</h1>
        <p className="welcome-tagline">Draw your database. Get your SQL. Ask AI anything about it.</p>

        {!showExamples ? (
          <div className="welcome-cards">
            <button className="welcome-card" onClick={handleScratch} data-testid="start-scratch-btn">
              <Database size={22} style={{ color: '#818cf8' }} />
              <strong>Start from scratch</strong>
              <span>Begin with a blank canvas</span>
            </button>
```

```
            <button className="welcome-card" onClick={() => setShowExamples(true)}
data-testid="try-example-btn">
              <BookOpen size={22} style={{ color: '#4ade80' }} />
              <strong>Try an example</strong>
              <span>Load a ready-made schema</span>
            </button>
            <button className="welcome-card" onClick={handleAI} data-
testid="describe-ai-btn">
              <Sparkles size={22} style={{ color: '#fbbf24' }} />
              <strong>Describe with AI</strong>
              <span>Generate schema from text</span>
            </button>
          </div>
        ) : (
          <div className="example-list">
            <p className="example-list-title">Choose an example schema:</p>
            {examples.map((ex) => (
              <button key={ex.key} className="example-item" onClick={() =>
handleExample(ex.key)} data-testid={`example-${ex.key}`}>
                <span>{ex.label}</span>
                <span className="example-tables">{ex.tables} tables</span>
              </button>
            ))}
            <button className="back-link" onClick={() =>
setShowExamples(false)}>Back</button>
          </div>
        )}
      </div>
    </div>
  );
```

```
}
```

```
============================================================
FILE: frontend\src\lib\utils.js
============================================================
```

```
import { clsx } from "clsx";
import { twMerge } from "tailwind-merge"

export function cn(...inputs) {
  return twMerge(clsx(inputs));
}
```